THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

循环神经网络与 Transformer

讲师：Hao Wang
AI Thrust，信息枢纽

致谢：部分幻灯片修改自斯坦福的 cs231n

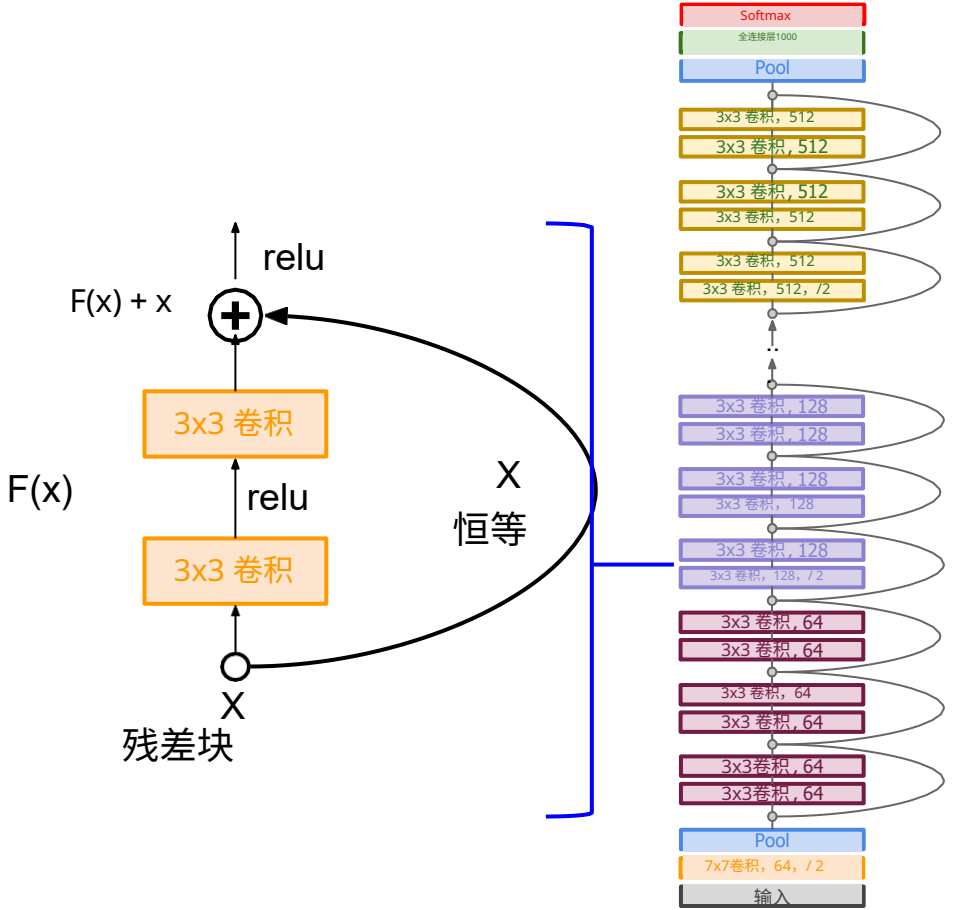
回顾上周



ResNet

[He et al., 2015]

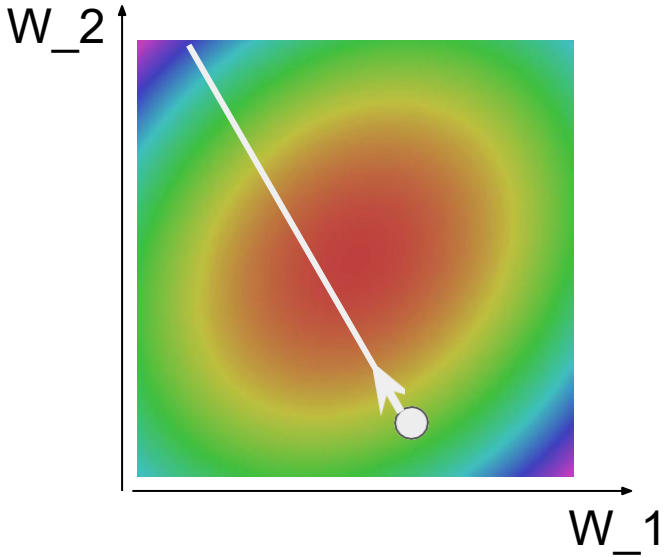
完整的 ResNet 架构： - 堆叠残差块 - 每个残差块都有两个 3x3 卷积层



优化（前文已介绍）

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

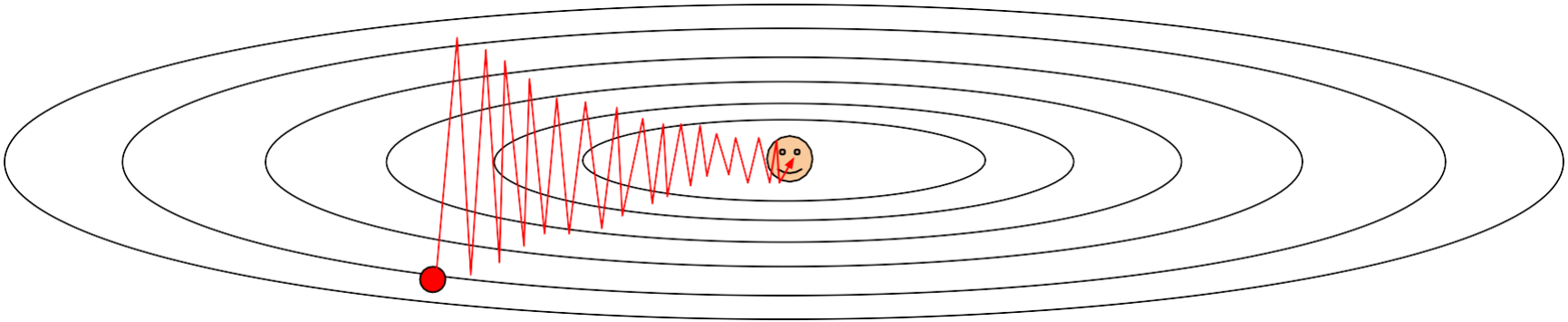




优化：问题 #1（使用 SGD）

如果损失在一个方向上变化很快，而在另一个方向上变化很慢，会怎样？ 梯度下降会怎么做？ 沿平缓维度进展非常缓慢，沿陡峭方向抖动

n

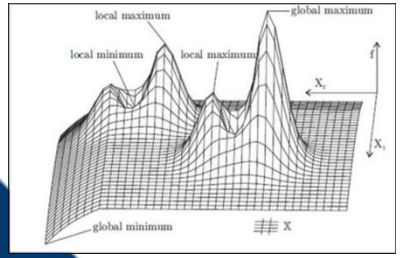


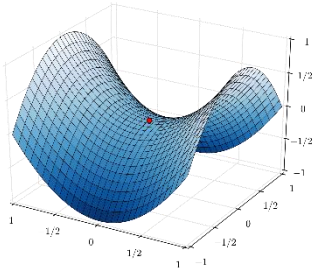
损失函数具有较高的 **条件数**：Hessian 矩阵的最大奇异值与最小奇异值之比很大

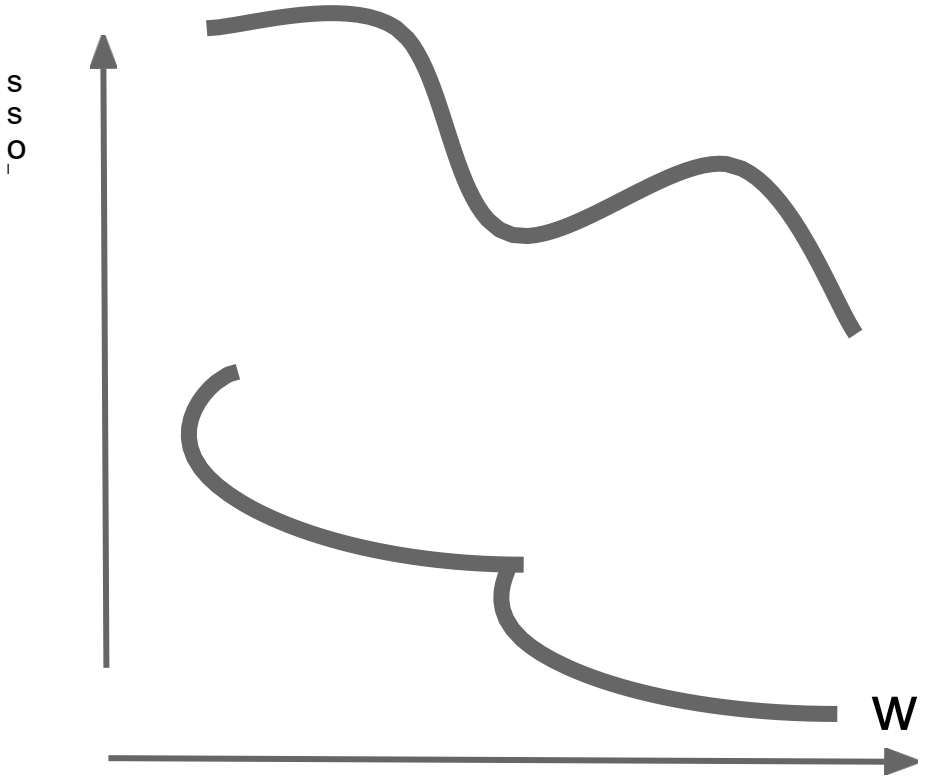


优化：问题 #2（使用 SGD）

如果损失函数存
在**局部极小值**
或**鞍点**，会怎样?





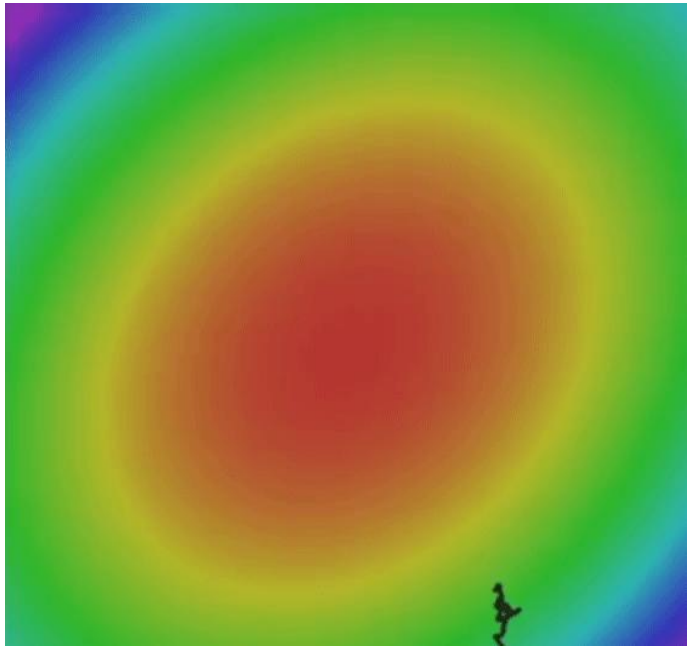


优化：使用 SGD 的问题 #3

我们的梯度来自小批量，因此它们可能含有噪声！

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$



SGD + 动量

继续像前几次迭代那样沿大致方向移动 SGD SGD+Momentum

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

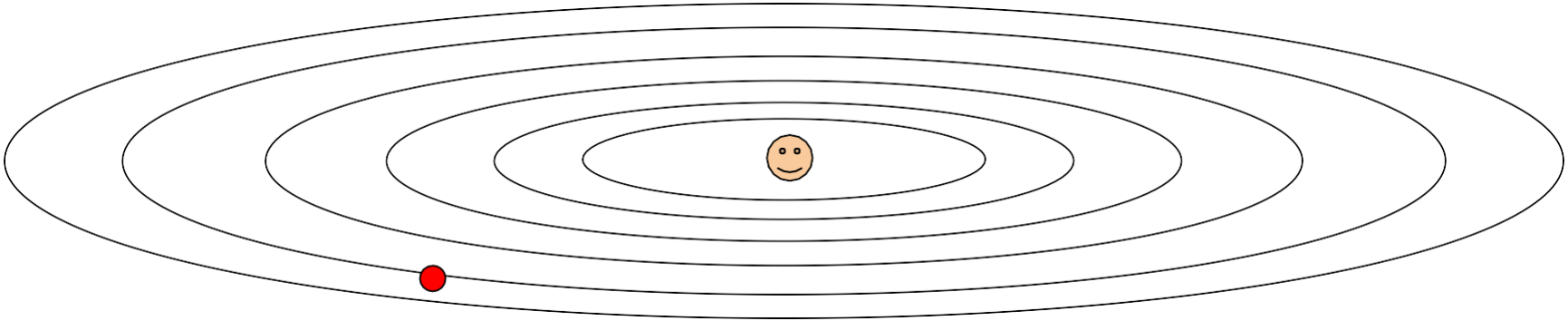
- - 将“速度”构建为梯度的滑动平均；Rho 提供“摩擦”；通常 rho=0.9 或 0.99



Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



问：AdaGrad 会发生什么？

沿“陡峭”方向的推进受到抑制；沿“平坦”方向的推进则被加速

如果 dx 很大，np.sqrt(grad_squared) 将迅速增大，分母将有助于降低学习率

RMSProp

步长随时间推移衰减至零

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Tieleman and Hinton, 2012

Adam （几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```



Kingma 和 Ba，“Adam：一种随机优化方法”，ICLR 2015

Adam （全称）

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

动量

偏差修正

AdaGrad / RMSProp

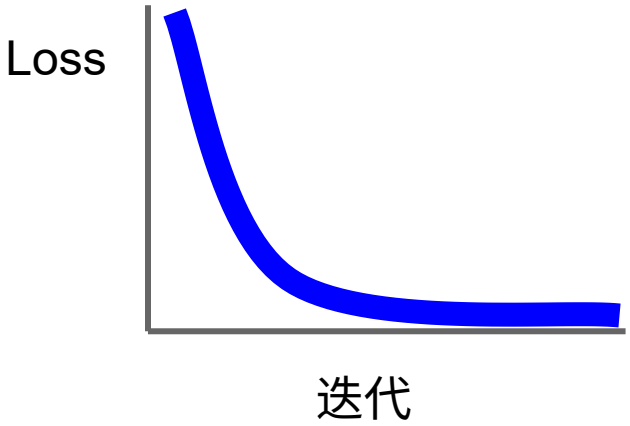
针对一阶和二阶矩估计从零开始这一事实进行偏差校正

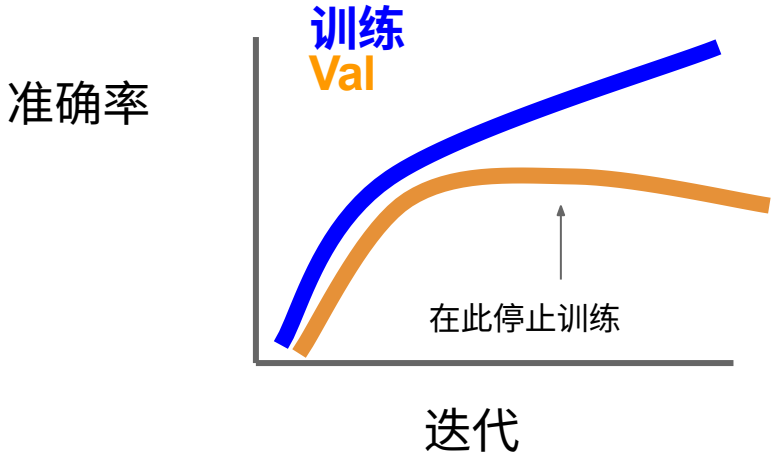
Adam 使用 beta1 = 0.9，beta2 = 0.999，以及 学习_率 = 1e-3 或 5e-4 是许多模型的绝佳起点！



Kingma 和 Ba，“Adam：一种随机优化方法”，ICLR 2015

早停





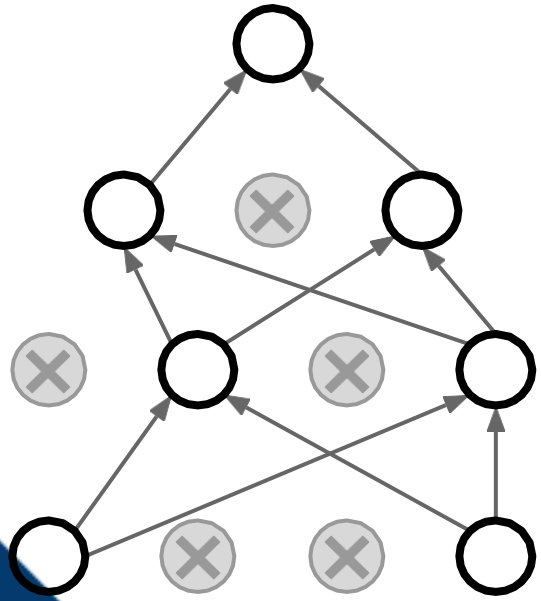
当验证集上的准确率下降时，停止训练模型

或者训练很长时间，但始终记录在验证集上表现最好的模型快照



正则化：Dropout

这怎么可能会是个好主意？



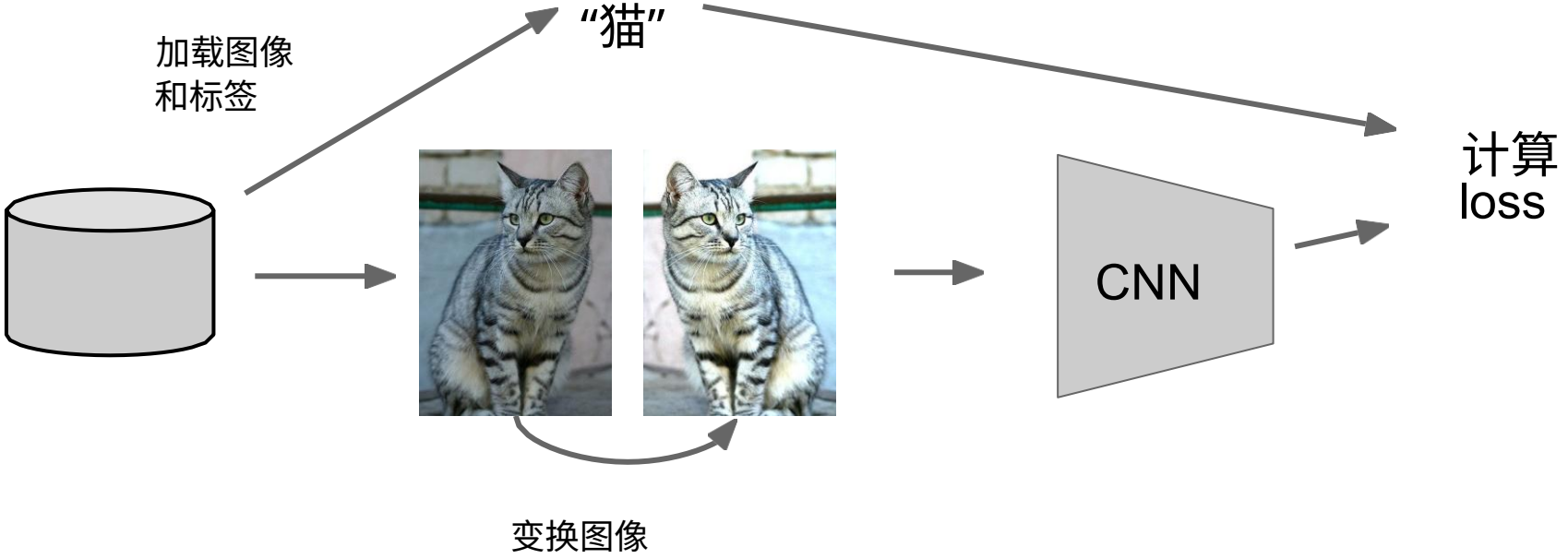
另一种解释：

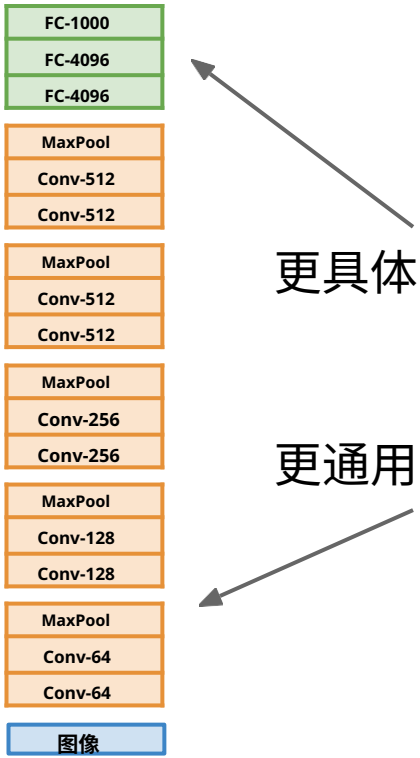
Dropout 是在训练一个由大量模型构成的大型**集成**（这些模型共享参数）。

每个二值掩码都是一个模型

一个具有 4096 个单元的 FC 层有 $2^{4096} \sim 10^{1233}$ 种可能的掩码！而宇宙中只有 $\sim 10^{82}$ 个原子.....

正则化：数据增强





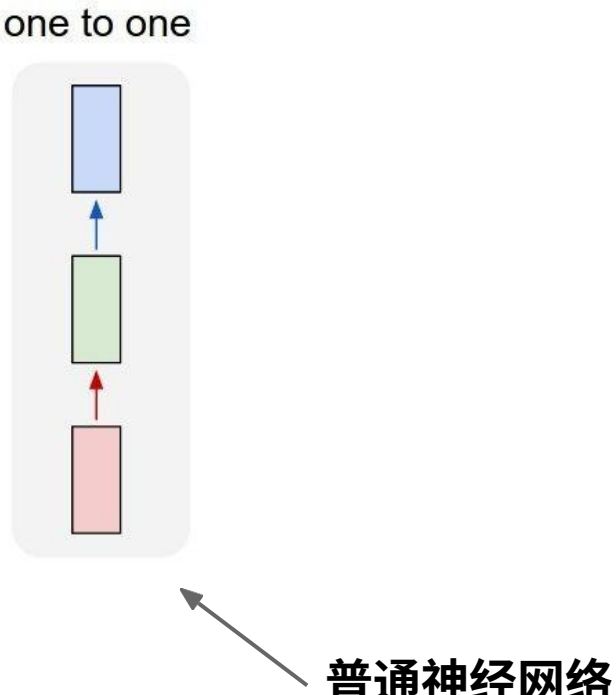
	非常相似数据集	**非常不同数据集**
数据非常少	使用线性分类器在顶层	你成功了 麻烦……试试线性分类器 来自不同的阶段
相当多的data	微调一个少量层	微调更多层

大纲

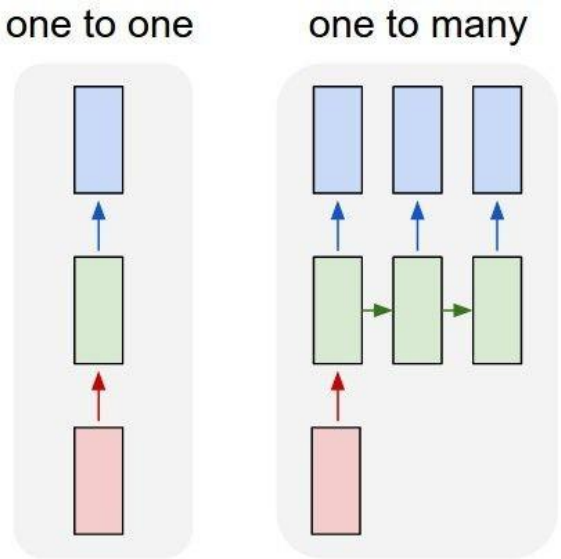
- **RNN 简介**- **基于 RNN 的注意力**- **通用注意力层**- 自注意力 - 位置编码 - 掩码注意力 - 多头注意力 - **Transformer**



“普通”神经网络

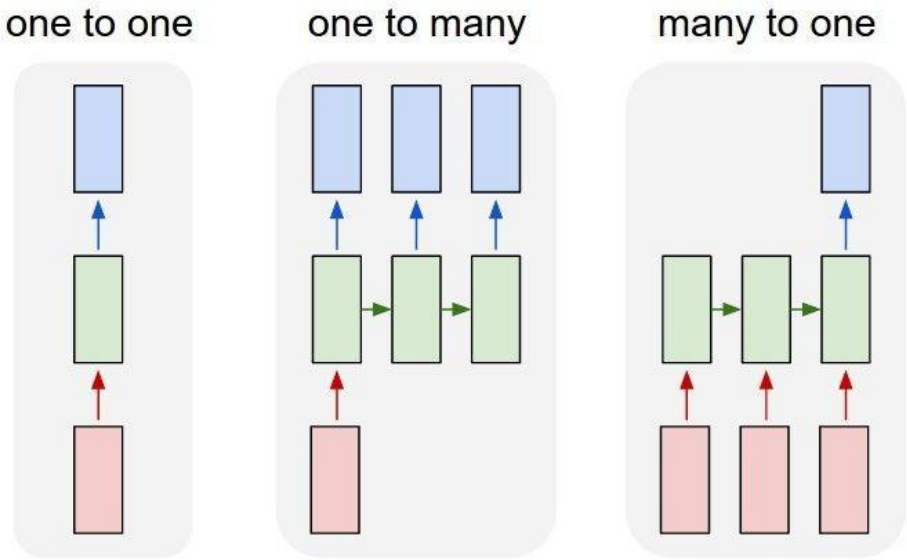


循环神经网络：处理序列



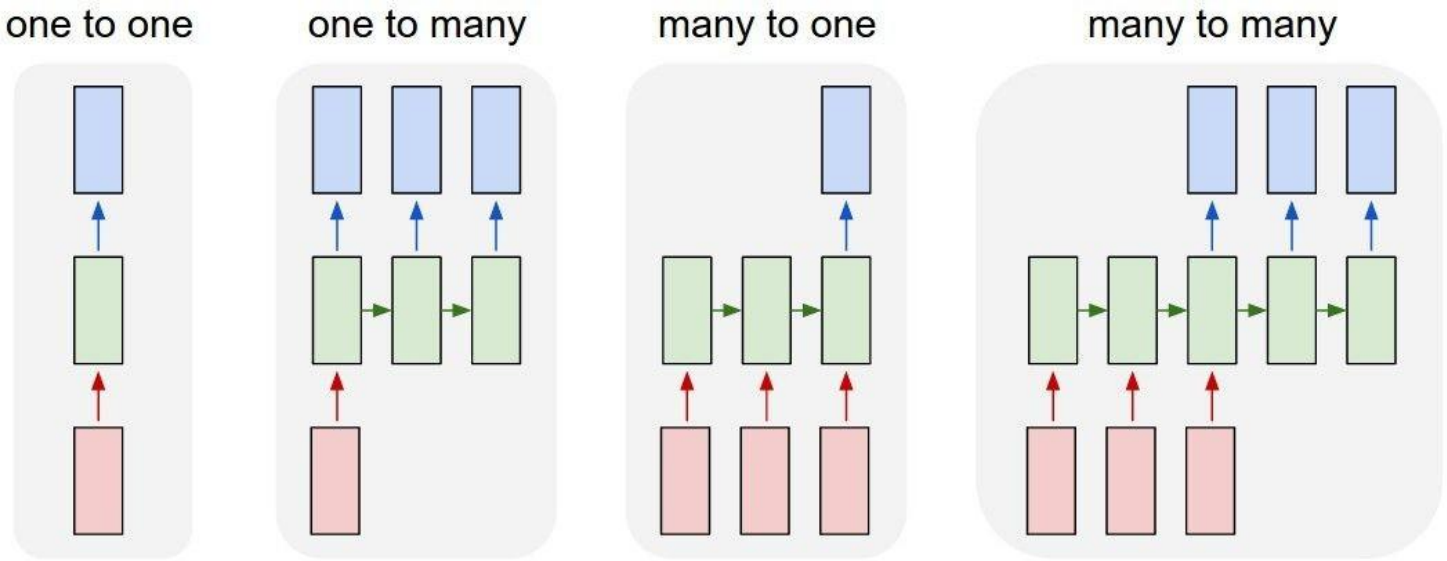
例如 **图像描述**
图像 -> 单词序列

循环神经网络： 处理序列



例如 **动作预测** 视频帧序列 -> 动作类别

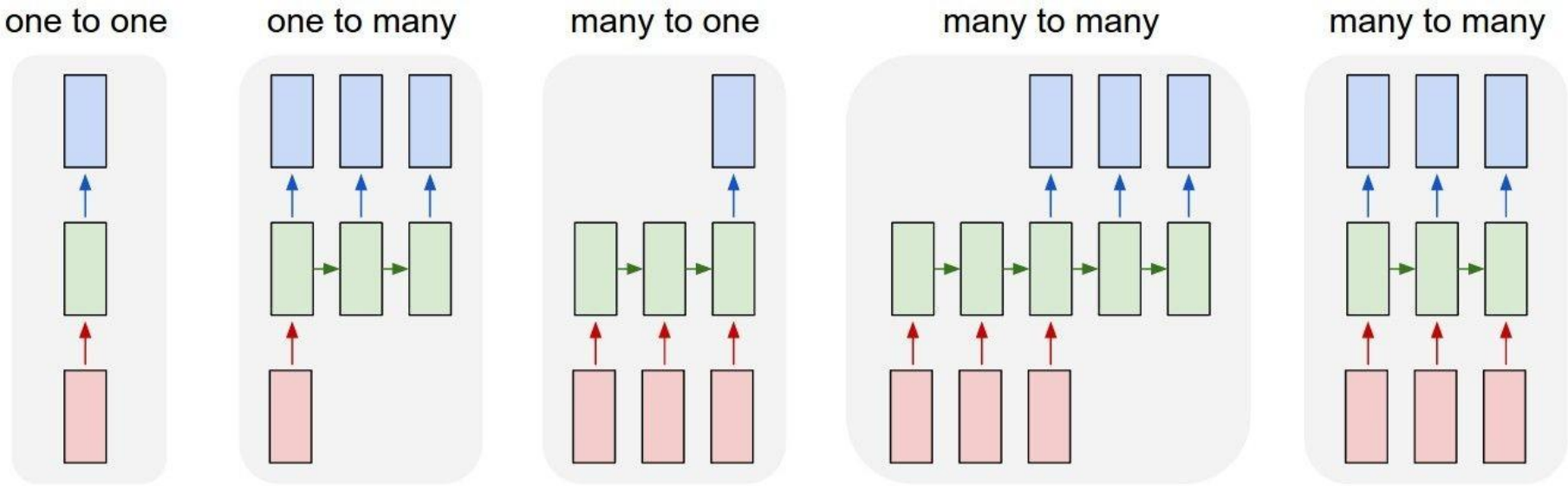
循环神经网络：处理序列



例如 **视频字幕生成** 视频帧序列 ->字幕



循环神经网络：处理序列



例如 **帧级别的视频分类**

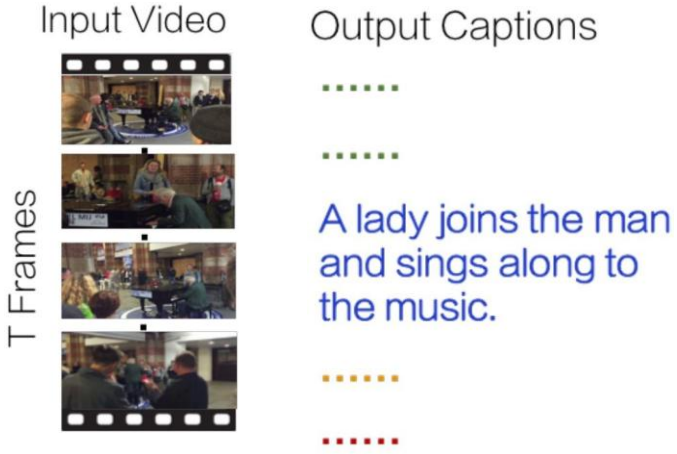
为什么现有 ConvNets 不足?

可变序列长度的输入和输出!

示例任务：视频字幕生成

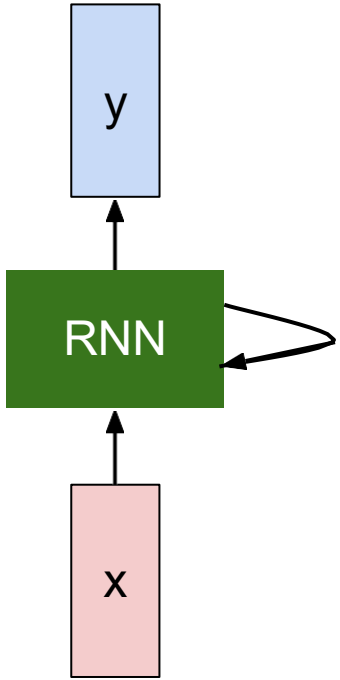
输入视频可以具有可变数量的帧

输出字幕可以具有可变长度。



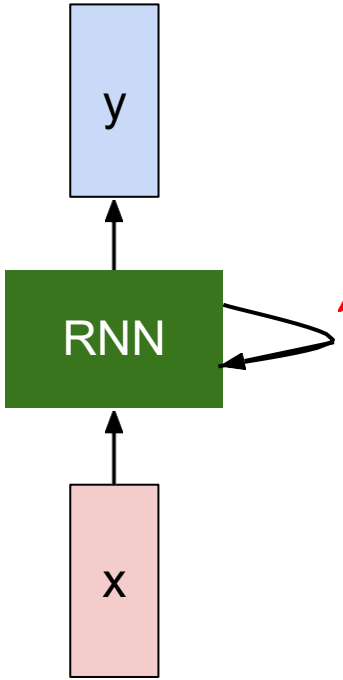
Krishna, Hata, Ren, Fei-Fei, Niebles. 视频中的密集事件描述. ICCV 2019

循环神经网络





循环神经网络



关键思想：RNNs 具有一个“内部状态”，该状态会随着序列被处理而更新



RNN 隐藏 状态更新

我们可以通过在每个时间步应用一个 **递推公式** 来处理向量序列 **x**：

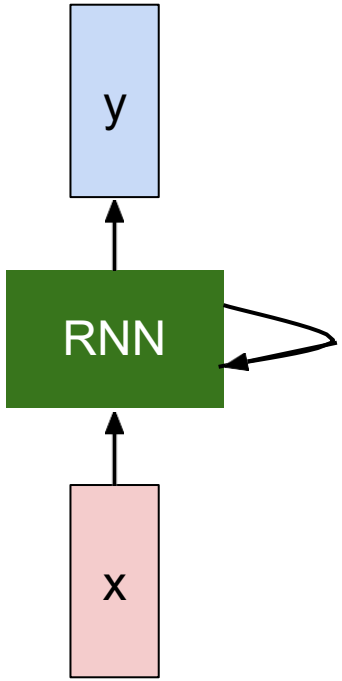
$$h_t = f_W(h_{t-1}, x_t)$$

新状态

旧状态

输入向量在某个时间步

某个函数具有参数 W



RNN 输出生成

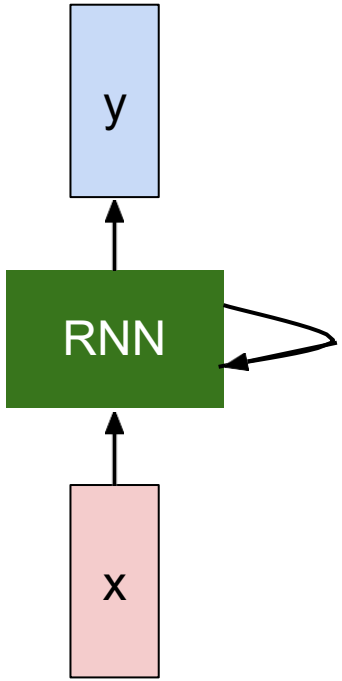
我们可以在每个时间步通过应用 **递推公式** 来处理向量序列 **x**：

$$y_t = f_{W_{hy}}(h_t)$$

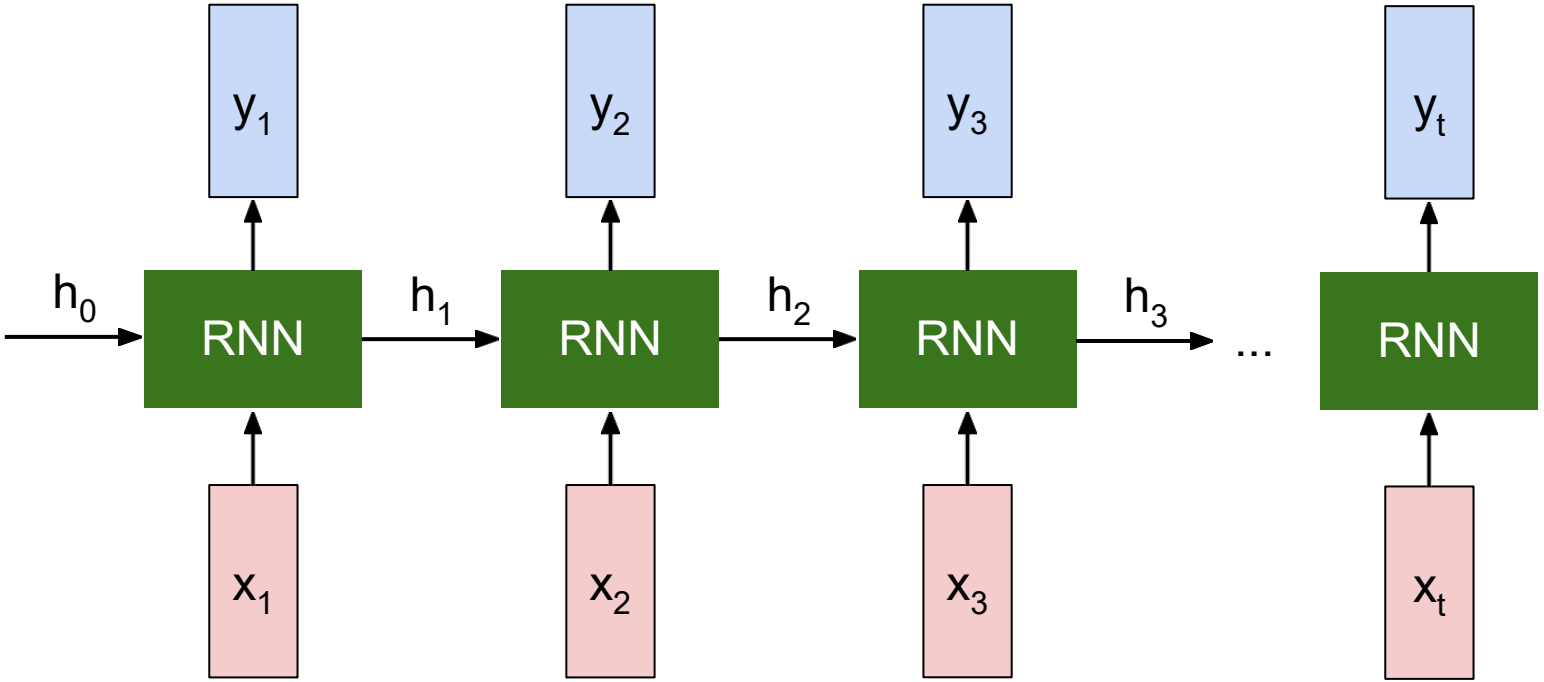
输出

新状态

另一个带有参数 W_o 的函数



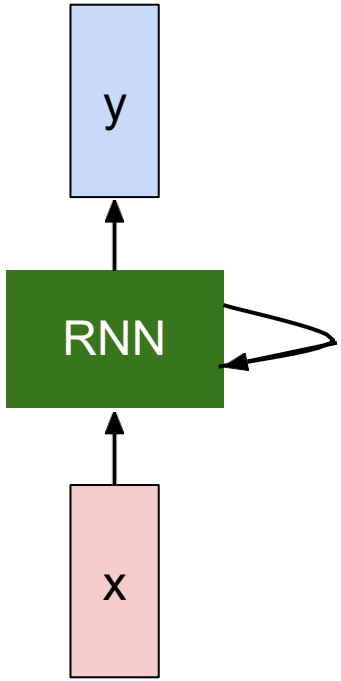
循环神经网络



循环神经 网络

我们可以处理一个向量序列 **x** ， 方式是在每个时间步
应用一个 **递推公式** ：

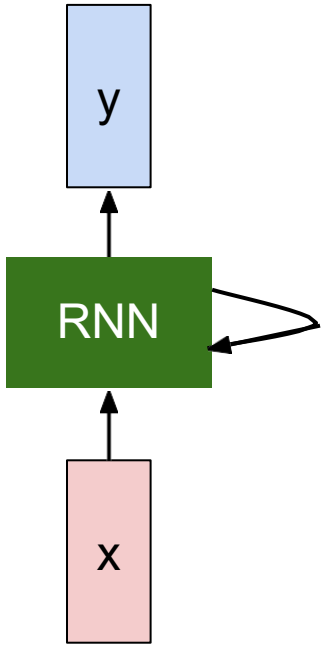
$$h_t = f_W(h_{t-1}, x_t)$$



注意：在每个时间步都使用相同的函数和相同的参数集。

（简单）循环神经 网络

状态由一个单独的“隐藏”向量**h**组成：



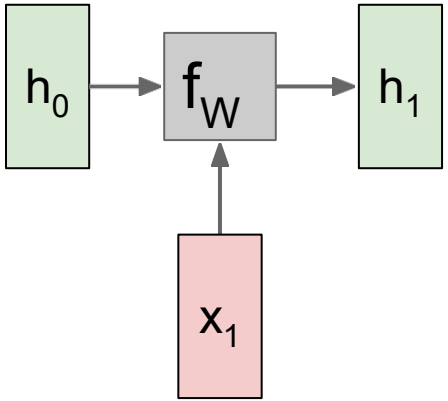
$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

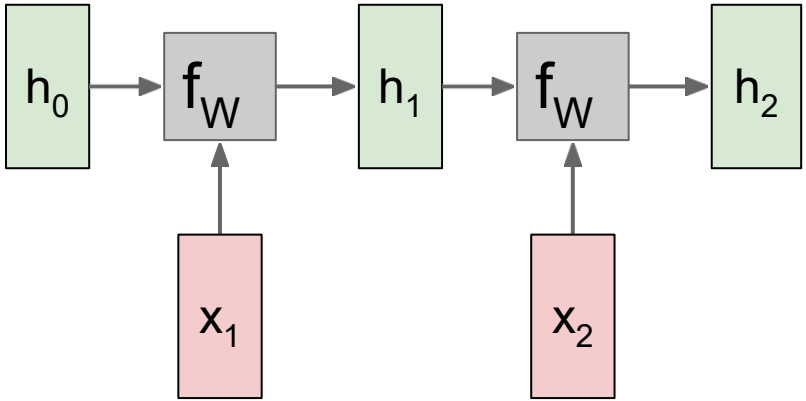
有时称为 “普通 RNN”，或称以 Jeffrey Elman 教授命名的“Elman RNN”

RNN：计算图



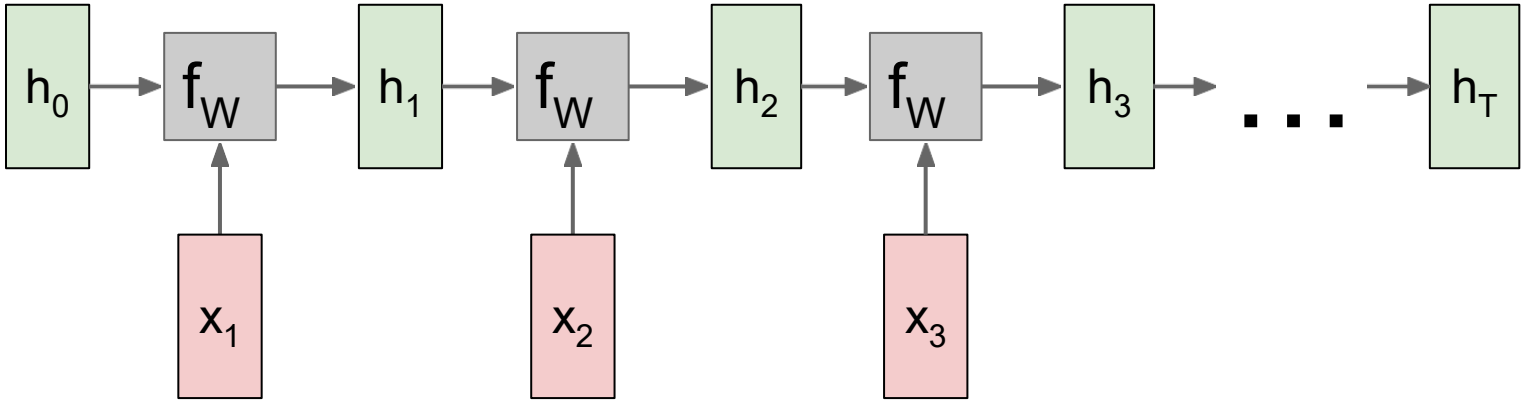


RNN：计算图





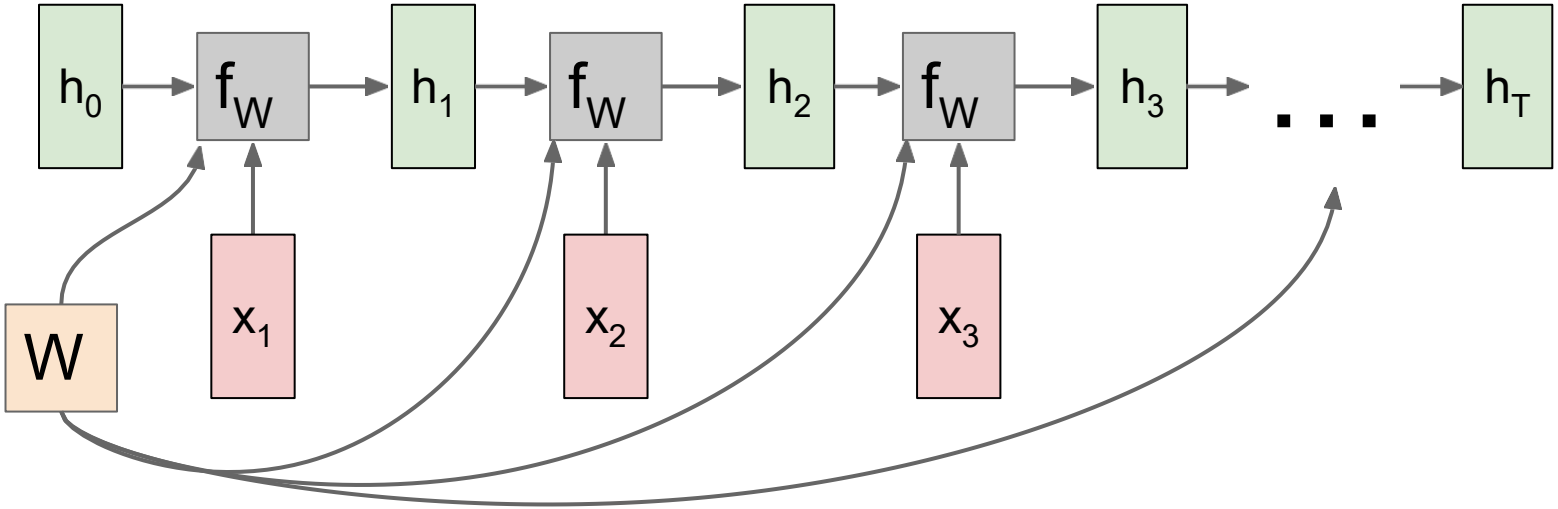
RNN：计算图



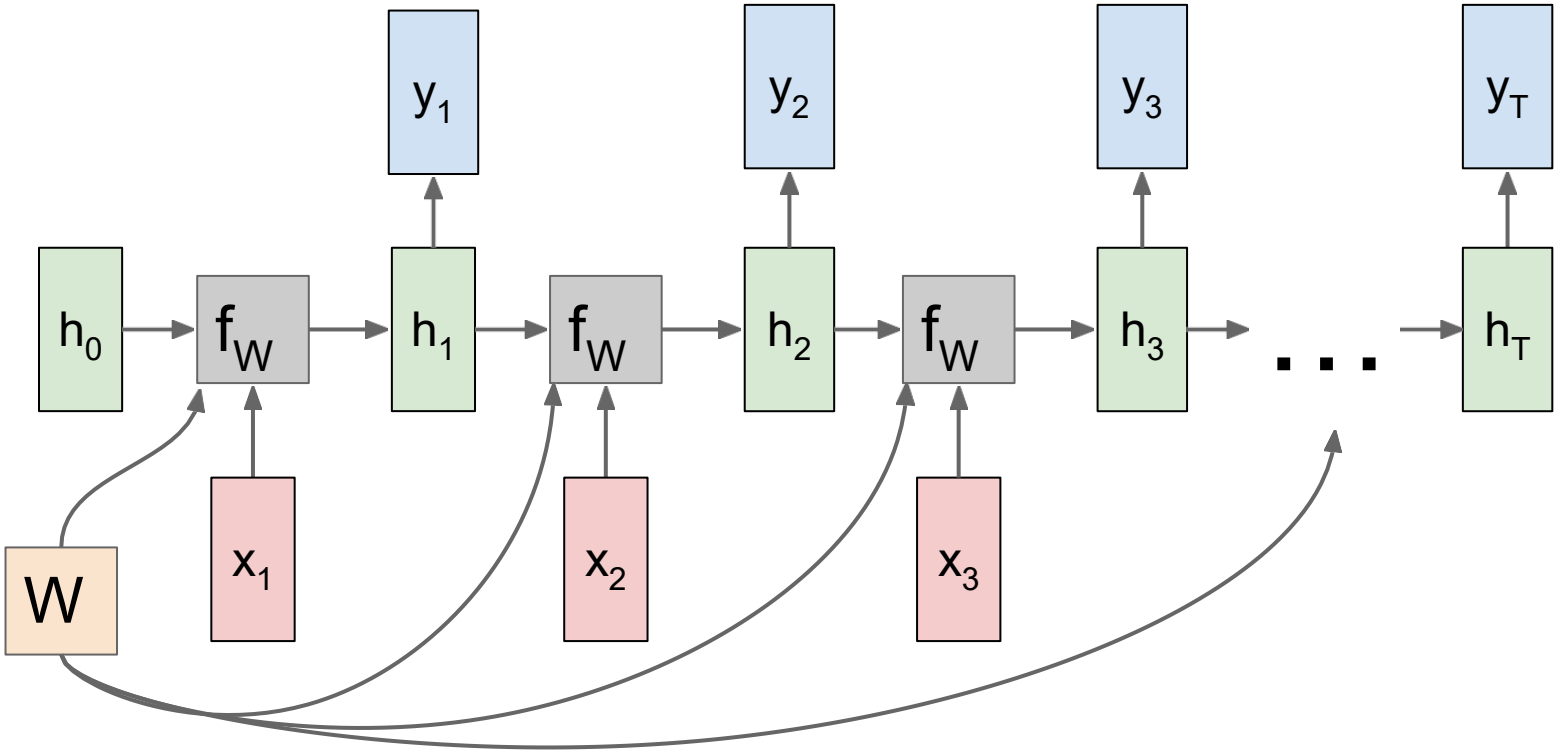


RNN：计算图

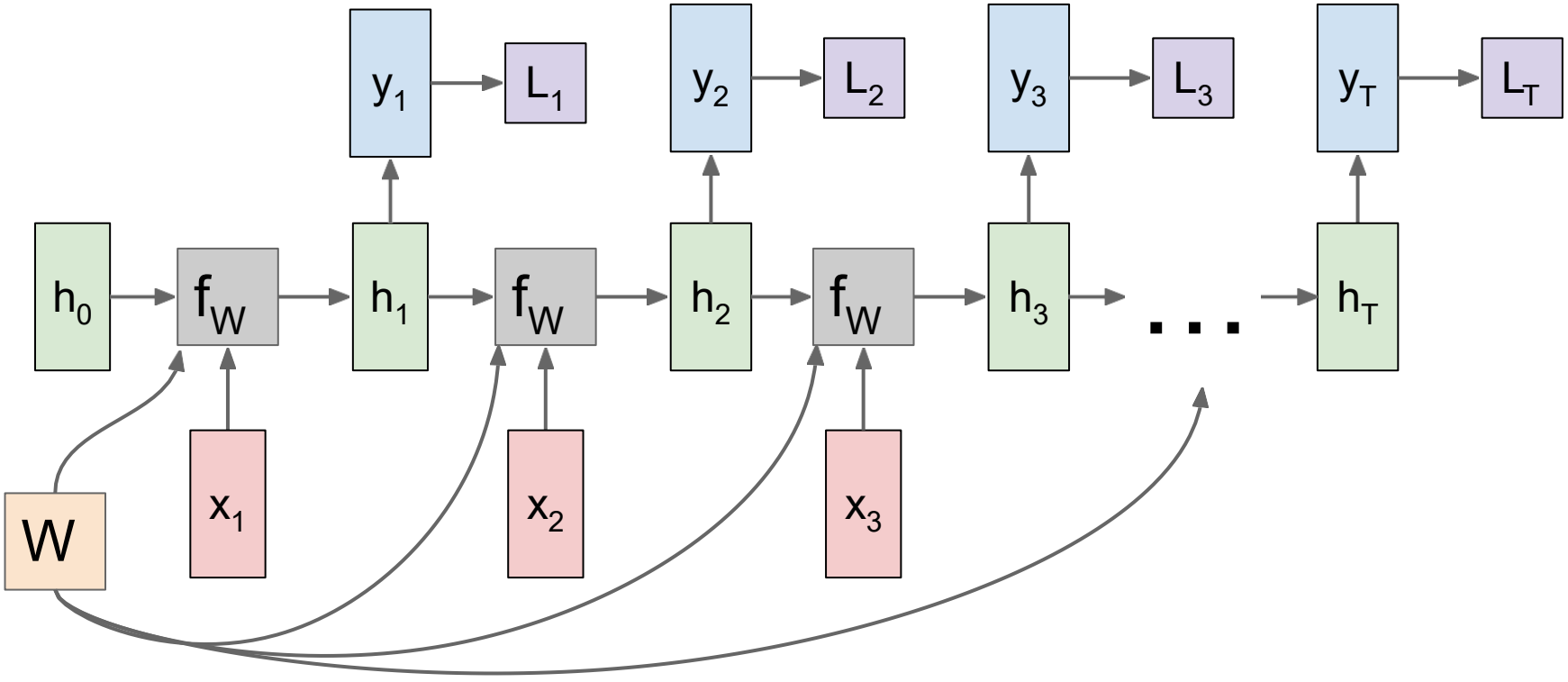
在每个时间步复用相同的权重矩阵



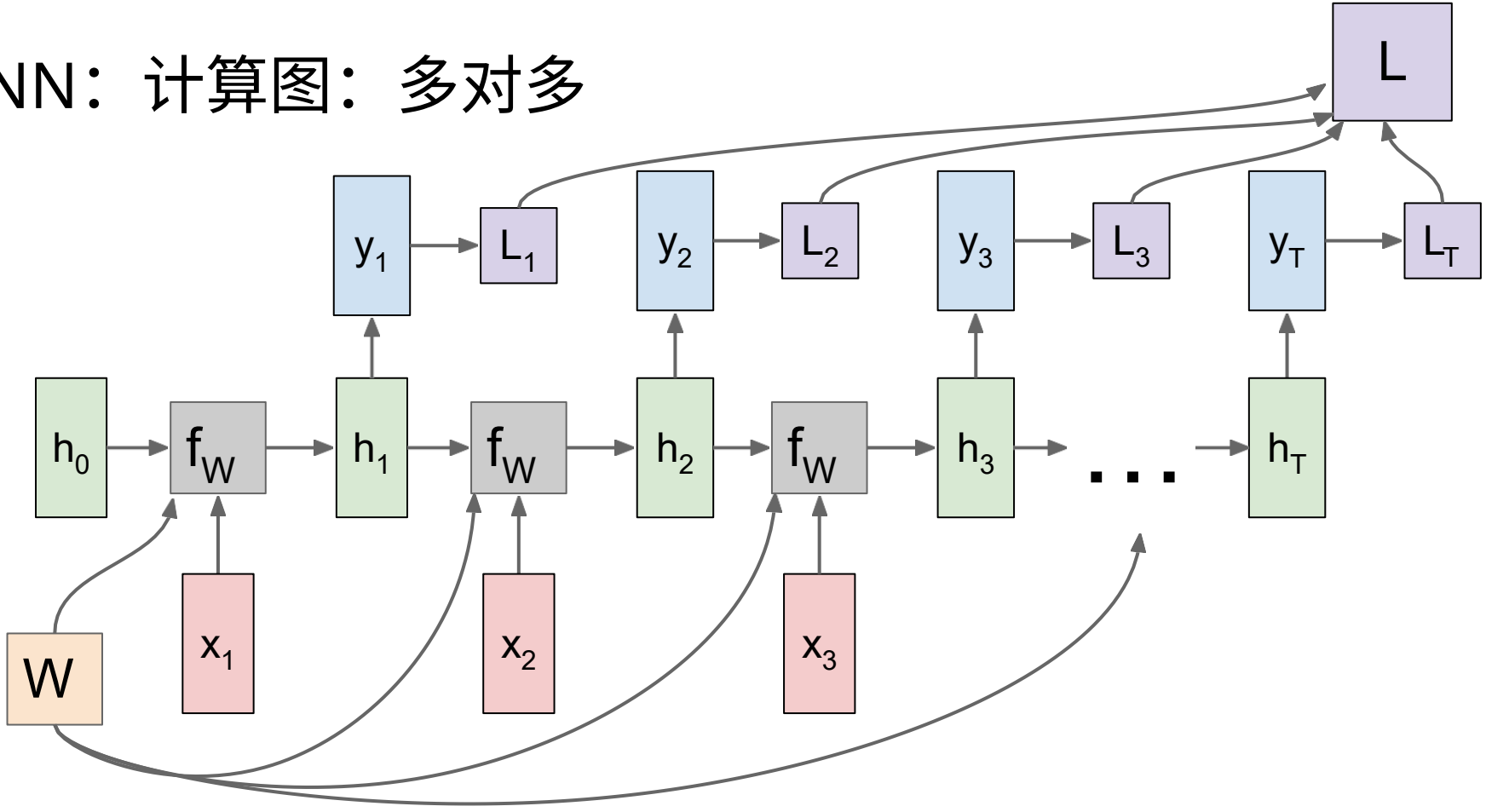
RNN：计算图：多对多



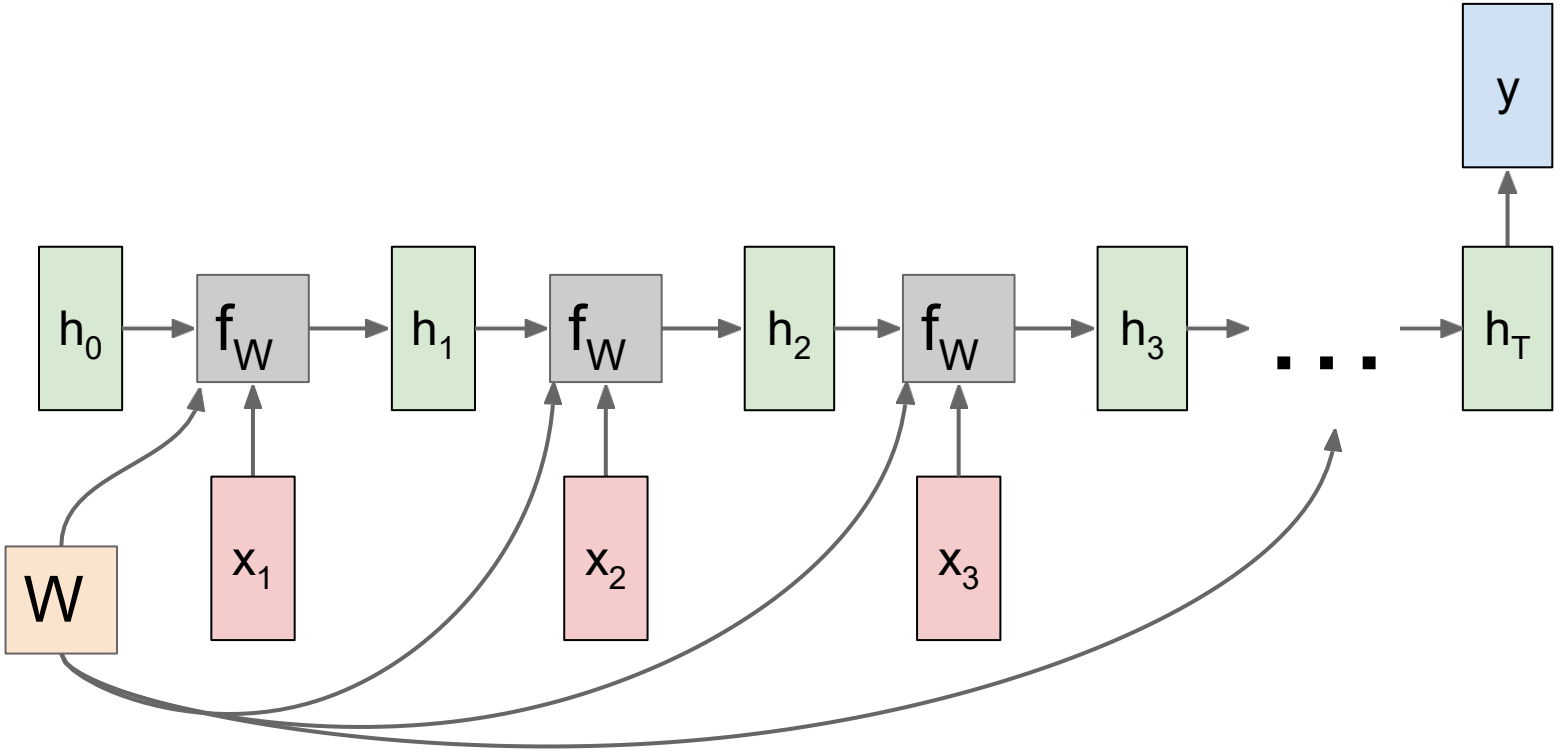
RNN：计算图：多对多



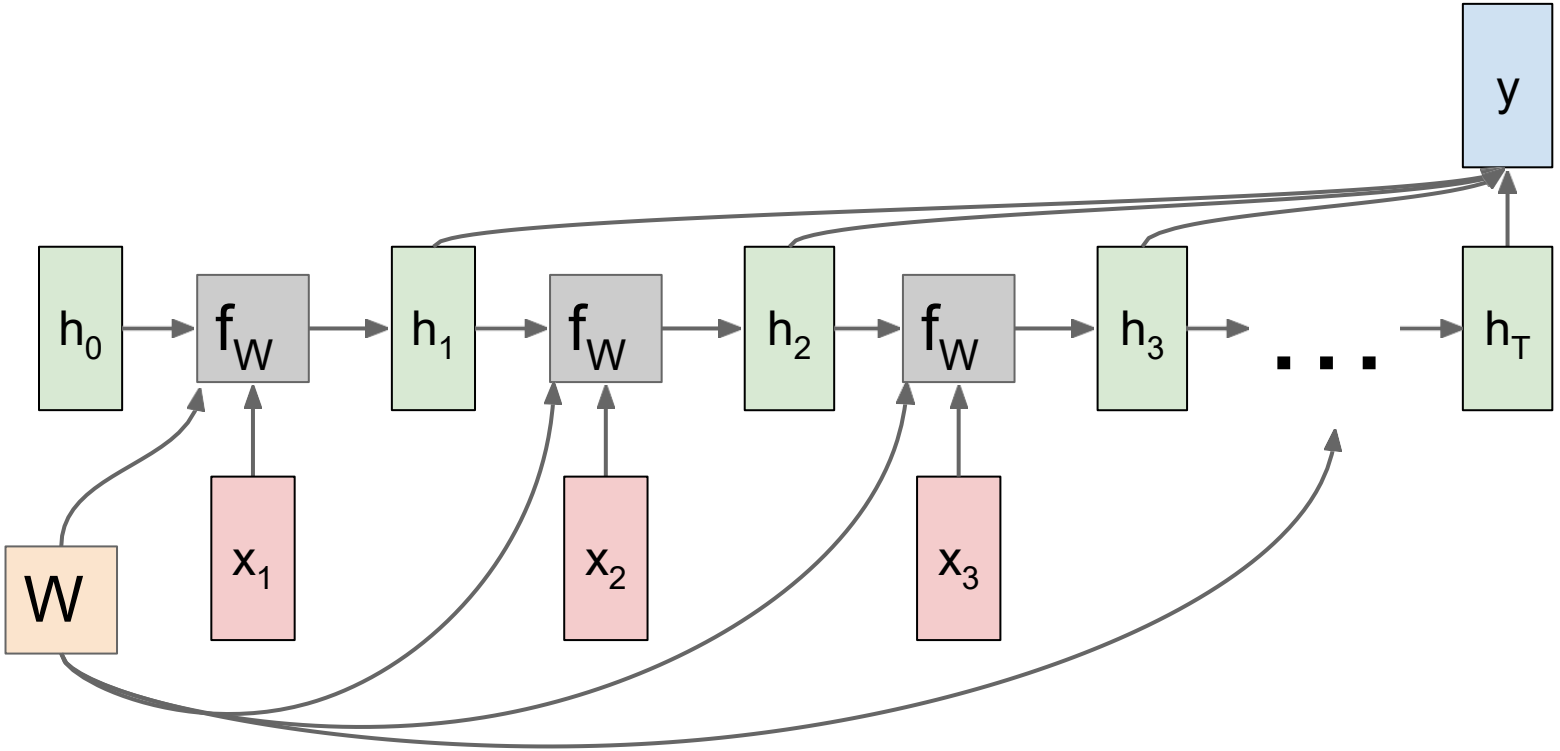
RNN：计算图：多对多



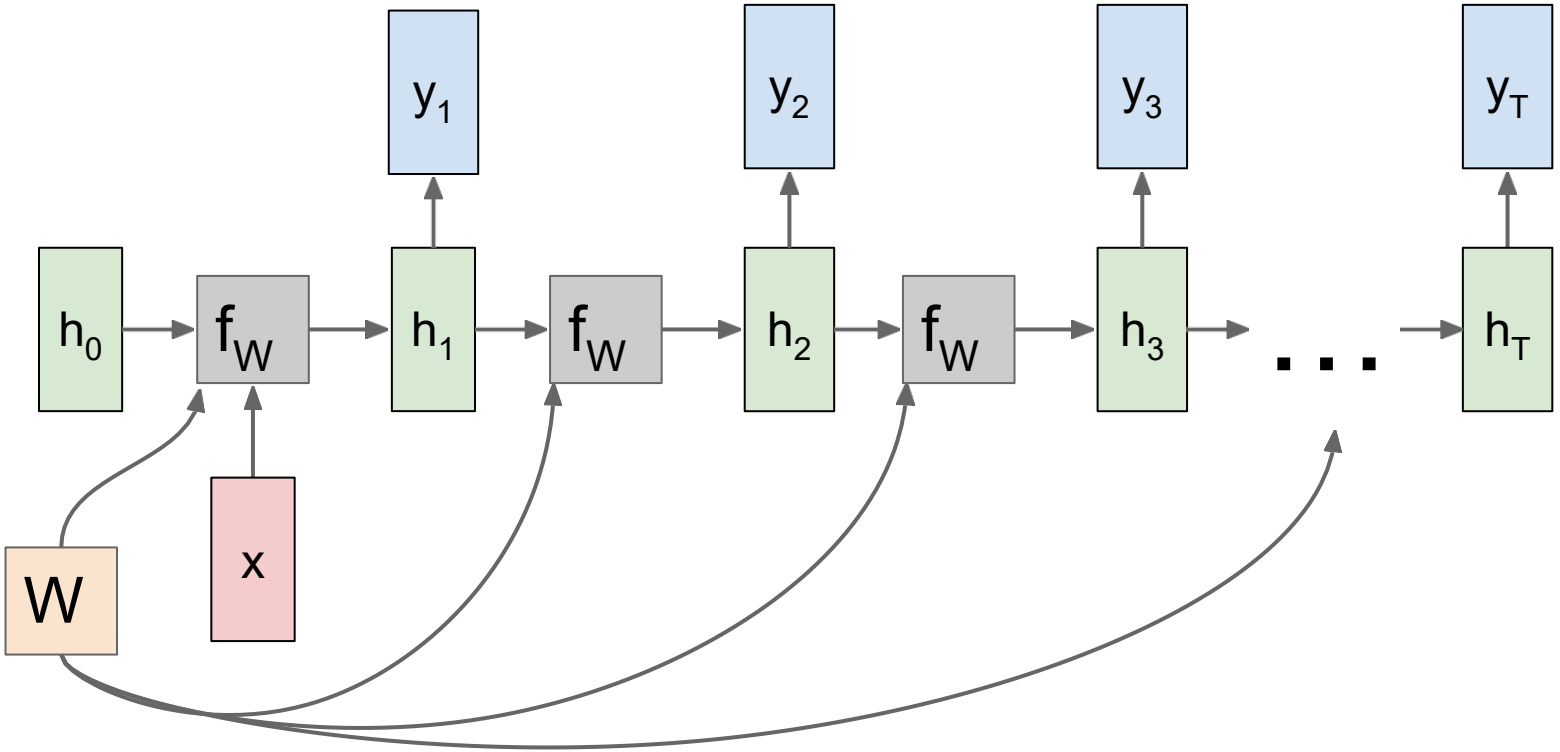
RNN：计算图：多对一



RNN：计算图：多对一

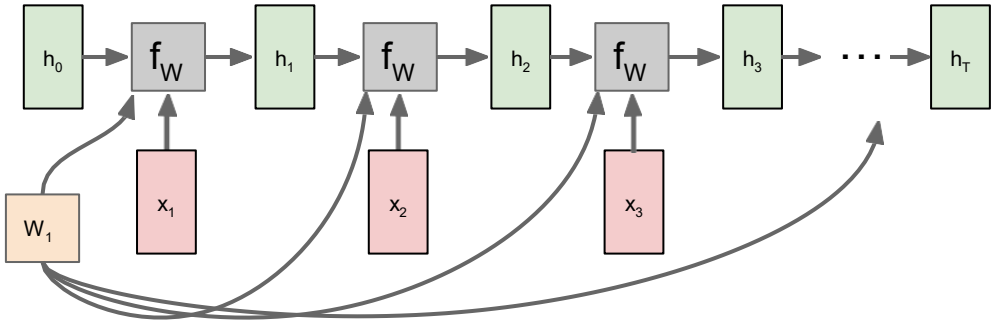


RNN：计算图：一对多



序列 到序列：多对一 +一对多

多对一：将输入序列编码为单个向量



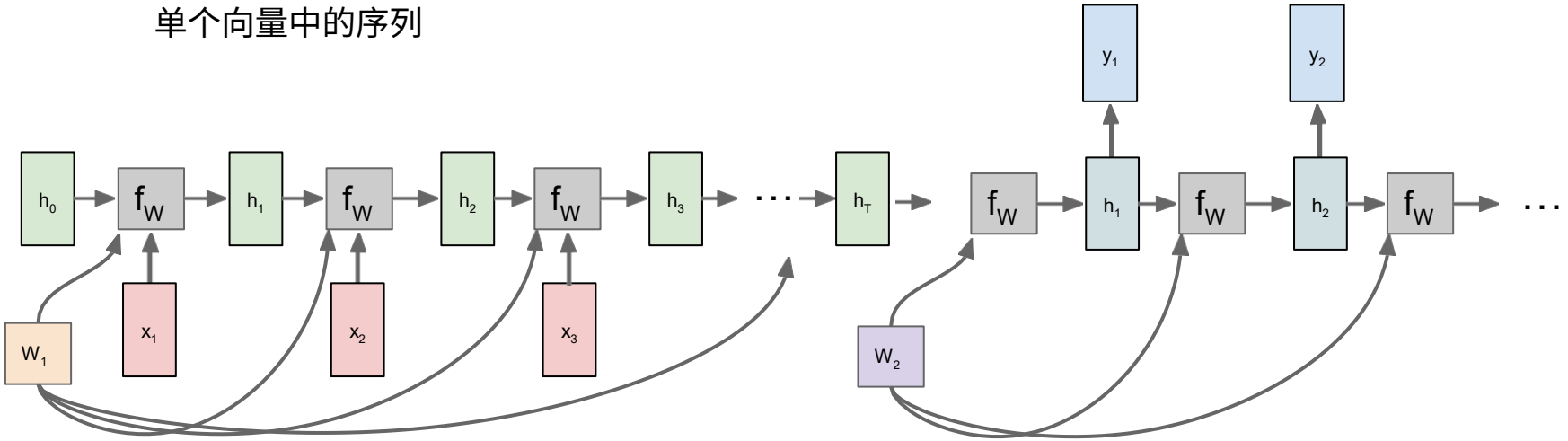
Sutskever et al, “Sequence to Sequence Learning with Neural Networks”, NIPS 2014



序列到序列：多对一 +一对多

One to many：从单个输入向量生成输出序列

多对一：编码输入
单个向量中的序列



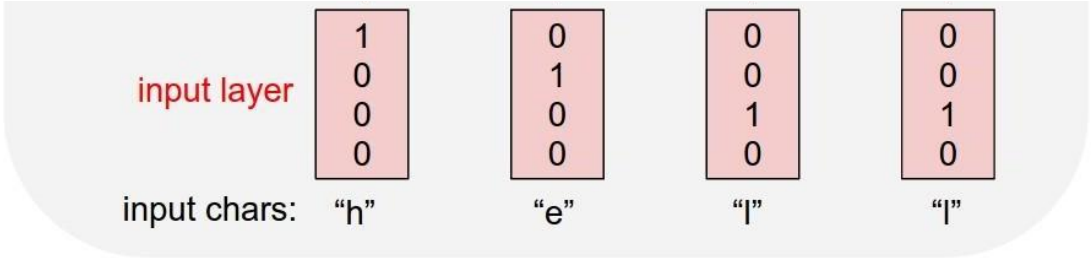
Sutskever et al, “Sequence to Sequence Learning with Neural Networks”, NIPS 2014



示例：字符级语言模型

词汇表：[
h,e,l,o]

示例训练序列：
“hello”

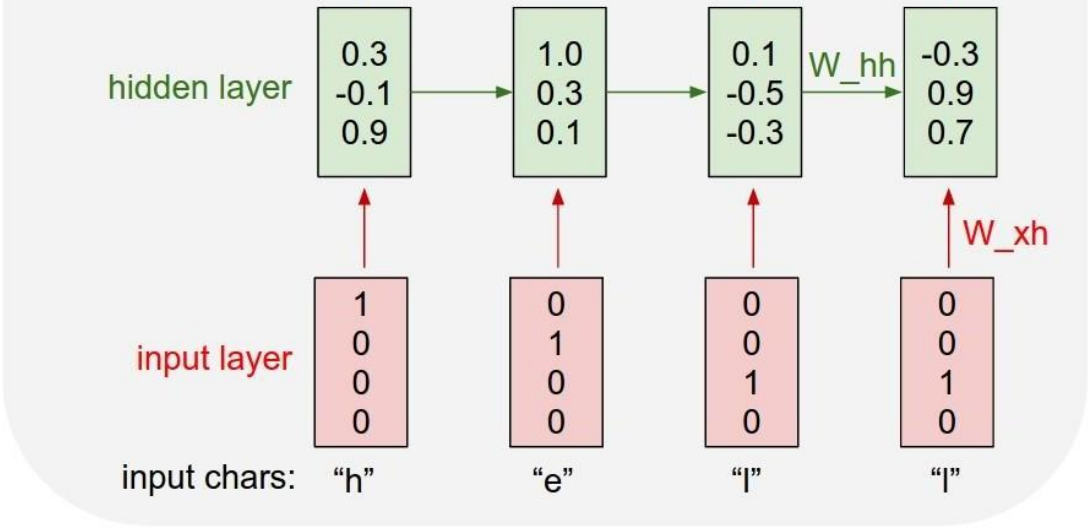


示例：字符级语言模型

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

词汇表：[
h,e,l,o]

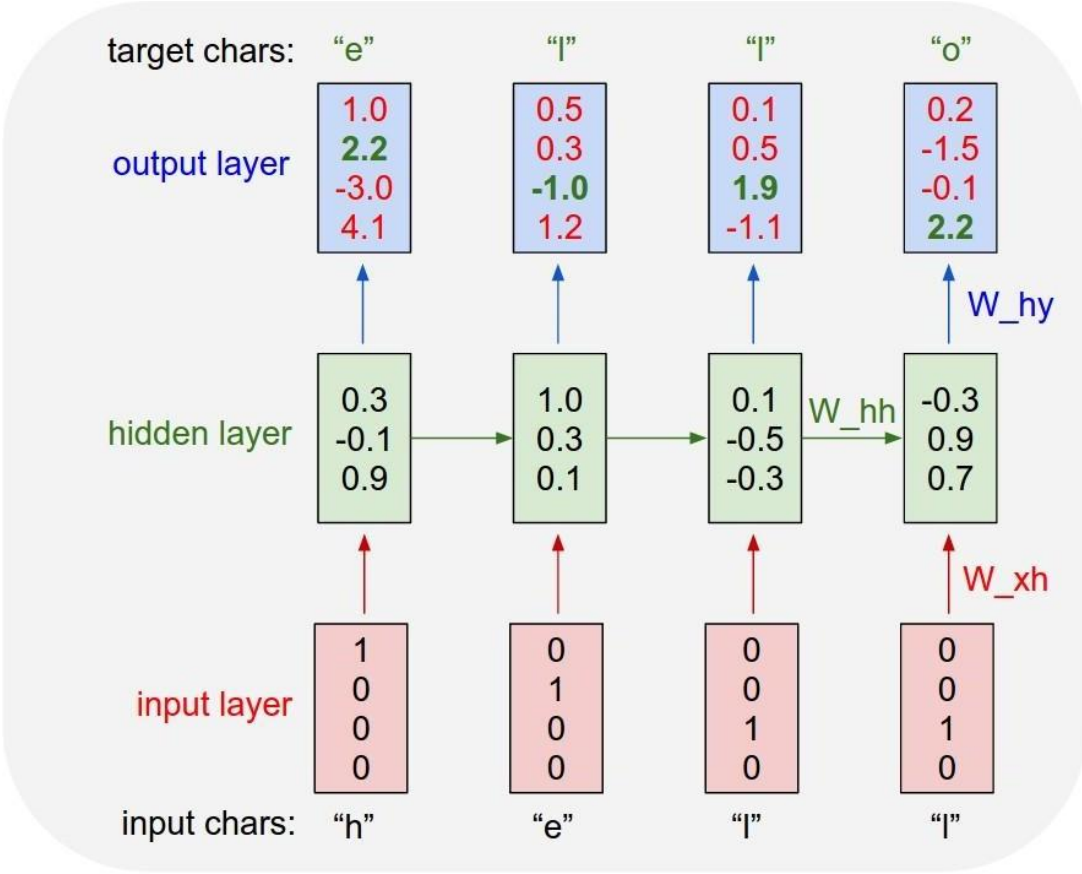
示例训练序列：
“hello”



示例：字符级语言模型

词汇表：[
h,e,l,o]

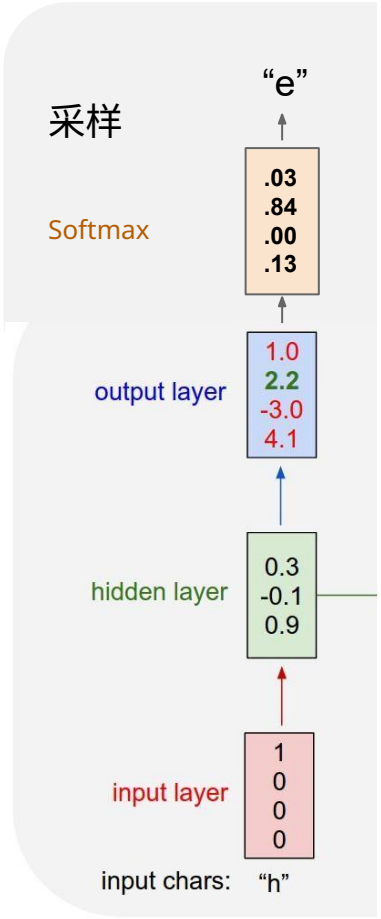
训练序列示例：
“hello”



示例：字符级语言模型采样

词汇表：[
h,e,l,o]

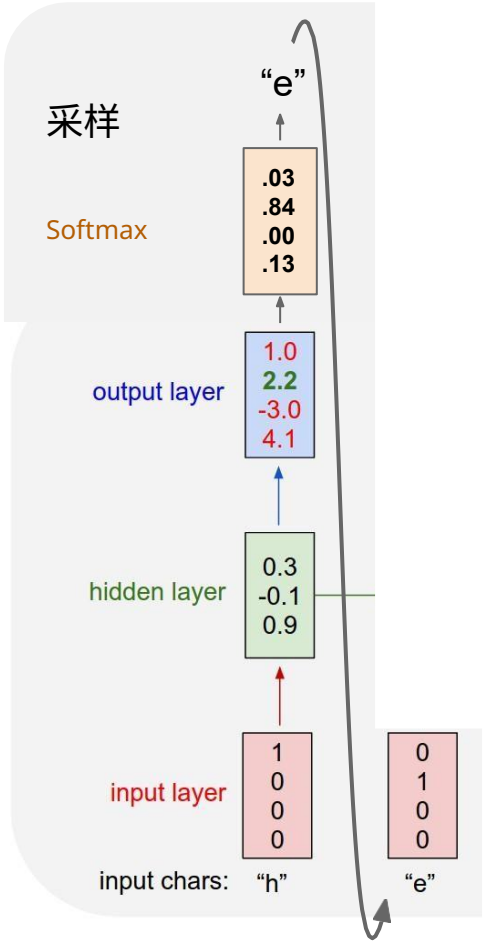
在测试时，一次采样一个字符，并反馈给模型



示例：字符级语言模型采样

词汇表：[
h,e,l,o]

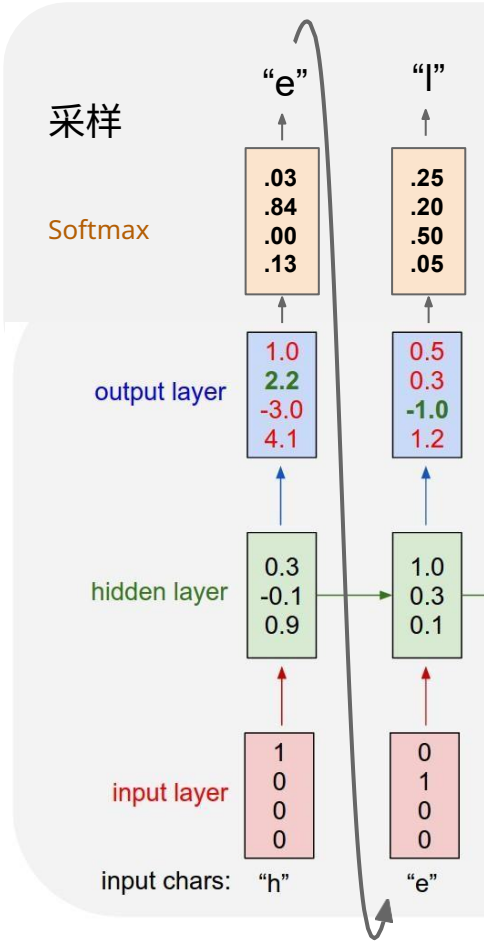
在测试时，一次采样一个字符，并将其反馈给模型



示例：字符级语言模型采样

词汇表：[
h,e,l,o]

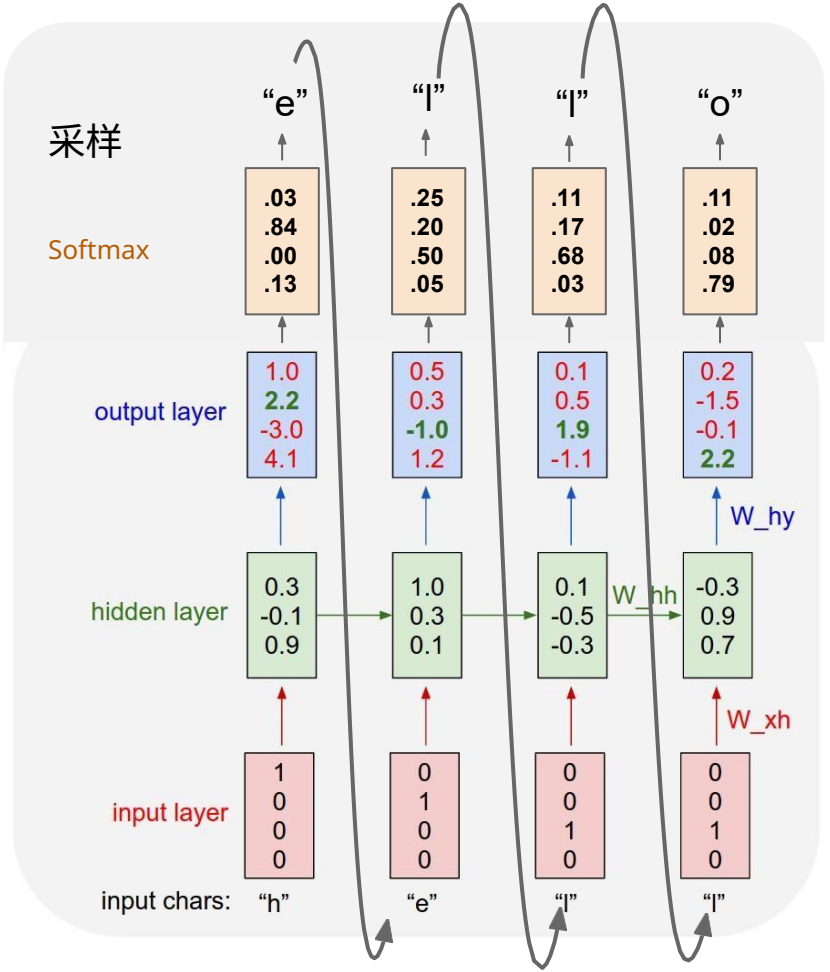
在测试时，一次采样一个字符，并反馈给模型



示例：字符级语言模型采样

词汇表：[
h,e,l,o]

测试时，每次采样一个字符，并将其反馈给模型



计算机视觉应用

- 视觉描述 – 用自然语言句子描述图像或视频的内容。



一只猫坐在松树旁，抬头仰望。



一只狗和一个女孩一起弹钢琴。

测试图像



本图像是CC0 公共领域

image

conv-64
conv-64
maxpool

conv-128
conv-128
maxpool

conv-256
conv-256
maxpool

conv-512
conv-512
maxpool

conv-512
conv-512
maxpool

FC-4096
FC-4096
FC-1000
softmax



测试图像

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096

FC-1000

softmax



测试图像

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

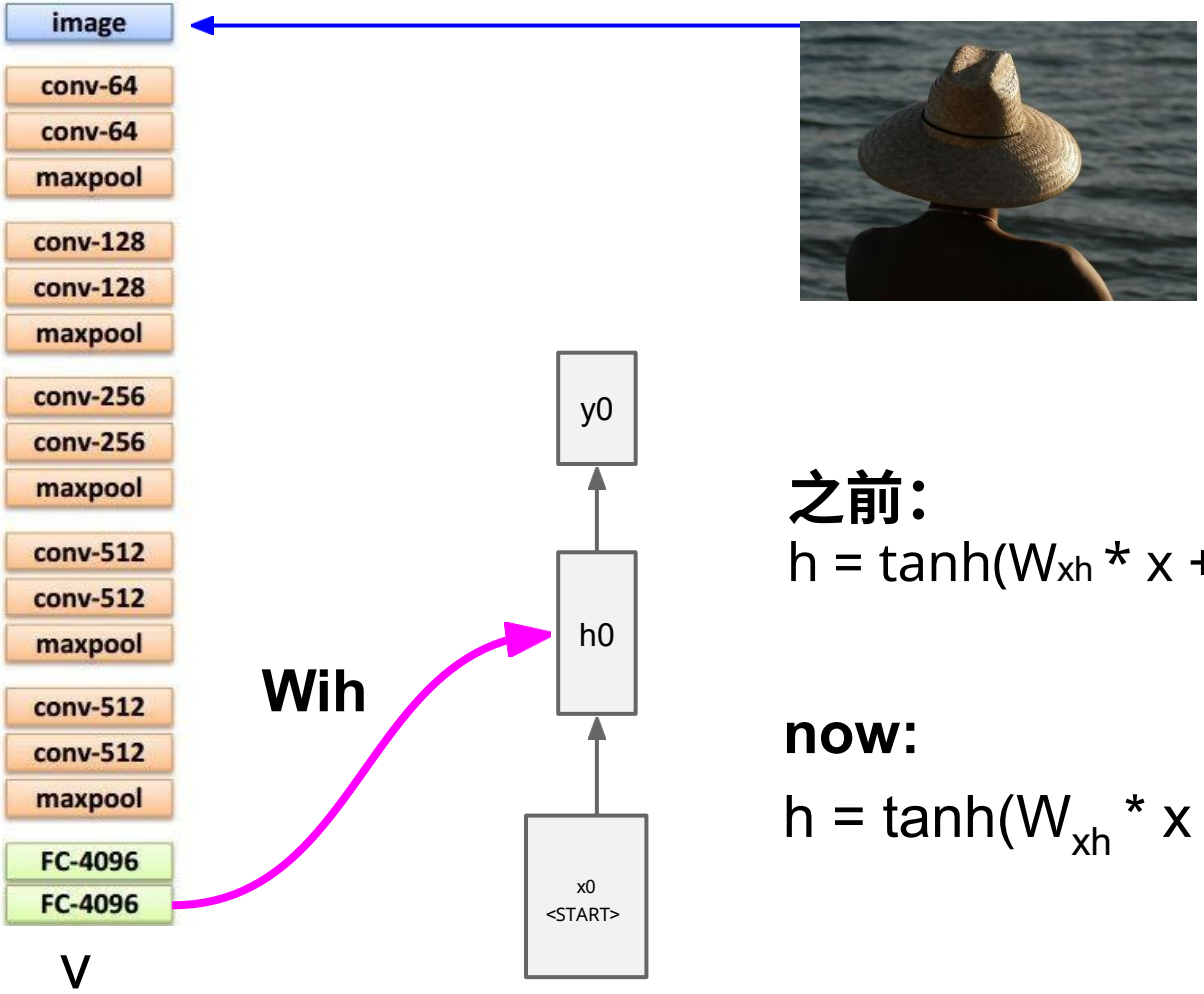
FC-4096



测试图像

x0<开始

>



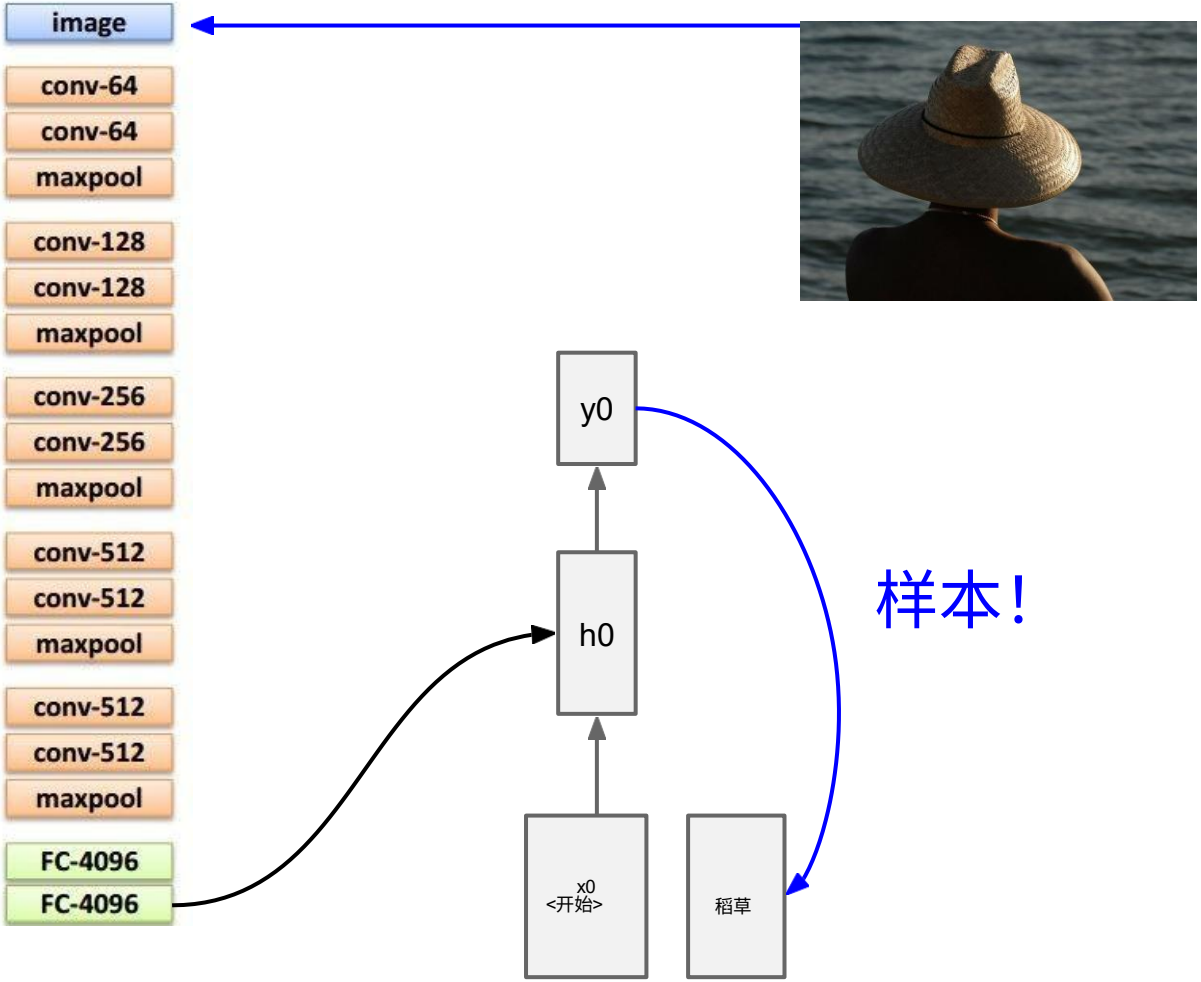
测试图像

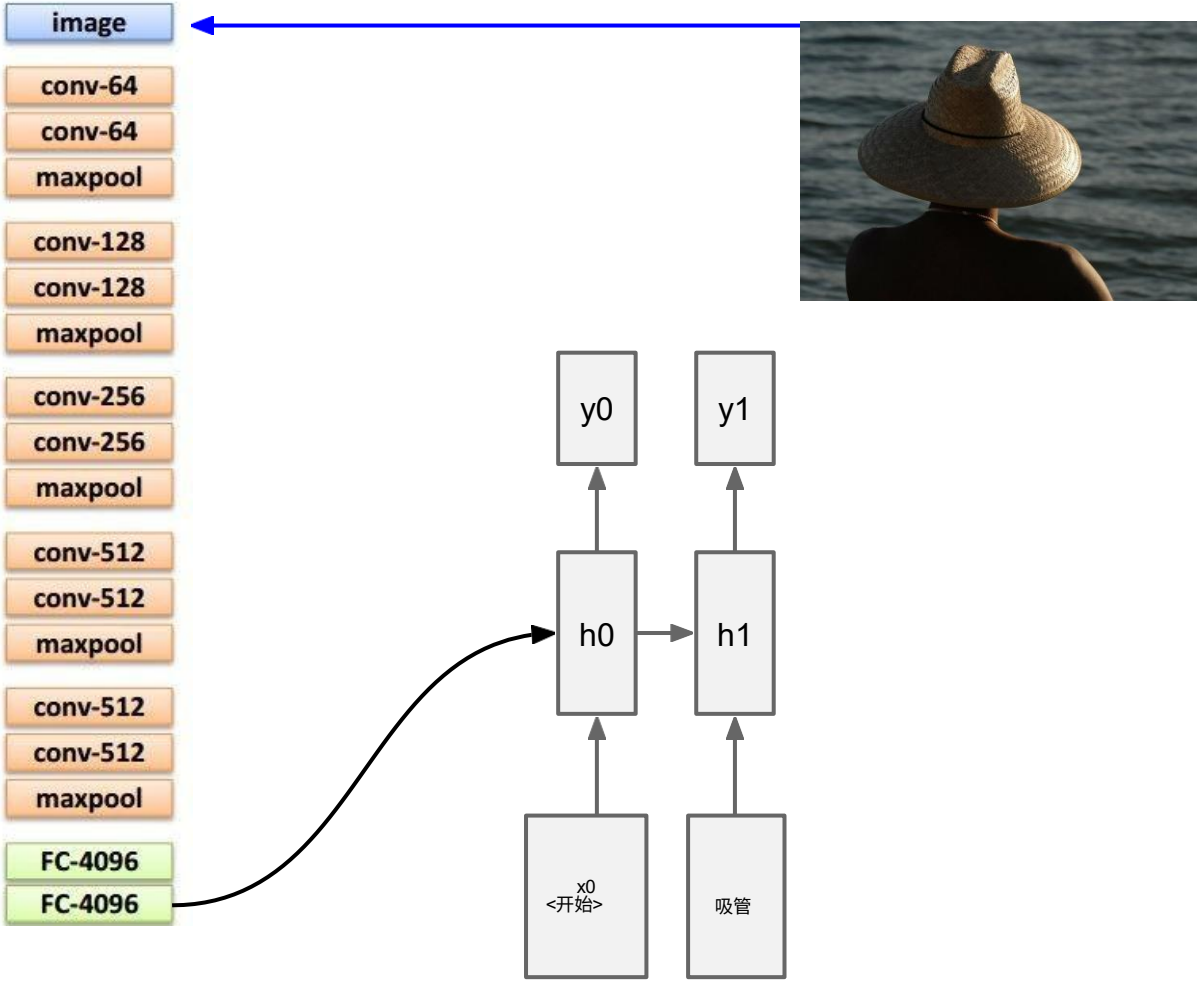
之前：

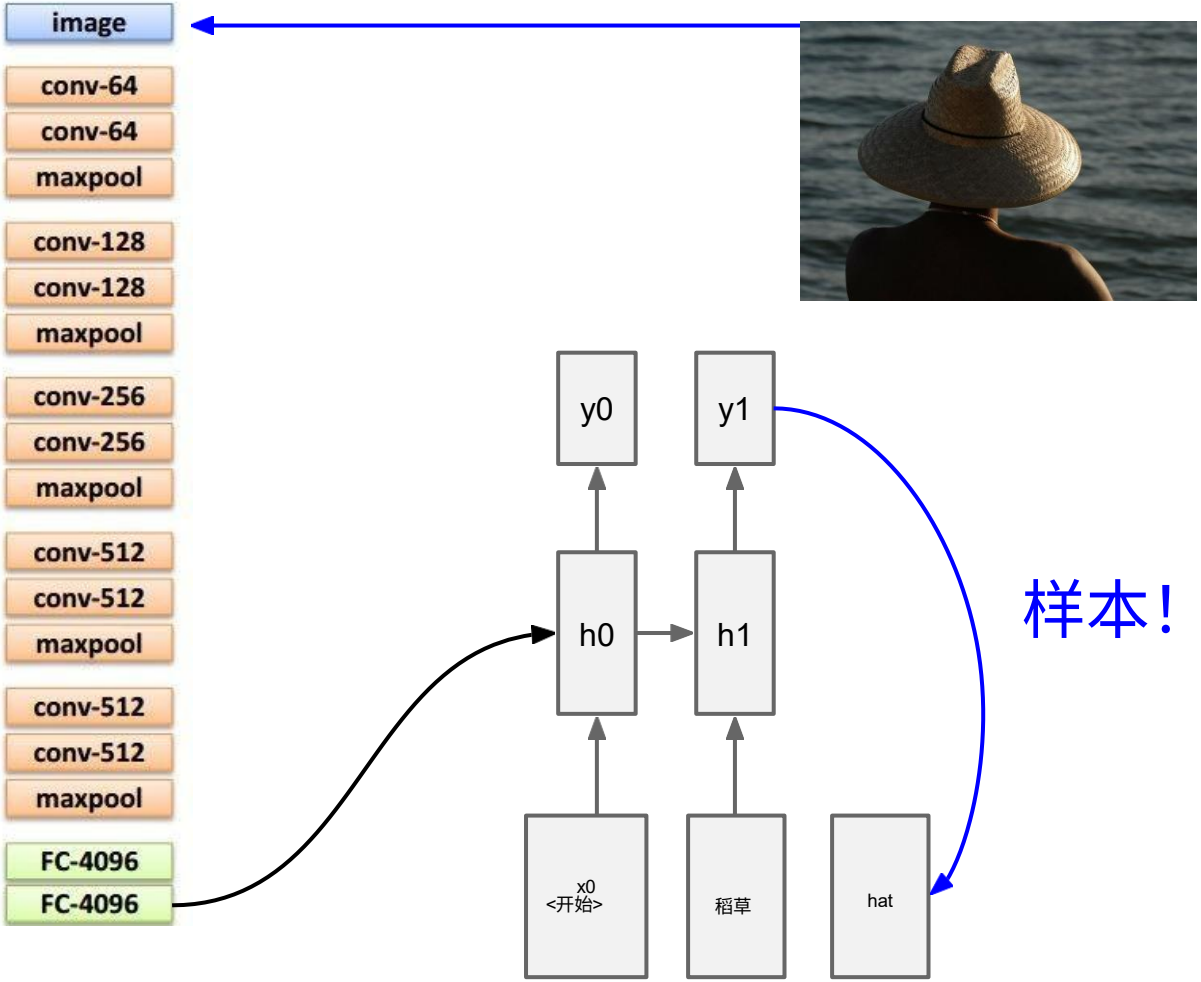
$h = \tanh(W_{xh} * x + W_{hh} * h)$

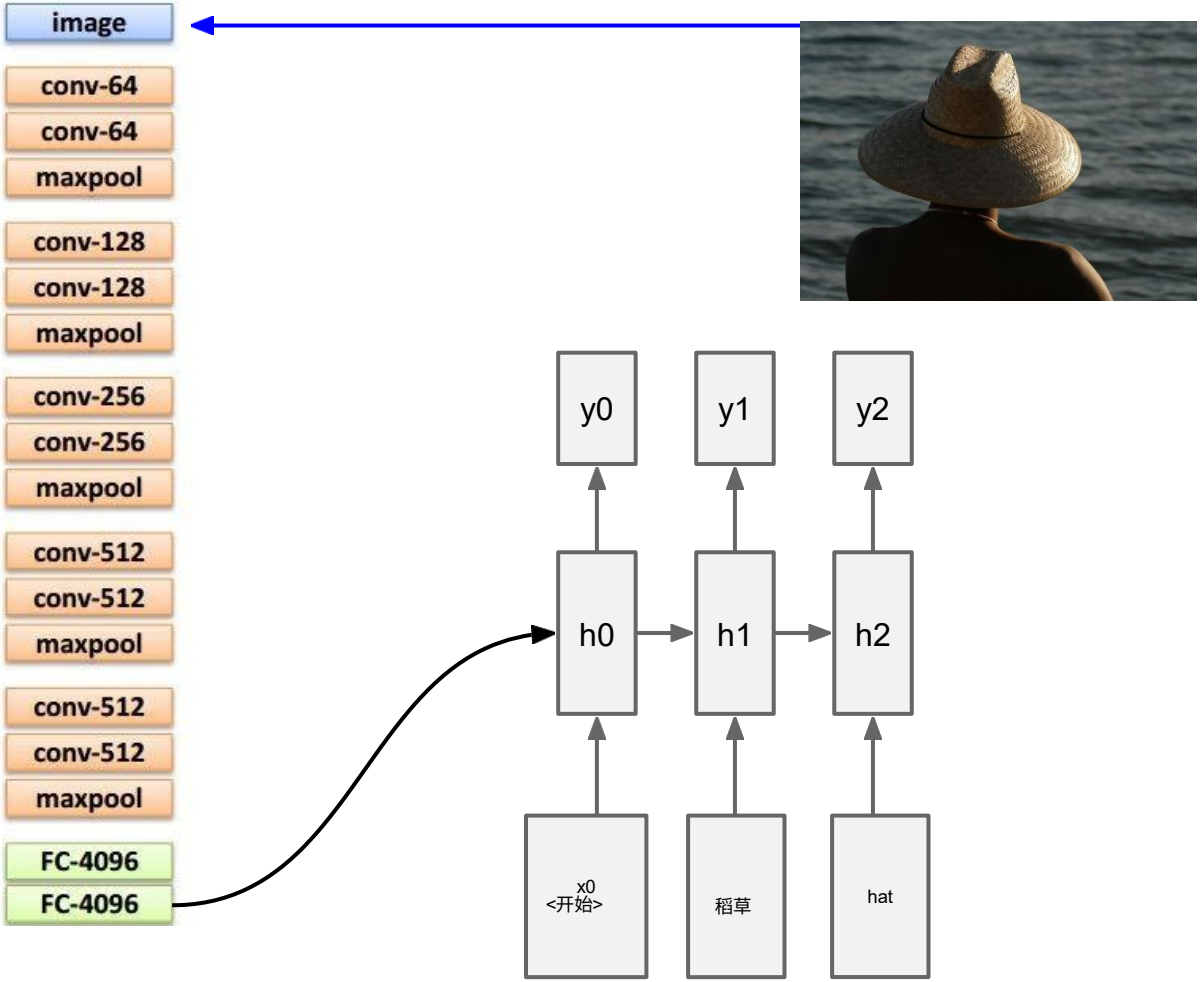
now:

$h = \tanh(W_{xh} * x + W_{hh} * h + \mathbf{W_{ih} * v})$

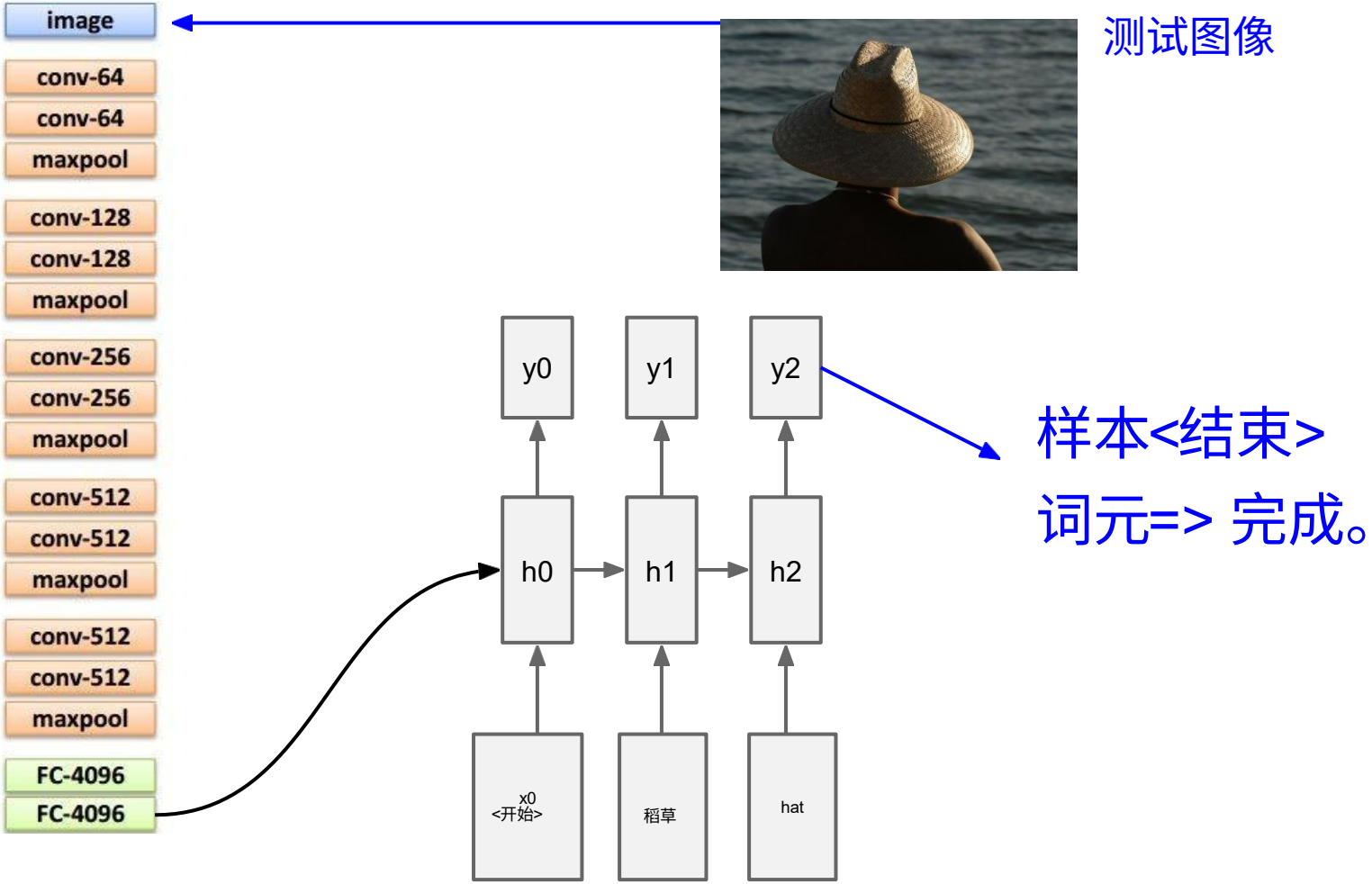

image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
h0
x0
<开始>
稻草
测试图像
样本!


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
h0
h1
x0
<开始>
吸管
测试图像


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
h0
h1
x0
<开始>
稻草
hat
测试图像
样本!


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
y2
h0
h1
h2
x0
<开始>
稻草
hat

测试图像


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
测试图像
y0
y1
y2
h0
h1
h2
x0
<开始>
稻草
hat
样本<结束>
词元=> 完成。

图像描述：示例结果

说明文字由 neuraltalk2 生成。所有图像均为 CC0 公有领域： 猫 手提箱， 猫 树， 狗，熊， 冲浪者， 网球， 长颈鹿，摩托车



一只猫坐在地板上的一个手提箱上



一只猫正坐在树枝上



一只狗叼着飞盘在草地上奔跑



一只白色泰迪熊坐在草地上



两个人拿着冲浪板在海滩上行走



球场上正在比赛的网球运动员



两只长颈鹿站在长满草的田野里



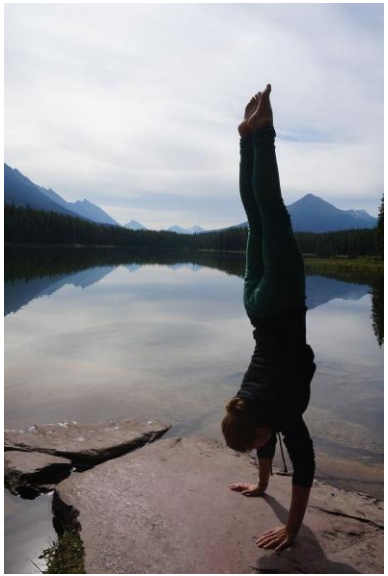
一名男子在泥土赛道上骑越野摩托车

图像描述：失败案例

使用 neuraltalk2 生成的说明文字。所有图像均为 CC0 公共领域：毛皮大衣，倒立，蜘蛛网，棒球



一位女士手里抱着一只猫



一位女士站在海滩上，手里拿着冲浪板



一只鸟栖息在树枝上



一个人拿着放在桌上的电脑鼠标



一名身穿棒球服、正在投球的男子

大纲

- **结合 RNNs 的注意力**- 在计算机视觉中 - 在自然语言处理中
- **通用注意力层**- 自注意力 - 位置编码 - 掩码注意力 - 多头注意力 - **Transformer**



大纲

- **使用 RNN 的注意力**- 在计算机视觉中 - 在自然语言处理中
- **通用注意力层**
  - 自注意力 - 位置编码 - 掩码注意力 - 多头注意力
- **Transformers**



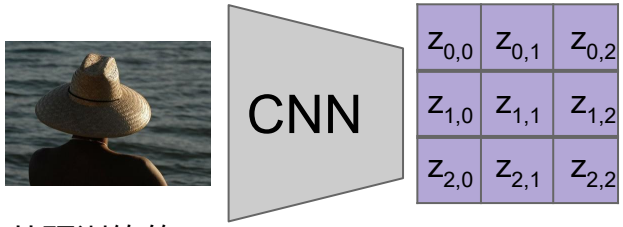
图像描述中，使用原始RNN时：

问题出在哪里？



使用空间特征的图像描述

输入：图像 **I** **输出**：序列 **y** = y_1, y_2,..., y_T



从预训练的 CNN 中提取空间特征

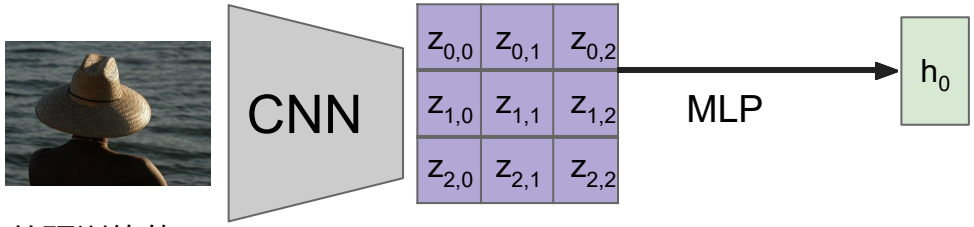
特征：
H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

使用空间特征的图像描述

输入：图像 **I** **输出**： 序列 **y** = y_1, y_2,..., y_T

编码器：h_0 =fw(**z**)，其中 **z** 是空间 CNN 特征，fw(**.**) 是一个 MLP



从预训练的 CNN 中提取空间特征

特征：
H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

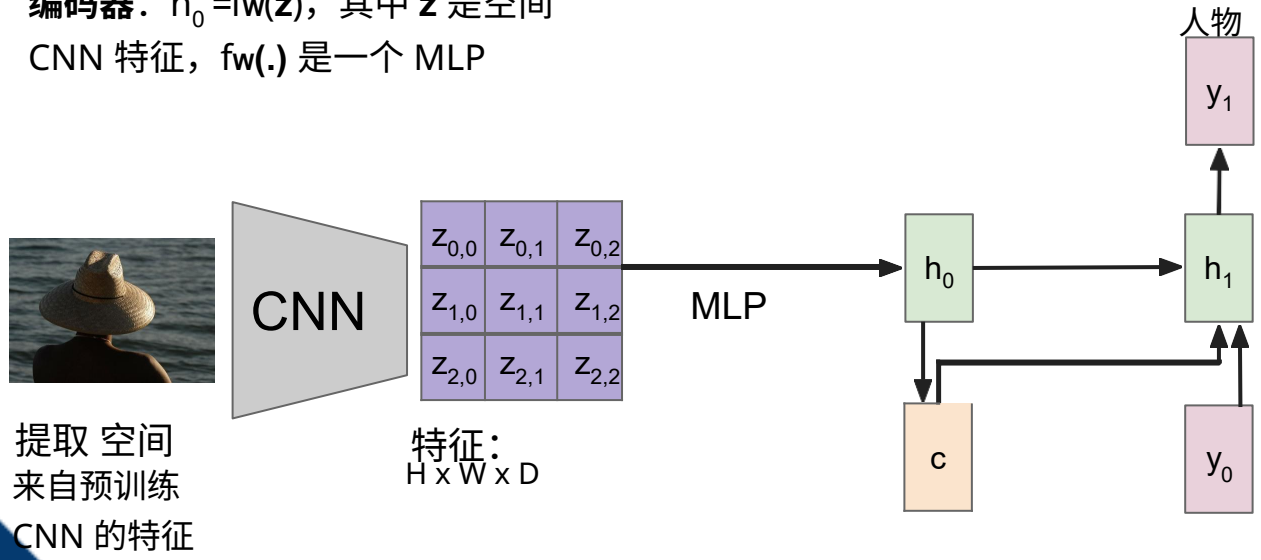
基于空间特征的图像描述

1 2 **输入**：图像 **I** **输出**：序列

y = y , y ,..., y_T

解码器：$y_t = gv(y_{t-1}, h_{t-1}, c)$，其中上下文向量 c 通常为 $c = h_0$

编码器：$h_0 = fw(z)$，其中 **z** 是空间 CNN 特征，fw(.) 是一个 MLP



Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

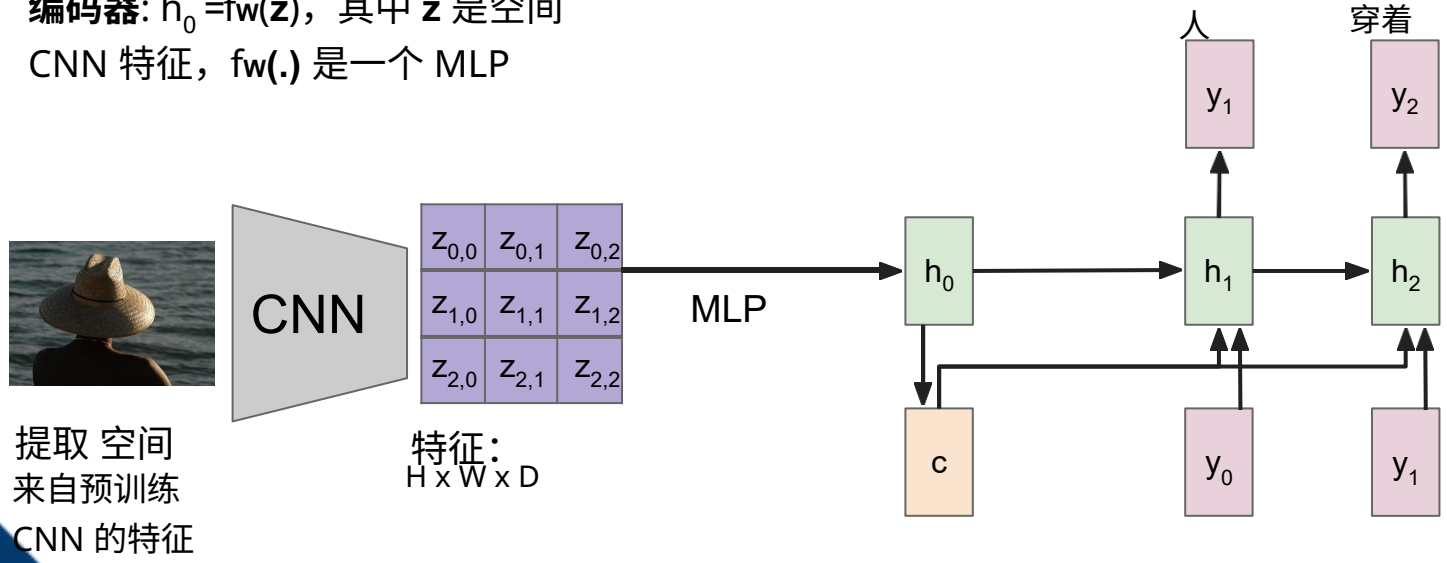
使用空间特征的图像描述

1 2 **输入**：图像 **I** **输出**： 序列

y = y , y ,..., y_T

解码器： $y_t = gv(y_{t-1}, h_{t-1}, c)$ 其中上下文向量 c 通常为 $c = h_0$

编码器: $h_0 = fw(z)$，其中 **z** 是空间 CNN 特征，fw(.) 是一个 MLP



Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

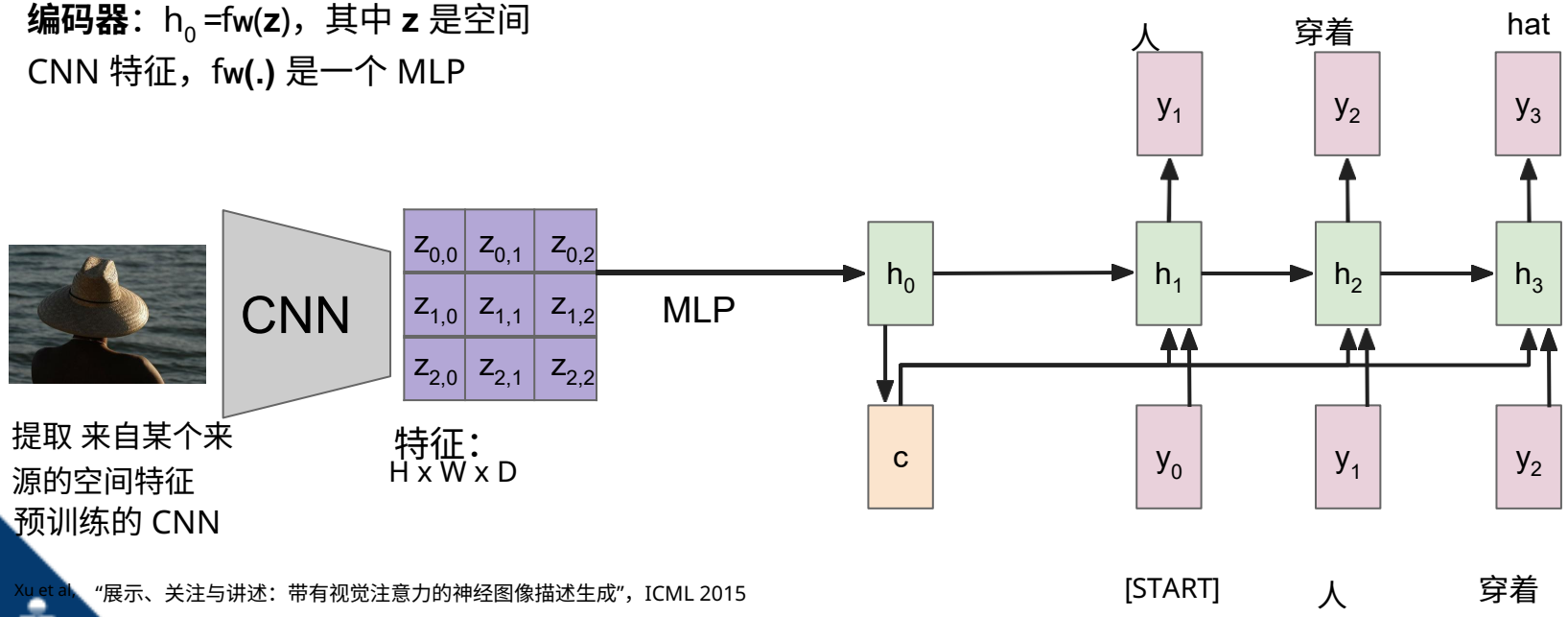
基于空间特征的图像描述

1 2 **输入**：图像 **I** **输出**：序列

y = $y_1, y_2, \ldots, y_T$

解码器：$y_t = g_v(y_{t-1}, h_{t-1}, c)$ 其中上下文向量 c 通常是 $c = h_0$

编码器：$h_0 = f_w(z)$，其中 **z** 是空间 CNN 特征，$f_w(.)$ 是一个 MLP



Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

使用空间特征的图像描述

1 2 **输入**：图像 **I** **输出**：序列

y = y , y ,..., y_T

解码器: $y_t = g_v(y_{t-1}, h_{t-1}, c)$ 其中上下文向量 c 通常为 $c = h_0$

编码器：$h_0 = f_w(z)$，其中 **z** 是空间 CNN 特征，$f_w(.)$ 是一个 MLP



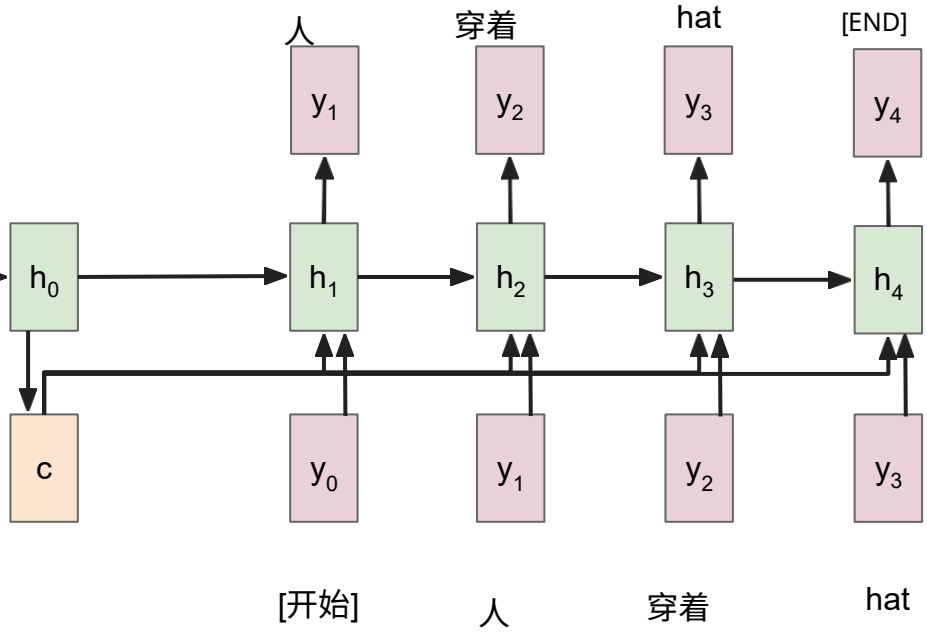
CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

MLP

从预训练的 CNN 中提取空间特征

特征：
H x W x D



Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

使用空间特征的图像描述

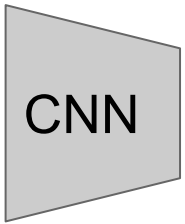
问题：输入通过 c 被“瓶颈化”

- 模型需要将其想要表达的所有内容都编码到 c 中

如果我们想要生成真正很长的描述——长达数百个词——这就是一个问题?



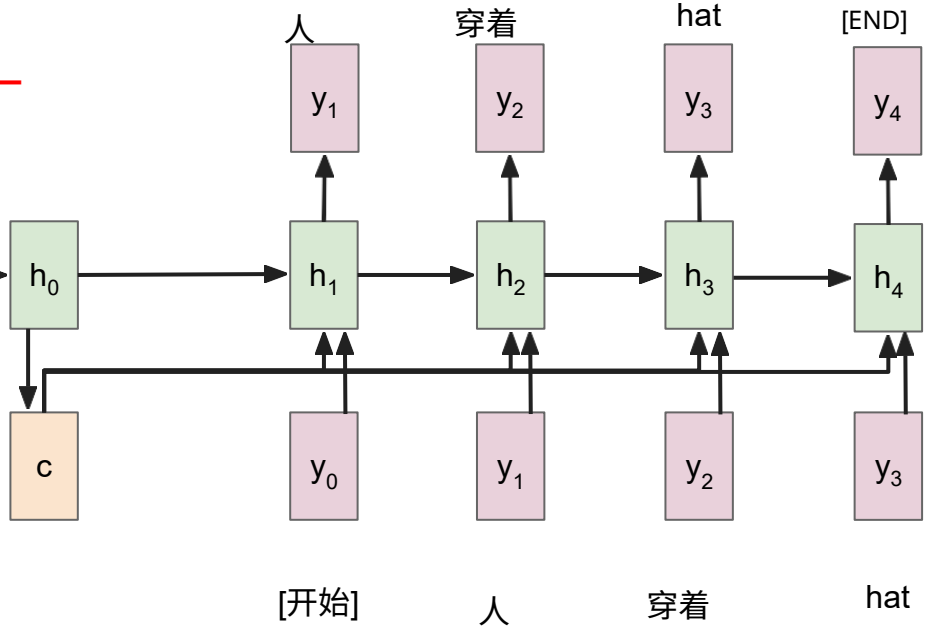
从预训练的 CNN 中提取空间特征





特征：
H x W x D

MLP



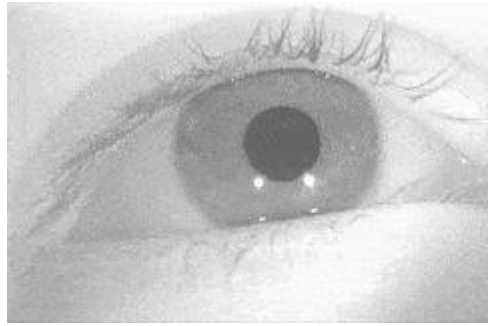
Xu et al. “展示、关注与描述：基于视觉注意力的神经图像描述生成”，ICML 2015

基于 RNN 的图像描述 与 注意力

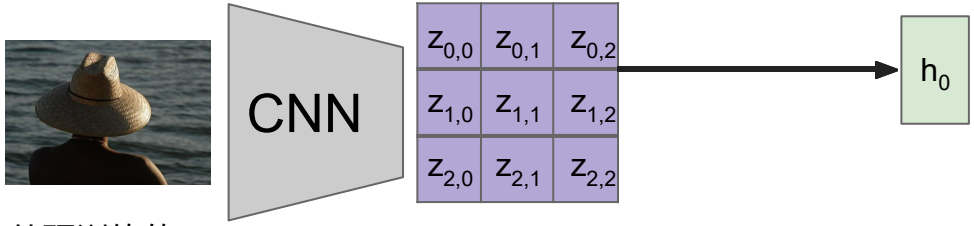
GIF 来源

注意力思想：每个时间步都有一个新上下文向量。

每个上下文向量将关注不同的图像区域



人类的注意力扫视



从预训练的 CNN 中提取空间特征

特征：
H x W x D

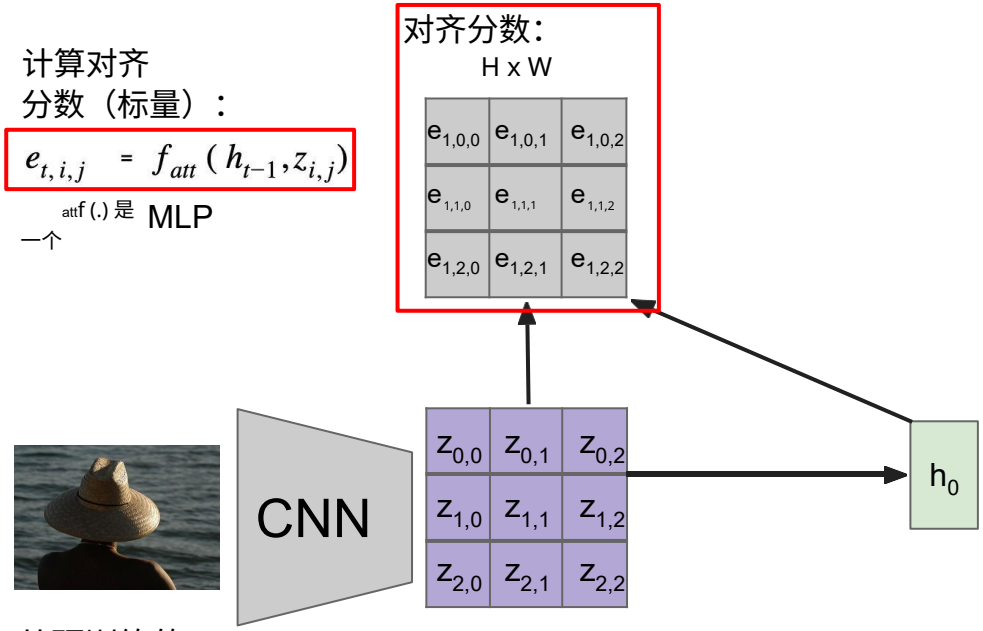
Xu et al. “展示、关注与描述：基于视觉注意力的神经图像描述生成”，ICML 2015

基于RNNs的图像描述与注意力机制

计算对齐分数（标量）：

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

f_{att}(.) 是一个 MLP



从预训练的 CNN 中提取空间特征

特征：
H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

图像描述：使用 RNN 和 注意力

计算对齐分数（标量）：

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

f_{att}(.) 是一个 MLP

对齐分数：
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

注意力：
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

归一化得到
注意力权重：

$$a_{t,:,:} = softmax(e_{t,:,:})$$

$a_{t,i,j}$ 0 < a < 1，注意力值之和为 1



CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

h_0

从预训练的 CNN 中提取空间特征

特征：
H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015



基于RNN的图像描述与注意力

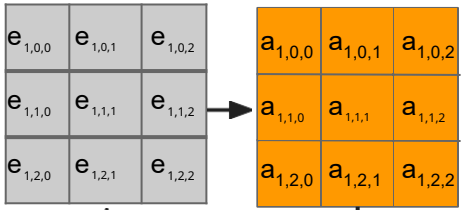
计算对齐得分（标量）：

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

f_{att}(.) 是一个MLP

对齐分数：
H x W

注意力：
H x W



归一化以获得注意力权重：

$$a_{t,:,:} = softmax(e_{t,:,:})$$

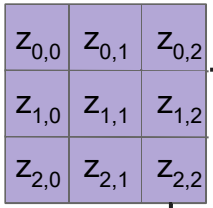
$a_{t,i,j}$ 0 < a < 1，注意力值之和为 1

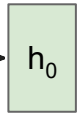
计算上下文向量：

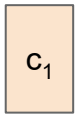
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN









提取空间特征，来自一个预训练的CNN

特征：
H x W x D

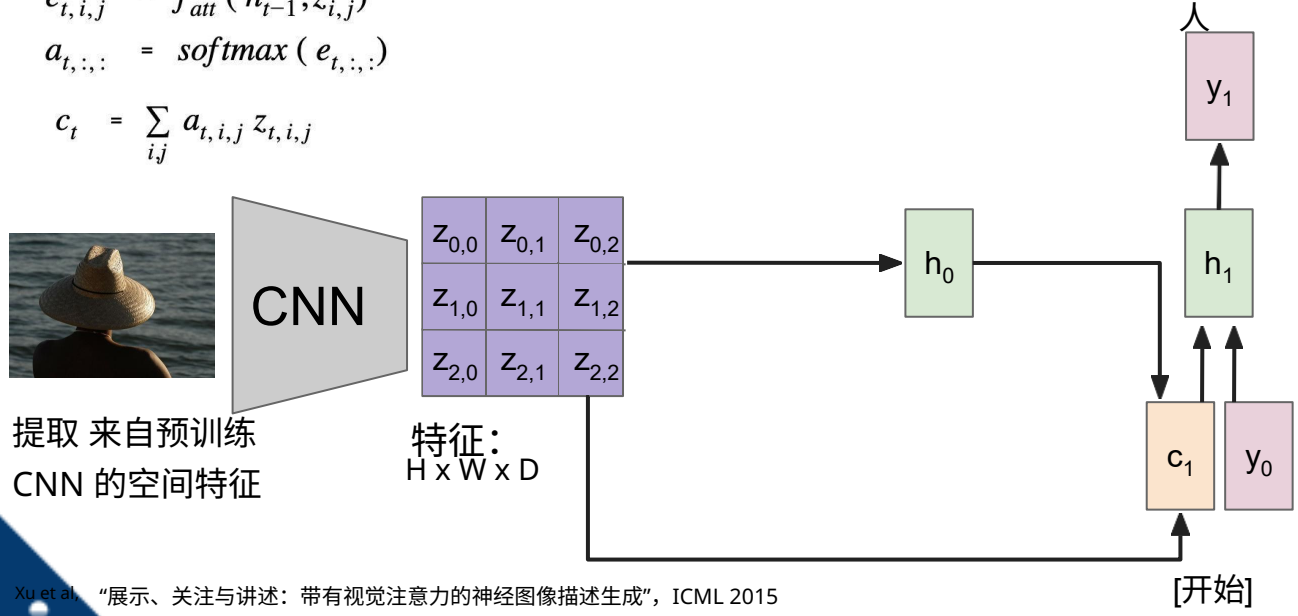
Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

基于 RNN 的图像描述与注意力

解码器的每个时间步都使用不同的上下文向量，该向量关注输入图像的不同部分

解码器： $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$ 每个时间步生成新的上下文向量

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$

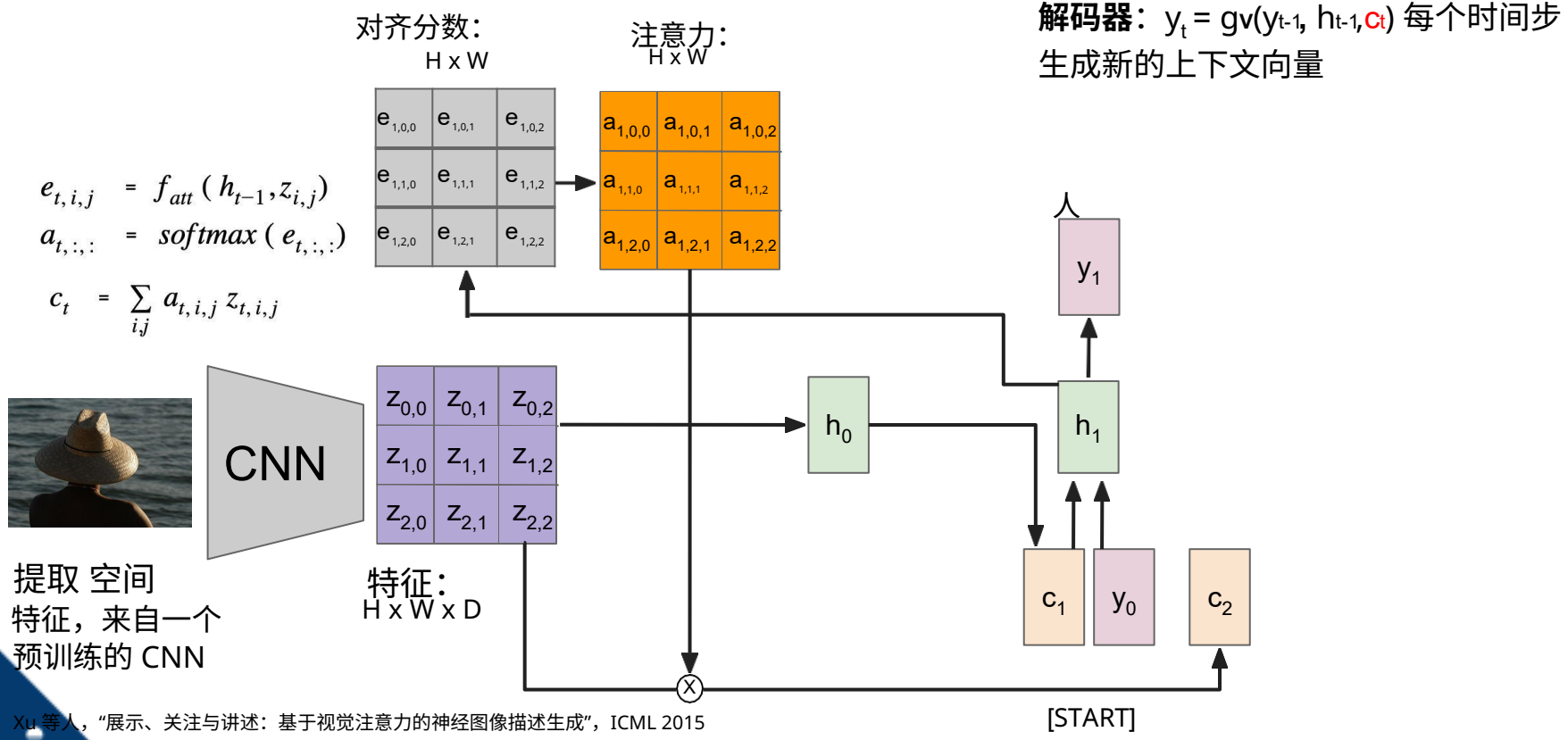


提取 来自预训练 CNN 的空间特征

特征：H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

图像描述：使用 RNNs 和 注意力



解码器： $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$ 每个时间步生成新的上下文向量

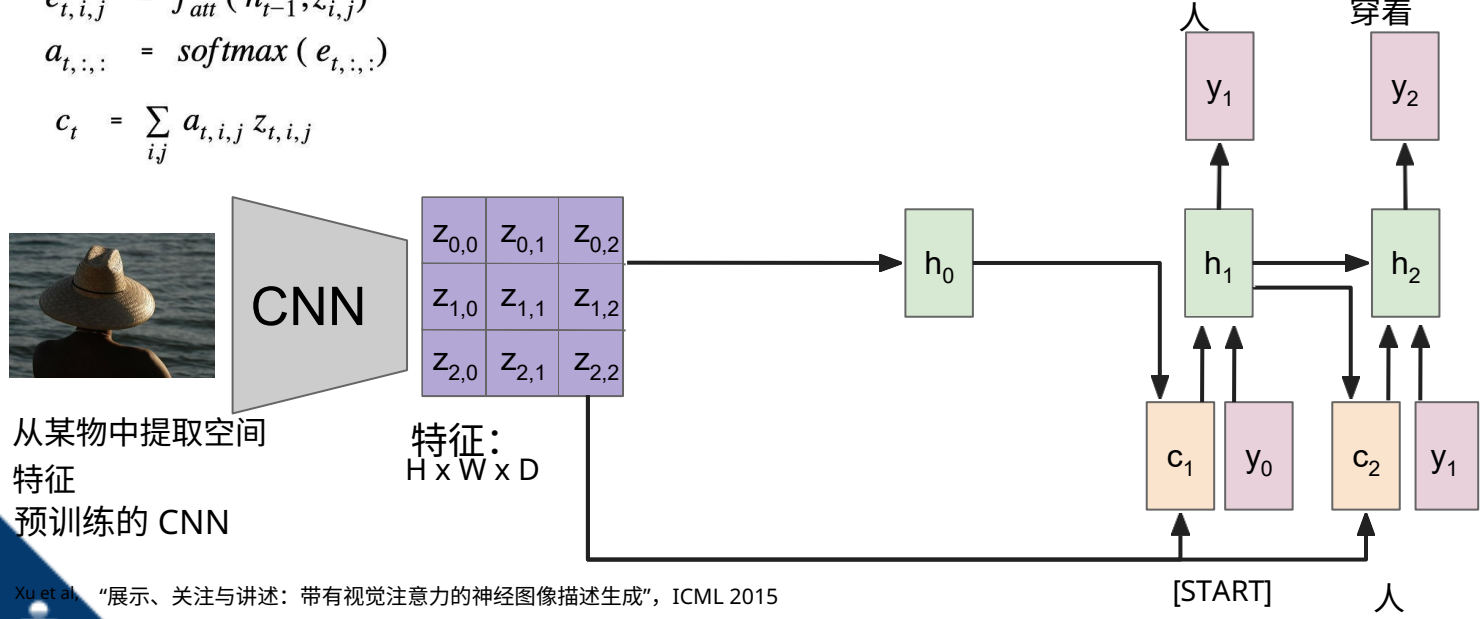
Xu 等人，“展示、关注与讲述：基于视觉注意力的神经图像描述生成”，ICML 2015

基于 RNN 和 注意力的图像描述

解码器的每个时间步都使用不同的上下文向量，该向量关注输入图像的不同部分

解码器： $y_t = gv(y_{t-1}, h_{t-1}, c_t)$ 每个时间步的新上下文向量

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



从某物中提取空间特征
预训练的 CNN

特征：
H x W x D

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

使用 RNN 的图像描述与注意力

解码器的每个时间步使用不同的上下文向量，该向量关注输入图像的不同部分

解码器： $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$ 每个时间步的新上下文向量

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

提取 来自一个的空间特征
预训练的CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

特征：
H x W x D

h_0 h_1 h_2 h_3

人 穿着 hat

y_1 y_2 y_3

c_1 y_0 c_2 y_1 c_3 y_2

[START] 人 穿着

Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

基于 RNN 和 注意力机制的图像描述

解码器的每个时间步使用不同的上下文向量，该向量关注输入图像的不同部分

解码器： $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$ 每个时间步的新上下文向量

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

提取 来自预训练CNN的空间特征

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

特征：
H x W x D

h_0 h_1 h_2 h_3 h_4

y_1 y_2 y_3 y_4

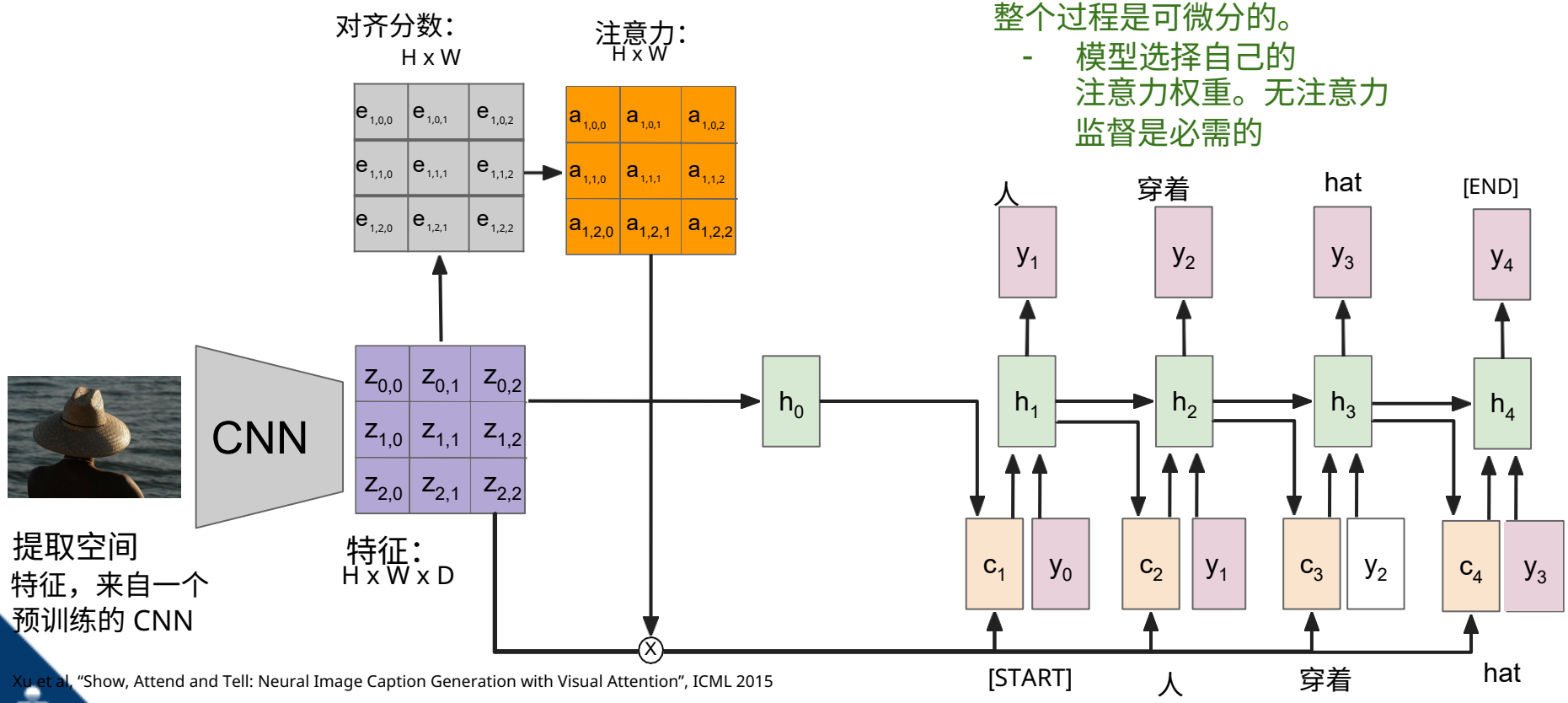
人 穿着 hat [END]

c_1 y_0 c_2 y_1 c_3 y_2 c_4 y_3

[START] 人 穿着 hat

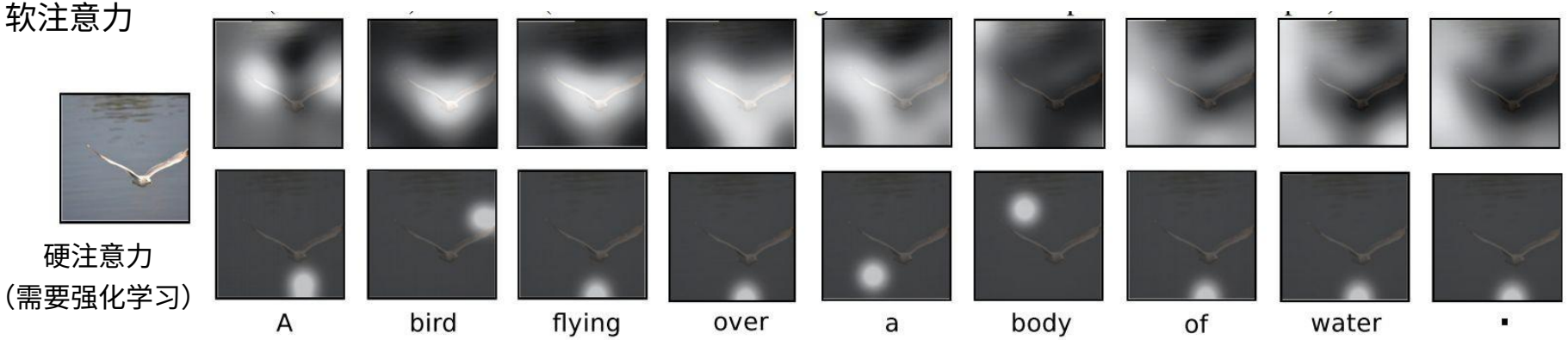
Xu et al. “展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML 2015

使用 RNN 与注意力机制的图像描述



Xu et al, “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”, ICML 2015

基于注意力的图像描述生成



Xu 等人，“展示、关注与讲述：带有视觉注意力的神经图像描述生成”，ICML2015 图片版权：Kelvin Xu、Jimmy Lei Ba、Jamie Kiros、Kyunghyun Cho、Aaron Courville、Ruslan Salakhutdinov、Richard S. Zemel 和 Yoshua Benchio，2015。经许可转载。

基于注意力的图像描述



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



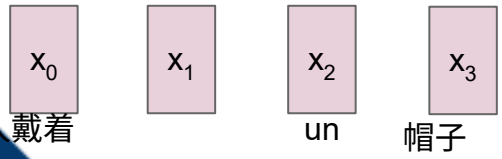
A giraffe standing in a forest with trees in the background.

Xu 等，“展示、关注与讲述：基于视觉注意力的神经图像描述生成”，ICML2015 图片版权归 Kelvin Xu、Jimmy Lei Ba、Jamie Kiros、Kyunghyun Cho、Aaron Courville、Ruslan Salakhutdinov、Richard S. Zemel 和 Yoshua Benchio 所有，2015。经许可转载。

类似的 NLP 任务 - 语言翻译示例

输入：序列 **x** = x_1, x_2,..., x_T **输出：**

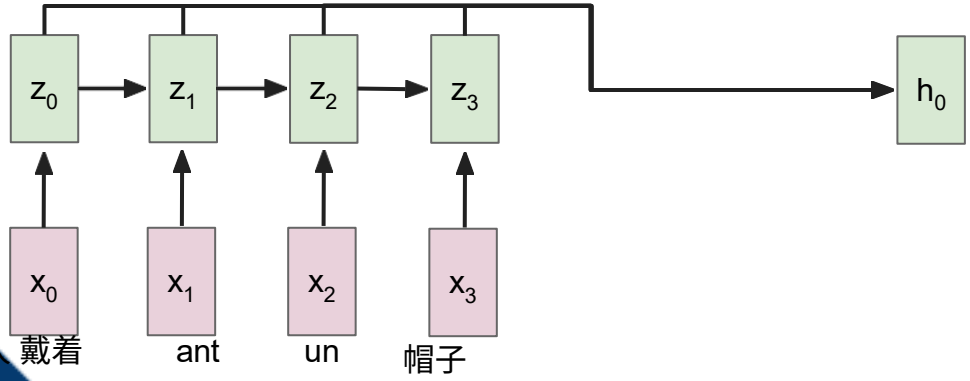
序列 **y** = y_1, y_2,..., y_T



类似的 NLP 任务 - 语言翻译示例

输入：序列 **x** = x_1, x_2,..., x_T **输出**：
序列 **y** = y_1, y_2,..., y_T

编码器：h_0= fw(**z**) 其中 z_t =
RNN(x_t, u_{t-1}) fw(.) 是 MLP，
u 是隐藏的 RNN 状态

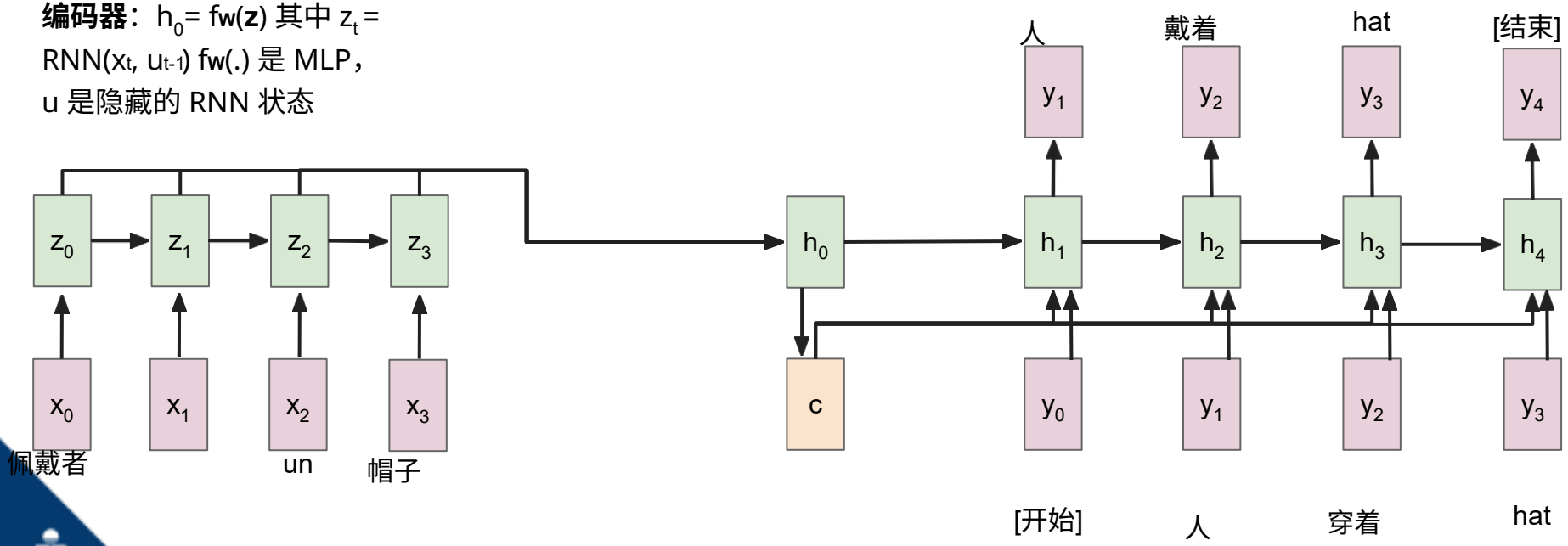


类似的NLP 中的任务——语言翻译示例

输入：序列 **x** = x , x ,..., x **输出**： 序列 **y** = y_1, y_2,..., y_T

解码器：$y_t = gv(y_{t-1}, h_{t-1}, c)$，其中上下文向量 c 通常为 $c = h_0$

编码器：$h_0 = fw(\mathbf{z})$ 其中 $z_t = RNN(x_t, u_{t-1})$ fw(.) 是 MLP，u 是隐藏的 RNN 状态

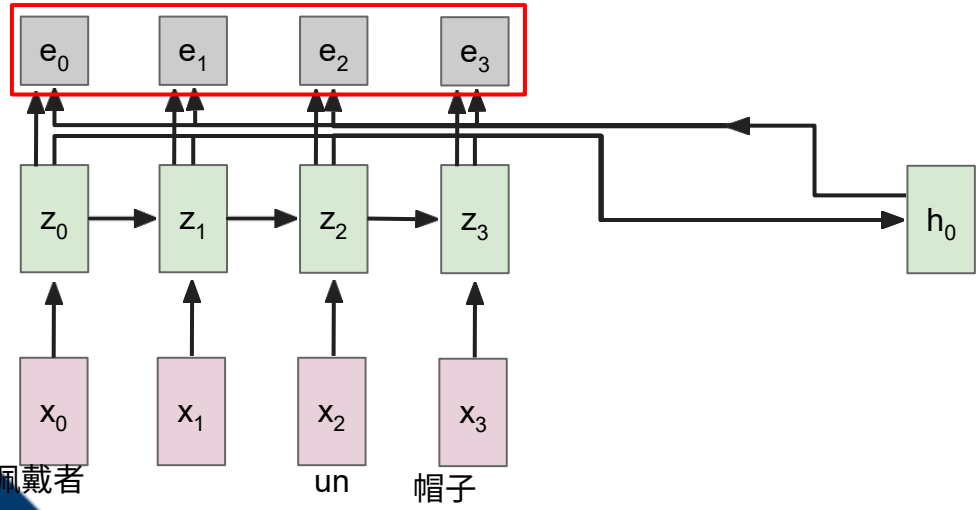


注意力 在 NLP 中 ——语言翻译示例

计算对齐分数（标量）：

$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

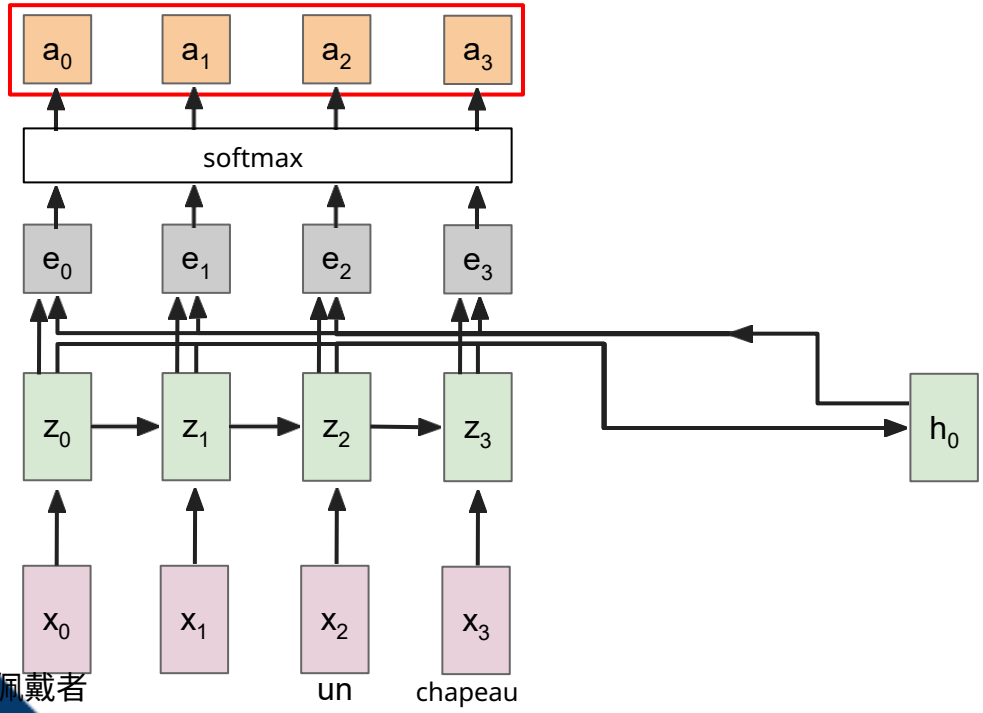
$f_{att}(.)$ 是一个 MLP



Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015



注意力 在NLP中——语言翻译示例



计算对齐分数（标量）：

$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

$f_{att}(.)$ 是一个 MLP

归一化以获取注意力权重：

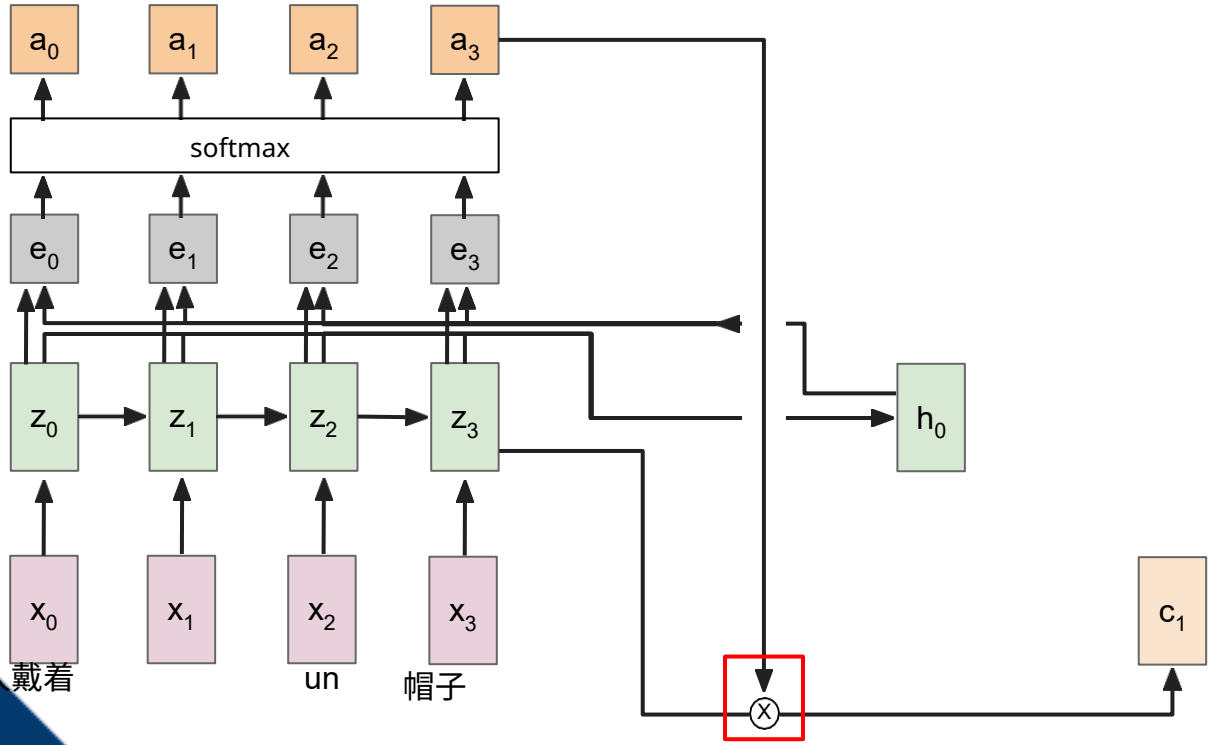
$$a_{t,:} = softmax(e_{t,:})$$

$0 < a_{t,i,j} < 1$，注意力值之和为1

Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015



注意力机制在 NLP 中——语言翻译示例



计算对齐分数（标量）：

$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

$f_{att}(.)$ 是一个多层感知机（MLP）。归一化以得到注意力权重：$0 < a_{t,i,j} < 1$，注意力值之和为 1。

$$a_{t,:} = softmax(e_{t,:})$$

计算上下文向量：

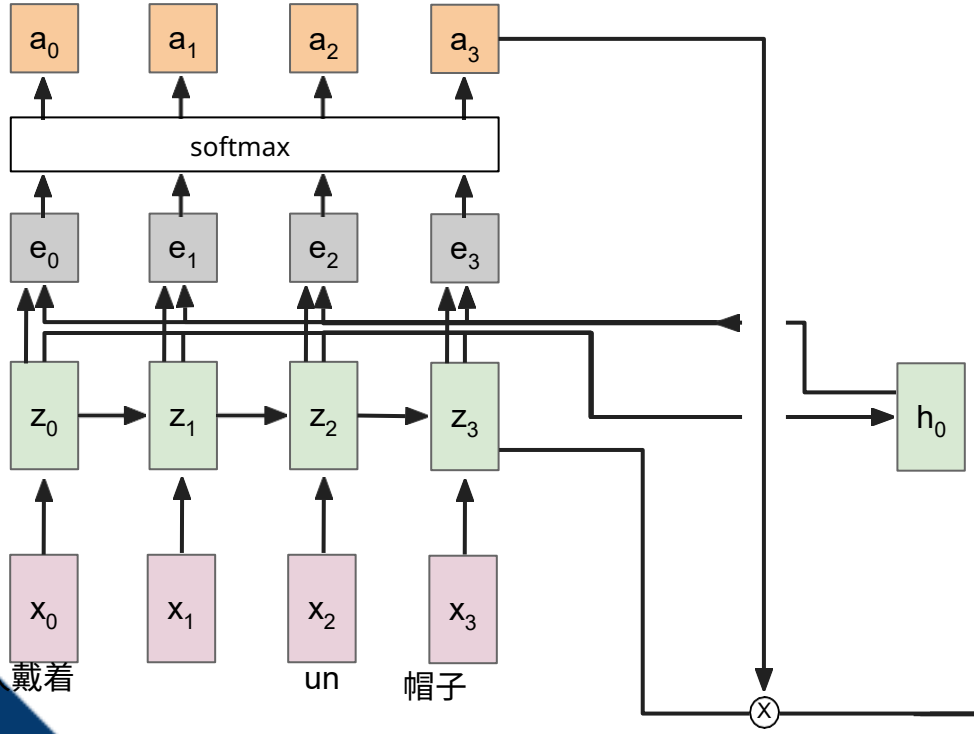
$$c_t = \sum_i a_{t,i} z_{t,i}$$

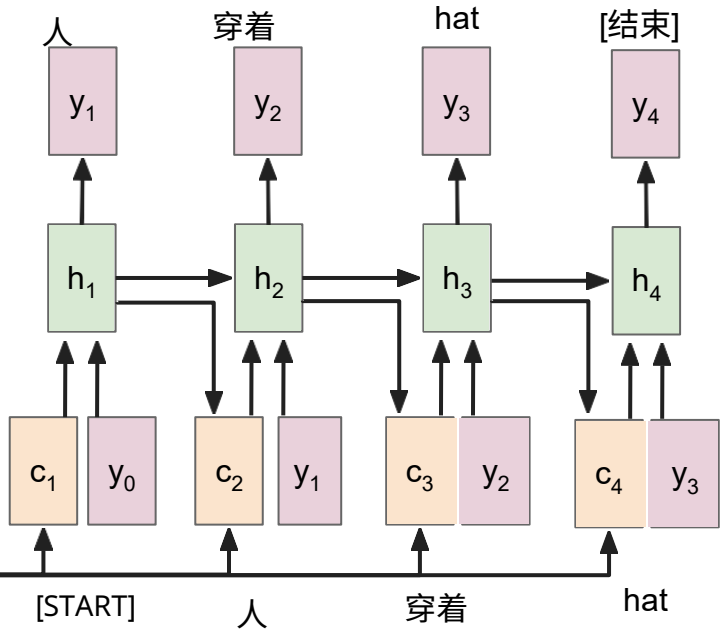
Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015



注意力机制 在 NLP 中——语言翻译示例

解码器： $y_t = g_v(y_{t-1}, h_{t-1}, c)$ 其中上下文向量 c 通常是 $c = h_0$





Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

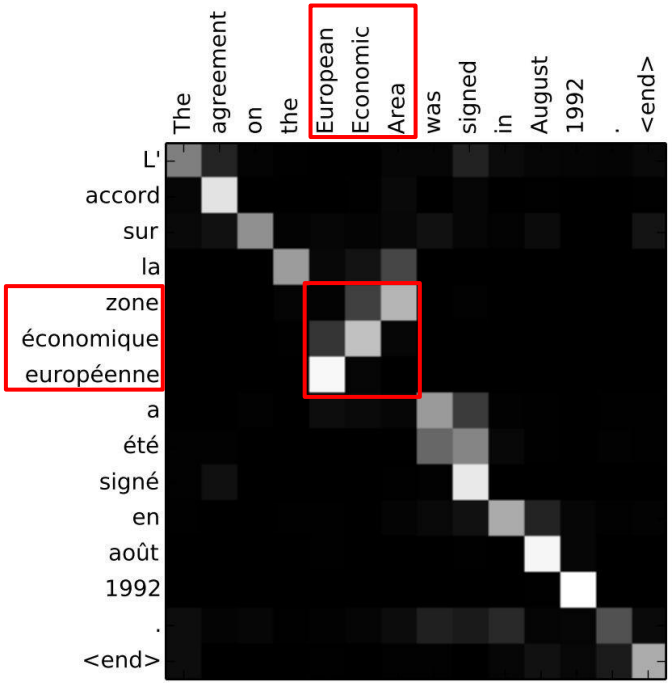
类似的注意力权重可视化

英语到法语翻译示例：

输入：“《欧洲经济区协定》于1992年8月签署。”

输出：“《欧洲经济区协定》于1992年8月签署。”

在没有任何注意力监督的情况下，模型会针对不同语言学习不同的词序



Bahdanau 等人，“通过联合学习对齐和翻译的神经机器翻译”，ICLR 2015。

今日议程：

- **使用RNN的注意力**- 在计算机视觉中 - 在自然语言处理中
- **一般注意力层**
  - 自注意力 - 位置编码 - 掩码注意力 - 多头注意力
- **Transformer**



注意力 我们刚刚在图像描述中看到的

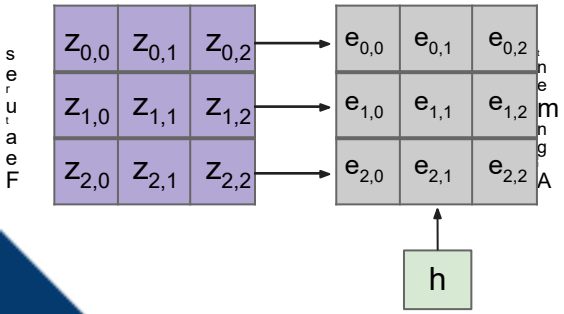
$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

h

输入：特征：**z** （形状：H x W x D） 查询：**h** （形状：D）

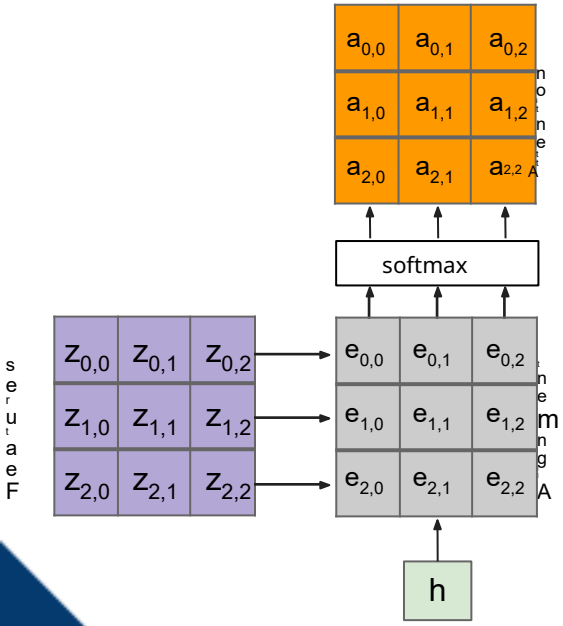
注意力 我们在图像描述中刚刚看到的

操作：
对齐：$e_{i,j} = f_{att}(h, z_{i,j})$



输入：特征：**z** （形状：H x W x D）查询：**h** （形状：D）

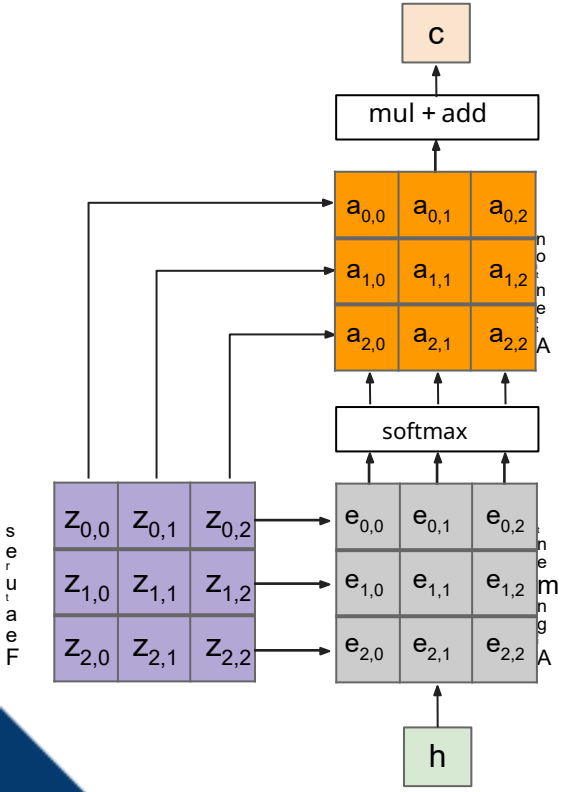
注意力 我们刚刚在图像描述中看到的



运算：
对齐：$e_{i,j} = f_{att}(h, z_{i,j})$ 注意力：
a = softmax(**e**)

输入：
特征：**z** （形状：H x W x D） 查询：**h** （形状：D）

注意力：我们刚刚在图像描述中看到的



输出：
上下文向量：**c** （形状：D）

运算：
对齐：$e_{i,j} = f_{att}(h, z_{i,j})$ 注意力：
a = softmax(**e**) 输出：**c**
$= \sum_{i,j} a_{i,j} z_{i,j}$

输入：
特征：**z** （形状：H x W x D） 查询：**h** （形状：D）

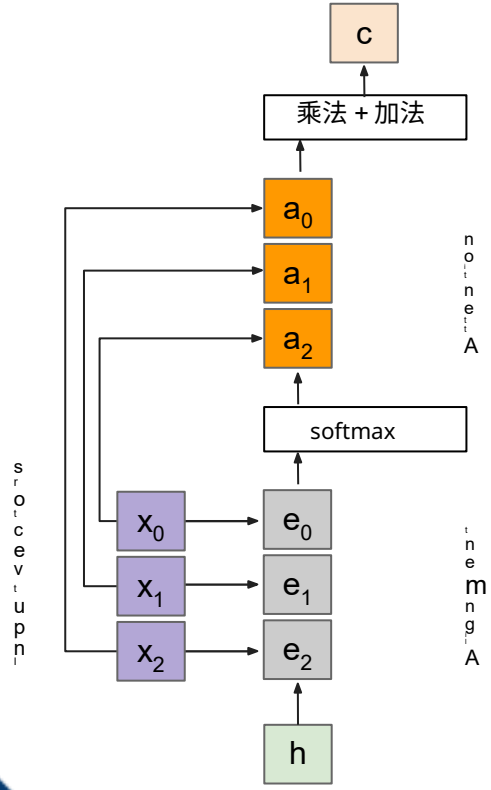
泛化
图像描述中的注意力

to

通用注意力层



通用注意力层



输出：
上下文向量：**c** （形状：D）

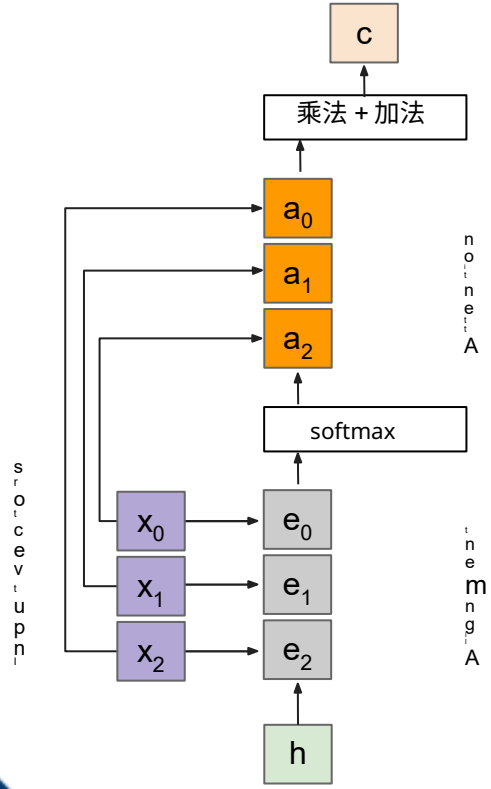
操作：
对齐：$e_i = f_{att}(h, x_i)$ 注意力：
$\mathbf{a} = \text{softmax}(\mathbf{e})$ 输出：$\mathbf{c} = \sum_i a_i x_i$

输入：
输入向量：**x** （形状：N x D）
查询：**h** （形状：D）

注意力操作是 **置换不变的。** - 不关心特征的顺序

- 将 H x W = N 展平为 N 个向量

通用注意力层



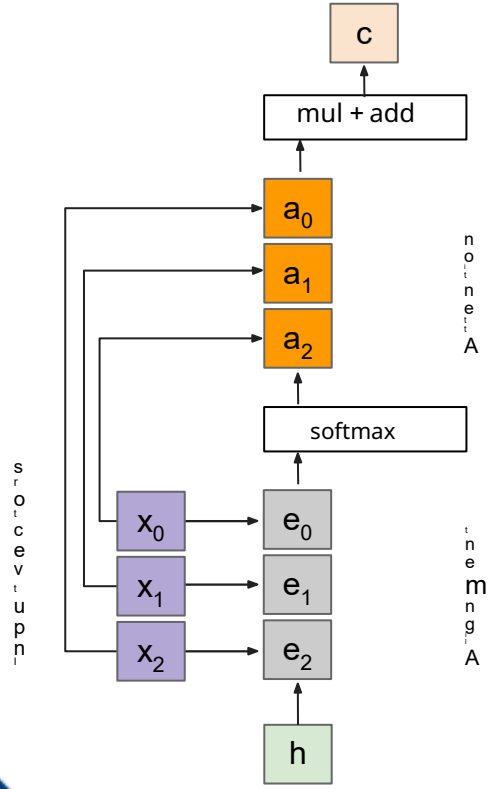
输出：上下文向量：**c**（形状：D）

运算：
对齐：$e_i = h \cdot x_i$ 注意力：**a** = softmax(**e**) 输出：$\mathbf{c} = \sum_i a_i x_i$

将 $f_{att}(.)$ 改为简单的点积
- 仅在与键和值变换技巧配合时效果良好（将在后面几张幻灯片中提及）

输入：输入向量：**x**（形状：N x D）查询：**h**（形状：D）

通用注意力层



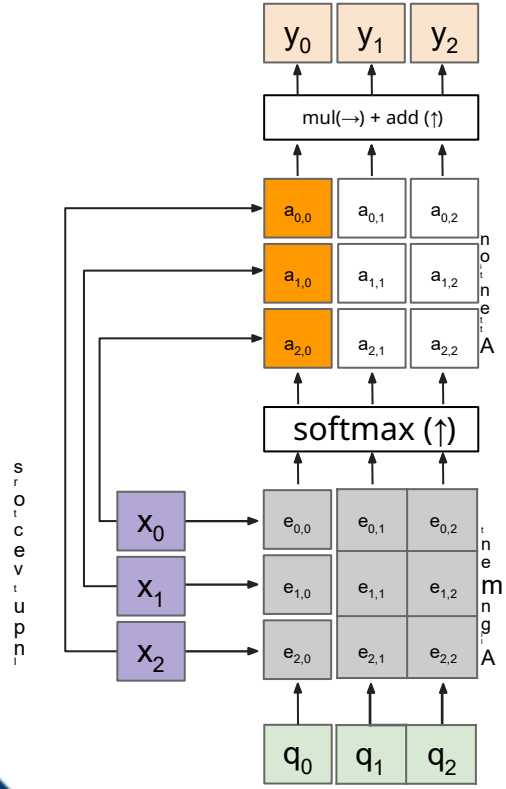
输出： 上下文向量：**c**（形状：D）

操作：
对齐：$e_i = h \cdot x_i / \sqrt{D}$ 注意力：**a** = softmax(**e**) 输出：**c** = $\sum_i a_i x_i$

将 $f_{att}(.)$ 改为一个 缩放后的 简单点积

输入： 输入向量：**x**（形状：N x D） 查询：**h**（形状：D）

通用注意力层



输出：

上下文向量：**y** （形状：D）

多个 查询向量 - 每个查询创建一个新的输出上下文向量

Op运算：

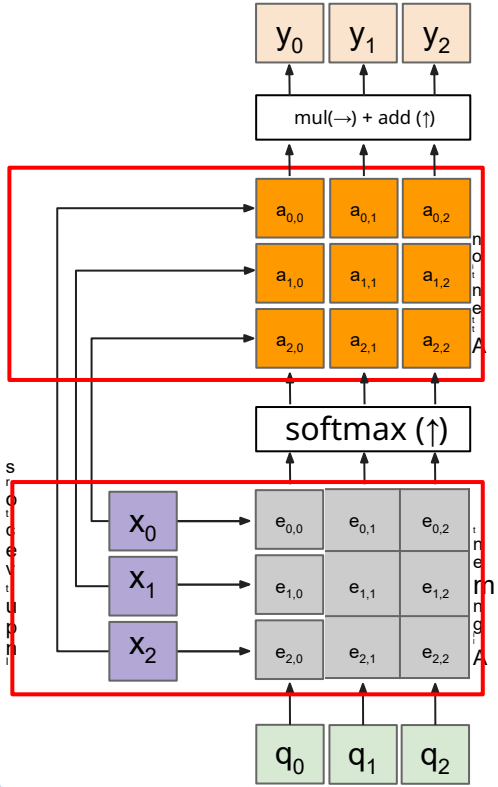
对齐：$e_{i,j} = q_j \cdot x_i / \sqrt{D}$ 注意力：**a** = softmax(**e**) 输出：$y_j = \sum_i a_{i,j} x_i$

多个查询向量

输入：

输入向量：**x** （形状：N x D）查询：**q** （形状：M x D）

通用注意力层



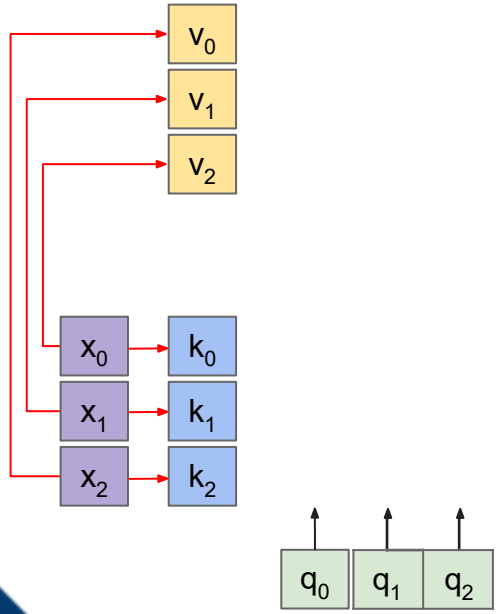
输出：
上下文向量：**y** （形状：D）

运算：
对齐：$e = q \cdot x / \sqrt{D_{i,j}}$ j i 注意力：**a** = softmax(**e**) 输出：$y_j = \sum_i a_{i,j} x_i$

注意，输入 向量既用于对齐，也用于注意力计算。- 我们可以通过在两个步骤之前分别添加一个不同的 FC 层，为层增加更多表达能力。

输入:
输入向量：**x** （形状：N x D） 查询：**q** （形状：M x D）

通用注意力层



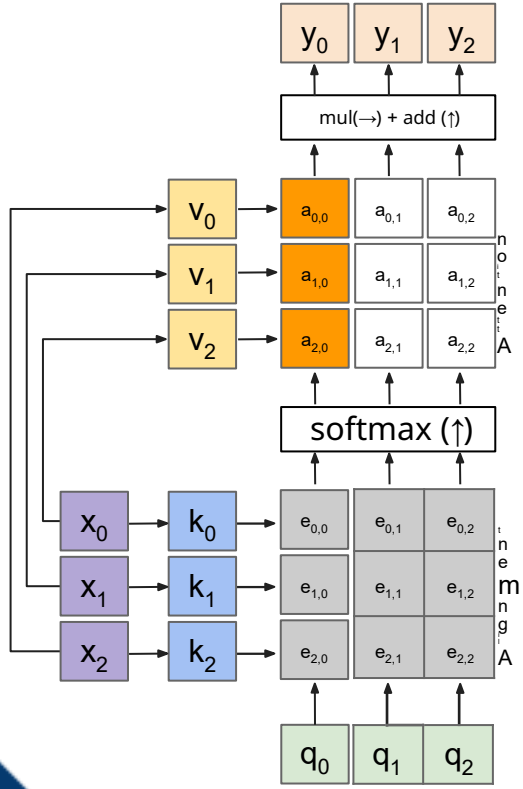
操作： 键向量： $k = xW_k$ 值向量： $v = xW_v$

输入： 输入向量：**x** （形状：N x D）查询：**q** （形状：M x D_k）

注意，输入的向量既用于对齐，也用于注意力计算。

- 我们可以通过在这两个步骤中的每一步之前添加一个不同的全连接（FC）层，为该层增加更多表达能力。

通用注意力层



输出： 上下文向量： **y**（形状 ：D_v）

输入和输出维度 现在可以随键和值 FC 层而变化

操作：

键向量： $k = xW_k$ 值向量： $v = xW_v$ 对齐：$e_{i,j} = q_j \cdot k_i / \sqrt{D}$

注意力： $a = \text{softmax}(e)$ 输出：$y_j = \sum_i a_{i,j} v_i$

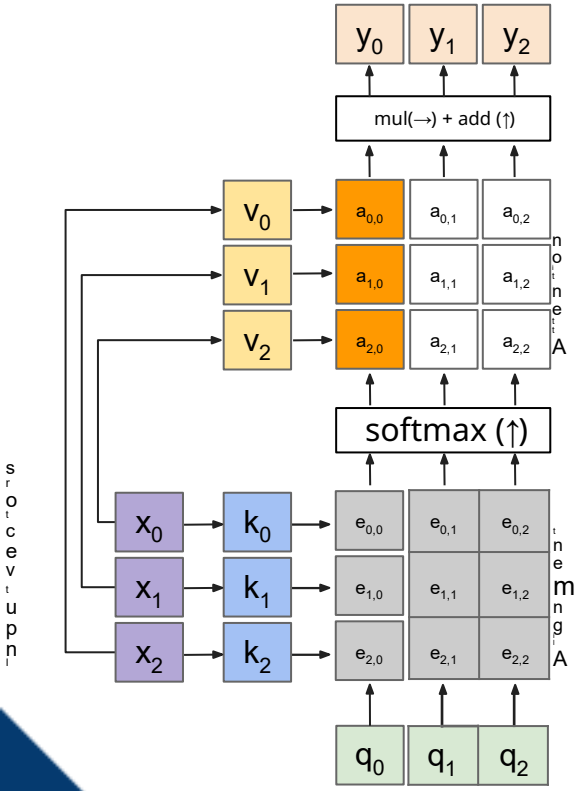
注意，输入向量同时用于对齐以及注意力计算。 - 我们可以通过在两个步骤之前各添加一个不同的 FC 层，为这一层增加更多表达能力。

输入：

输入向量： **x** （形状：N x D）

查询： **q** （形状：M x D_k）

通用注意力层



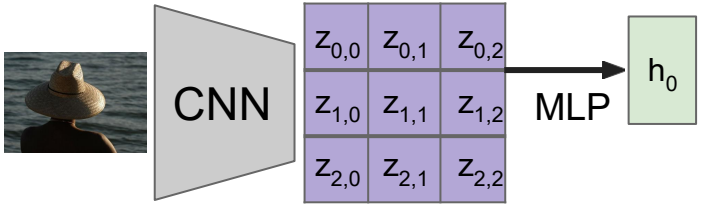
输出： 上下文向量： **y** （形状： D_v）

运算：
键向量： $\mathbf{k} = \mathbf{x}W_k$ 值向量： $\mathbf{v} = \mathbf{x}W^v$ $_{i,j\ j}$ 对齐： $e = q \cdot k_i / \sqrt{}$
D 注意力： **a** = softmax(**e**) 输出： $y_j = \sum_i a_{i,j} v_i$

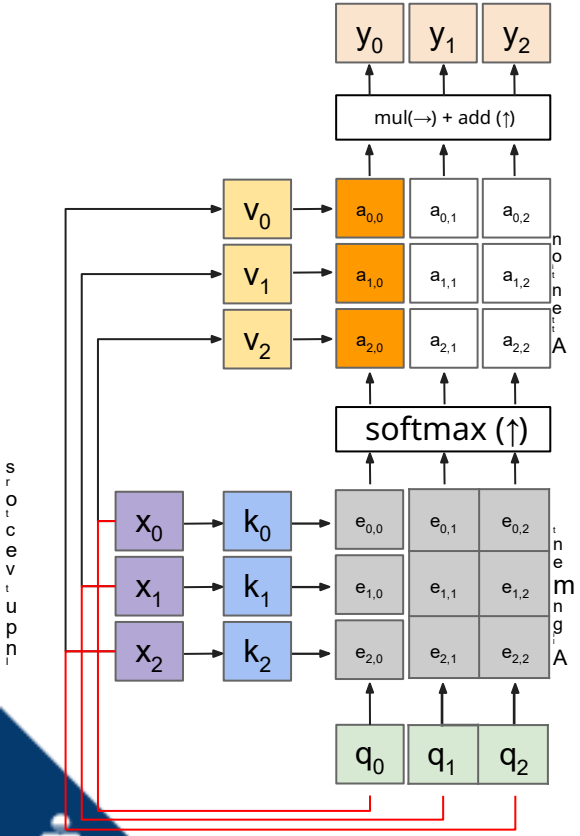
输入：
输入向量： **x** (形状：N x D) 查询： **q** (形状：M x D_k)

回顾一下，查询 向量是输入向量的函数

编码器： h = f (**z**)$_0$ w其中 **z** 是空间 CNN 特征 wf **(.)** 是一个 MLP



通用注意力层 ➡ 自注意力层



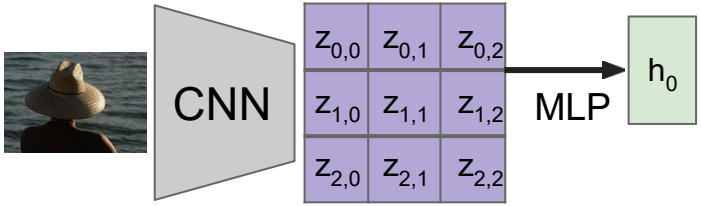
输出：
上下文向量：**y** （形状：D_v）

操作：
键向量： $k = xW_k$
值向量： $v = xW_v$
查询向量： $q = xW_q$
对齐： $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
注意力：**a** = softmax(**e**)
输出： $y_j = \sum_i a_{i,j} v_i$

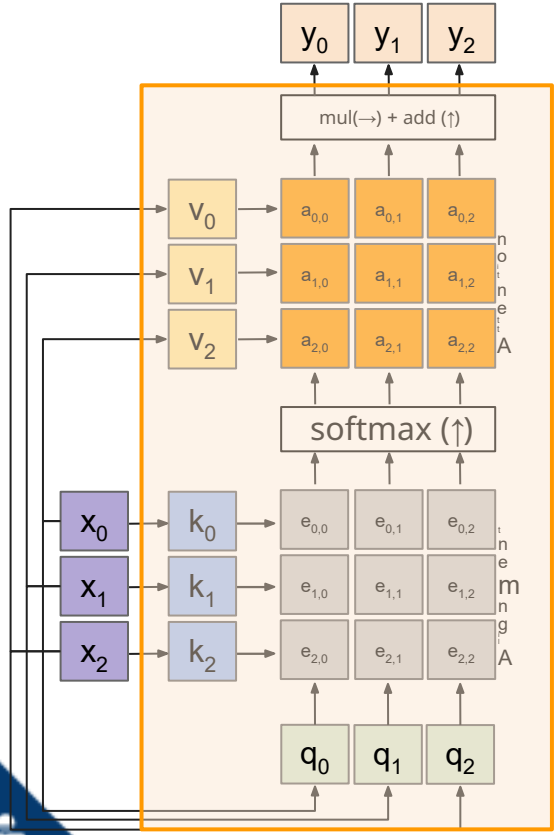
输入：
输入向量：**x** （形状：N x D）
查询： **q** （形状：M x D_k）

回想一下，查询向量是输入向量的函数

编码器：h = f (**z**)$_0$ **w**，其中 **z** 是空间 CNN 特征，**w**f (**.**) 是一个多层感知机（MLP）

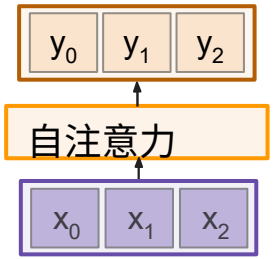


自注意力层 - 对输入集合进行注意力计算



输出：
上下文向量：**y**（形状：D_v）

操作：
键向量：$k = xW_k$ 值向量：$v = xW_v$ 查询向量：$q = xW_q$ 对齐：$e_{i,j} = q_j \cdot k_i / \sqrt{D}$ 注意力：$a = \text{softmax}(e)$ 输出：$y_j = \sum_i a_{i,j} v_i$



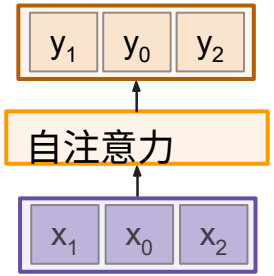
输入：
输入向量：**x**（形状：N x D）

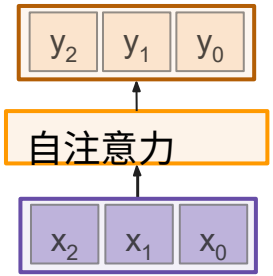
自注意力层——关注输入集合

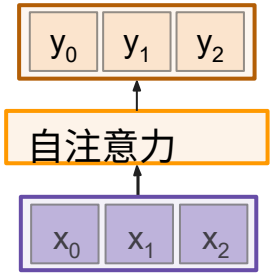
s

置换不变

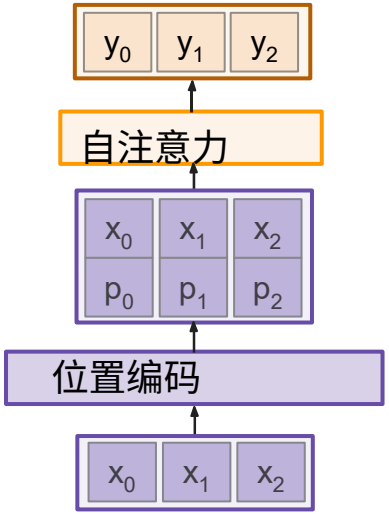
问题： 我们如何对像语言或空间有序的图像特征这样的有序序列进行编码？







位置编码



将特殊位置编码 p_j 拼接到每个输入向量 x_j

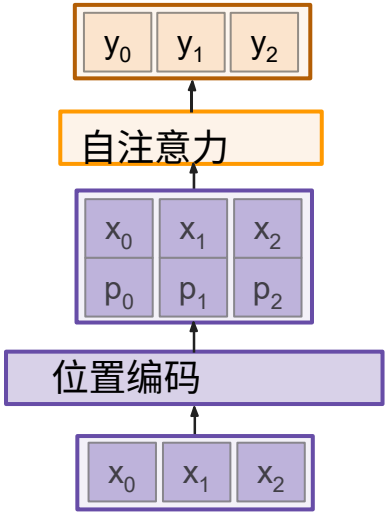
我们使用一个函数 pos：$N \rightarrow R^d$
将向量的位置 j 处理为 d 维向量

因此，$p_j = pos(j)$

关于$pos(.)$ 的期望特性：

1. 它应为每个时间步（单词在句子中的位置）输出一个 **唯一的** 编码 **2. 距离** 在任意两个时间步之间应在不同长度的句子中保持一致。3. 我们的模型应无需任何努力即可泛化到 **更长的** 句子。其取值应当有界。4. 它必须是 **确定性的**。

位置编码



拼接特殊位置编码p_j 到每个输入向量x_j

我们使用一个函数 *pos*：$N \rightarrow R^d$ 将向量的位置 j 处理为 d 维向量

因此，$p_j = pos(j)$

关于 *pos*(.) 的选项

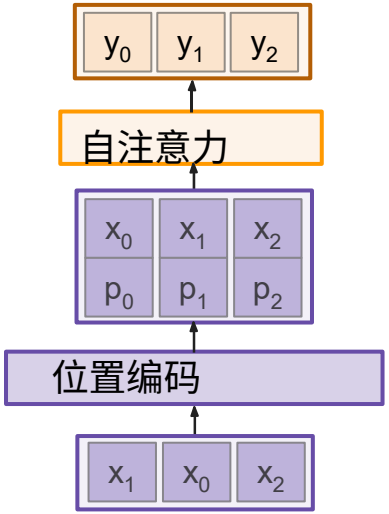
1. 学习查找表：○ 学习用于 *pos*(t) 的参数，其中 t ε [0, T) ○ 查找表包含 T xd 个参数。

关于 *pos*(.) 的期望特性：

1. 它应为每个时间步（单词在句子中的位置）输出一个 **唯一**编码。**2. 距离**在任意两个时间步之间应保持一致，且在不同长度的句子中应保持一致。3. 我们的模型应无需付出任何努力即可泛化到 **更长**的句子。其值应有界。4. 它必须是 **确定性的**。

Vaswani et al, “Attention is all you need”, NeurIPS 2017

位置编码



拼接特殊位置编码 p_j 到每个输入向量 x_j

我们使用一个函数 *pos*：$N \rightarrow R^d$
将向量的位置 j 处理为 d 维向量

因此，$p_j = pos(j)$

用于 *pos*(.) 的选项

1. 学习一个查找表：
   - ◯ 学习用于 *pos*(t) 的参数，其中 t ε [0, T) ○ 查找表包含 T xd 个参数。

2. 设计满足期望条件的固定函数

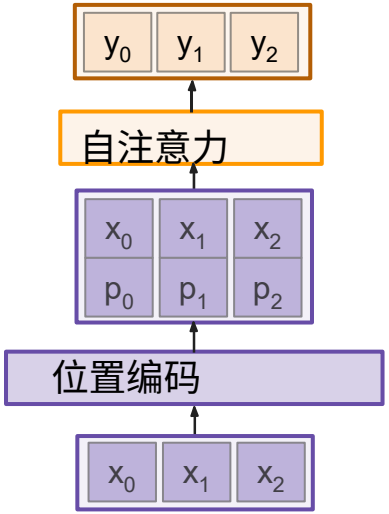
○**p(t)** =，其中

$$\begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_d$$

$$\omega_k = \frac{1}{10000^{2k/d}}$$

Vaswani et al, “Attention is all you need”, NeurIPS 2017

位置编码



将特殊位置编码 p_j 连接到每个输入向量 x_j

我们使用一个函数 *pos*：$N \rightarrow R^d$ 来处理向量中位置 j，将其转换为一个 d 维向量

因此，$p_j = pos(j)$

pos(.) 的选项

1. 学习一个查找表：
   - ◯ 学习用于 *pos*(t) 的参数，其中 t ε [0, T) ○ 查找表包含 T xd 个参数。

2. 设计一个满足期望特性的固定函数
   - ○

$$\mathbf{p(t)} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_d$$

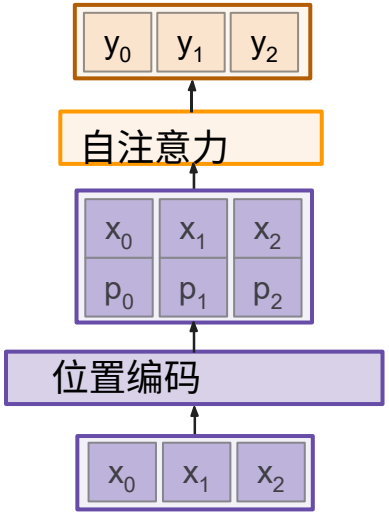
直觉：

0：	0	0	0	0	8：	1	0	0	0
1：	0	0	0	1	9：	1	0	0	1
2：	0	0	1	0	10：	1	0	1	0
3：	0	0	1	1	11：	1	0	1	1
4：	0	1	0	0	12：	1	1	0	0
5：	0	1	0	1	13：	1	1	0	1
6：	0	1	1	0	14：	1	1	1	0
7：	0	1	1	1	15：	1	1	1	1

其中 $\omega_k = \frac{1}{10000^{2k/d}}$

Vaswani 等人，“注意力就是你所需要的一切”，NeurIPS 2017

位置编码



将特殊位置编码 p_j 连接到每个输入向量 x_j

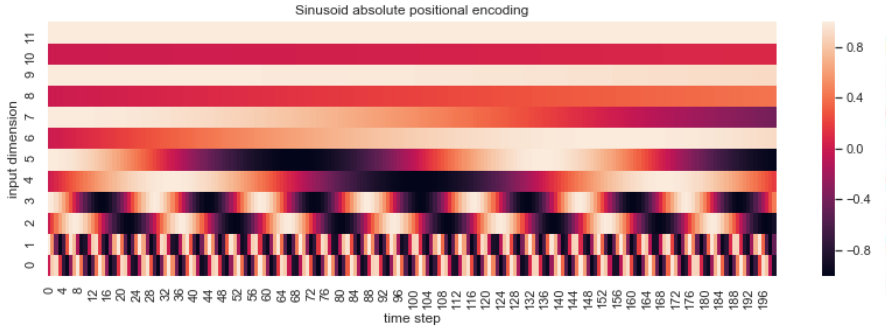
我们使用一个函数 $pos: N \rightarrow R^d$ 用于处理位置 j 的向量并将其转换为 d 维向量

因此，$p_j = pos(j)$

用于 $pos(.)$的选项

1. 学习一个查找表：
   - ○ 学习用于 $pos(t)$ 的参数，其中 $t \varepsilon [0, T)$ ○ 查找表包含 T xd 个参数。

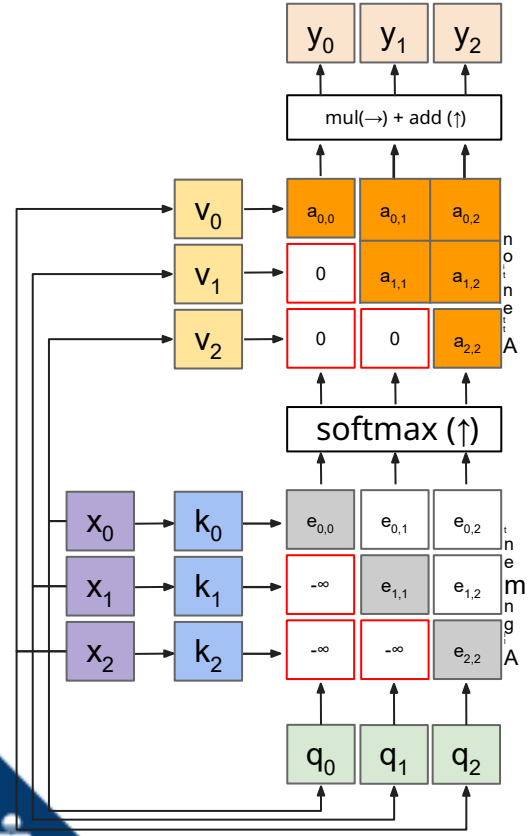
2. 设计一个固定函数



$$\begin{bmatrix} \sin(\omega_{d/2}.t) \\ \cos(\omega_{d/2}.t) \end{bmatrix}_d \quad \text{其中} \quad \omega_k = \frac{1}{10000^{2k/d}}$$

Vaswani et al, “Attention is all you need”, NeurIPS 2017

掩码自注意力层



输出：上下文向量：**y** （形状：D_v）

运算：

键向量： $k = xW_k$ 值向量： $v = xW_v$ 查询向量： $q = xW_q$

对齐： $e_{i,j} = q_j \cdot k_i / \sqrt{D}$ 注意力： $a = \text{softmax}(e)$ 输出： $y_j = \sum_i a_{i,j} v_i$

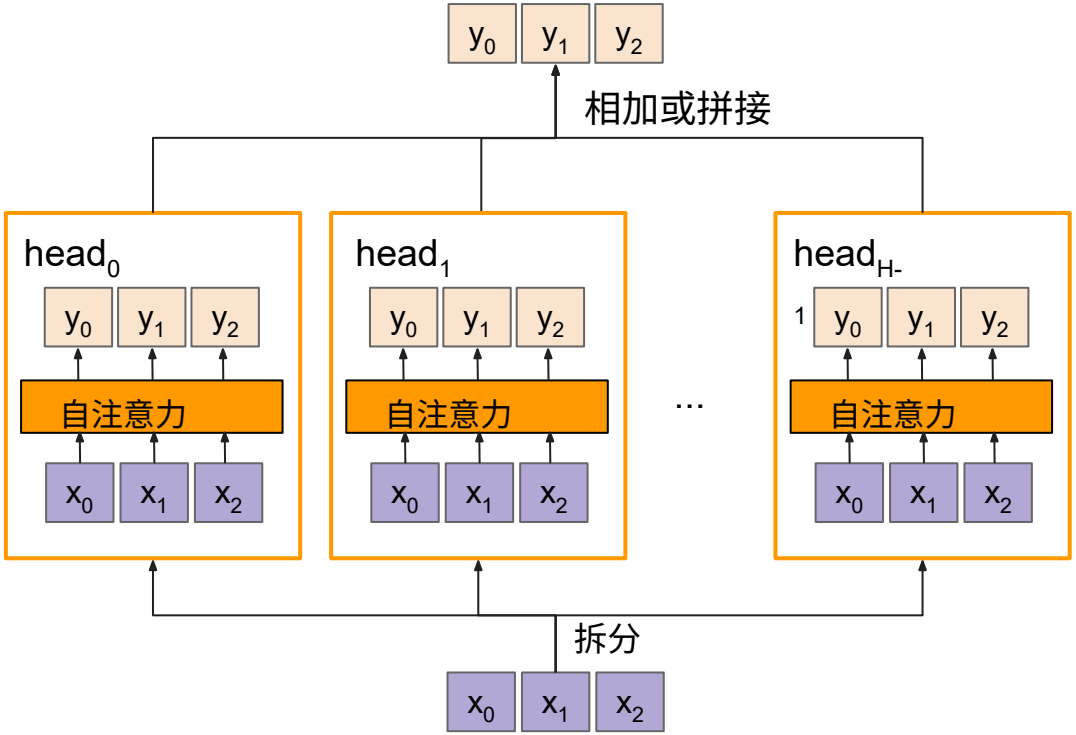
输入：

输入向量：**x** （形状：N x D）

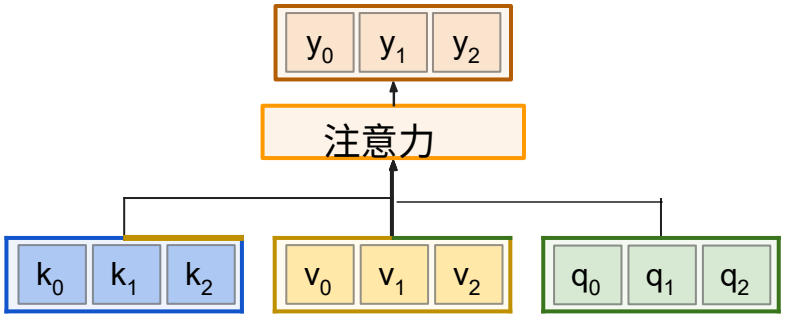
- 防止向量关注未来向量。
- 手动将对齐分数设为 -infinity

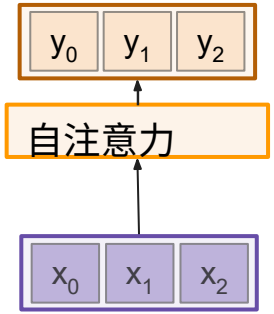
多头自注意力层

- 多个自注意力头并行



一般注意力 与自注意力





比较 RNN 与 Transformer

RNNs

(+) LSTM 对长序列的处理效果相当不错。 (-) 期望输入为有序序列 (-) 顺序计算：后续隐藏状态只能在前面的隐藏状态计算完成后才能计算。

Transformer：

(+) 擅长处理长序列。每次注意力计算都会查看所有输入。 (+) 可作用于无序集合，也可作用于带位置编码的有序序列。 (+) 并行计算：所有输入的所有对齐分数和注意力分数都可以做到

in

并行。 (-) 需要大量内存：N x M 对齐和注意力缩放器需要计算

ated

并存储用于单个自注意力头。（但 GPU 正变得更大更好）



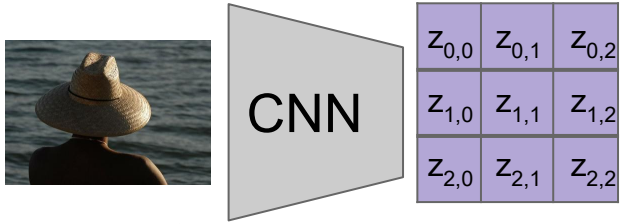
大纲

- **结合 RNN 的注意力**- 在计算机视觉中 - 在自然语言处理中
- **通用注意力层**
  - 自注意力 - 位置编码 - 掩码注意力 - 多头注意力
- **Transformer**



使用 transformers 进行图像描述

输入： 图像 **I** **输出：** 序列 **y** = y_1, y_2,..., y_T



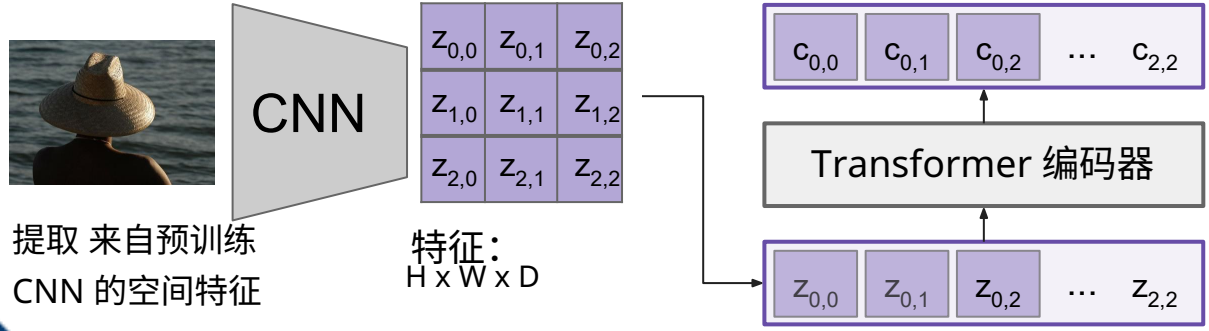
提取 预训练 CNN的空间特征

特征：
H x W x D

使用 Transformer 的图像描述

输入：图像 **I** **输出**：序列 **y** = y_1, y_2,..., y_T

编码器：**c** = Tw(**z**) 其中 **z** 是空间 CNN 特征 Tw(**.**) 是 Transformer 编码器



提取 来自预训练 CNN 的空间特征

特征：H x W x D

使用 transformers 的图像描述

输入：图像 **I** **输出**： 序列 **y** = y_1, y_2,..., y_T

解码器：$y_t = T_D(\mathbf{y}_{0:t-1}, \mathbf{c})$ 其中 T_D(.) 是 Transformer 解码器

编码器：**c** = T_W(**z**) 其中 **z** 是空间 CNN 特征 T_W(.) 是 Transformer 编码器



CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

提取 空间特征来自一个预训练 CNN

特征：
H x W x D

$c_{0,0}$ $c_{0,1}$ $c_{0,2}$ … $c_{2,2}$

Transformer 编码器

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$ … $z_{2,2}$

穿着的人 hat [END]

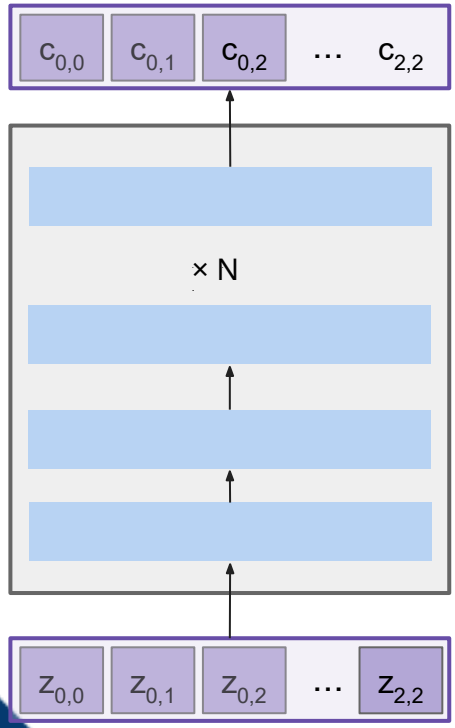
y_1 y_2 y_3 y_4

Transformer 解码器

y_0 y_1 y_2 y_3

[START] 人 穿着 hat

该 Transformer 编码器块



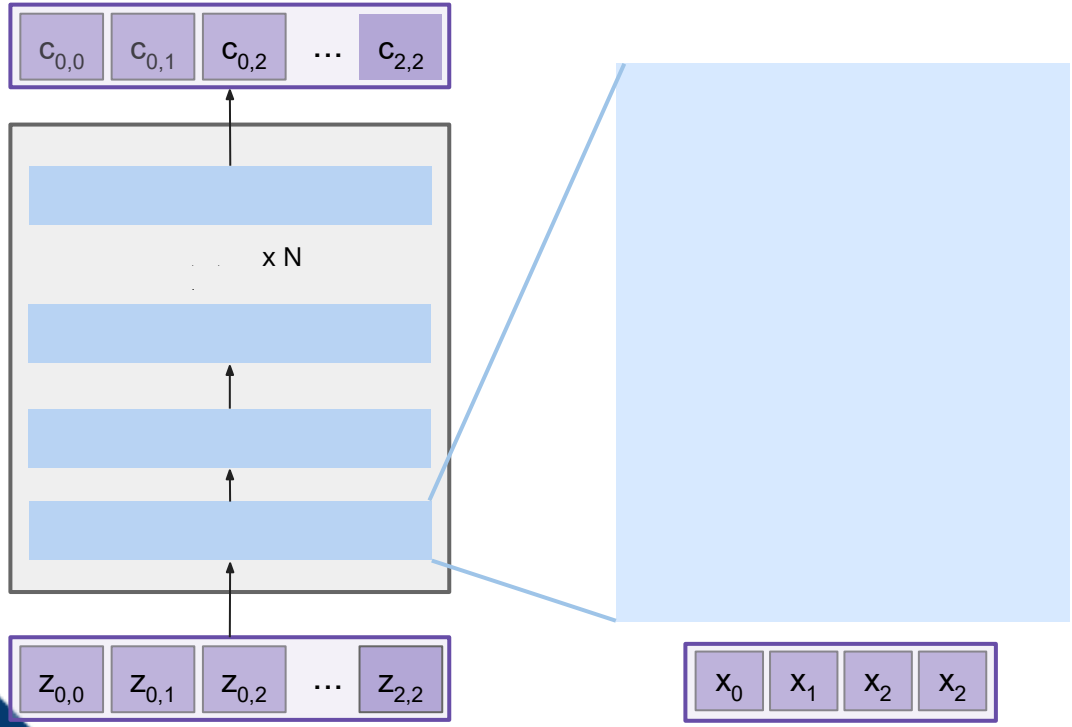
Transformer encoder

由 N 个编码器块组成。

在 vaswani et al. 中，N = 6，D_q = 512

Vaswani 等人，“Attention is all you need”，NeurIPS 2017

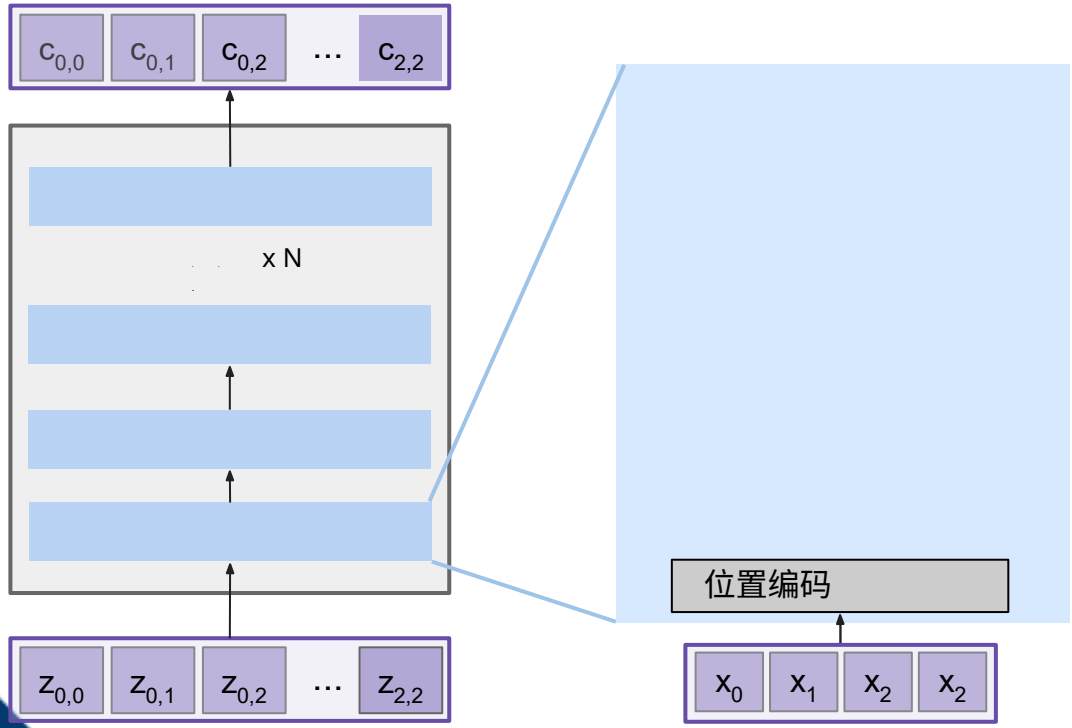
该 Transformer 编码器块



让我们深入探讨一个编码器块

Vaswani 等人，“Attention is all you need”，NeurIPS 2017

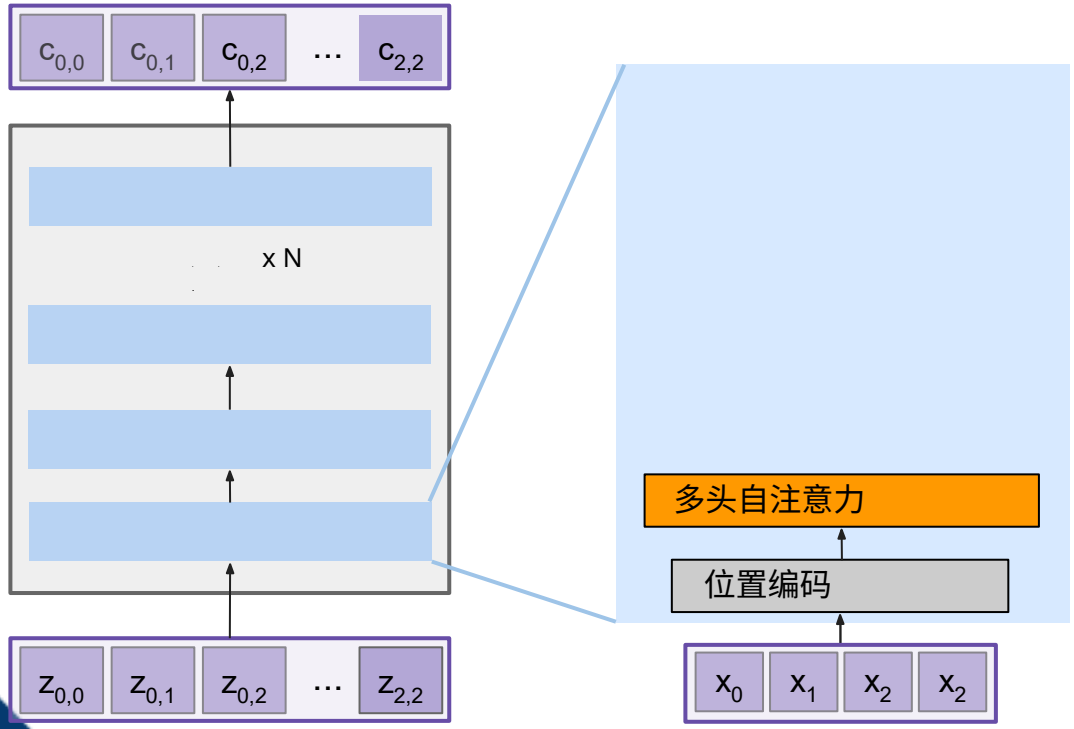
该 Transformer 编码器块



添加位置编码

Vaswani 等人，“Attention is all you need”，NeurIPS 2017

该 Transformer 编码器块

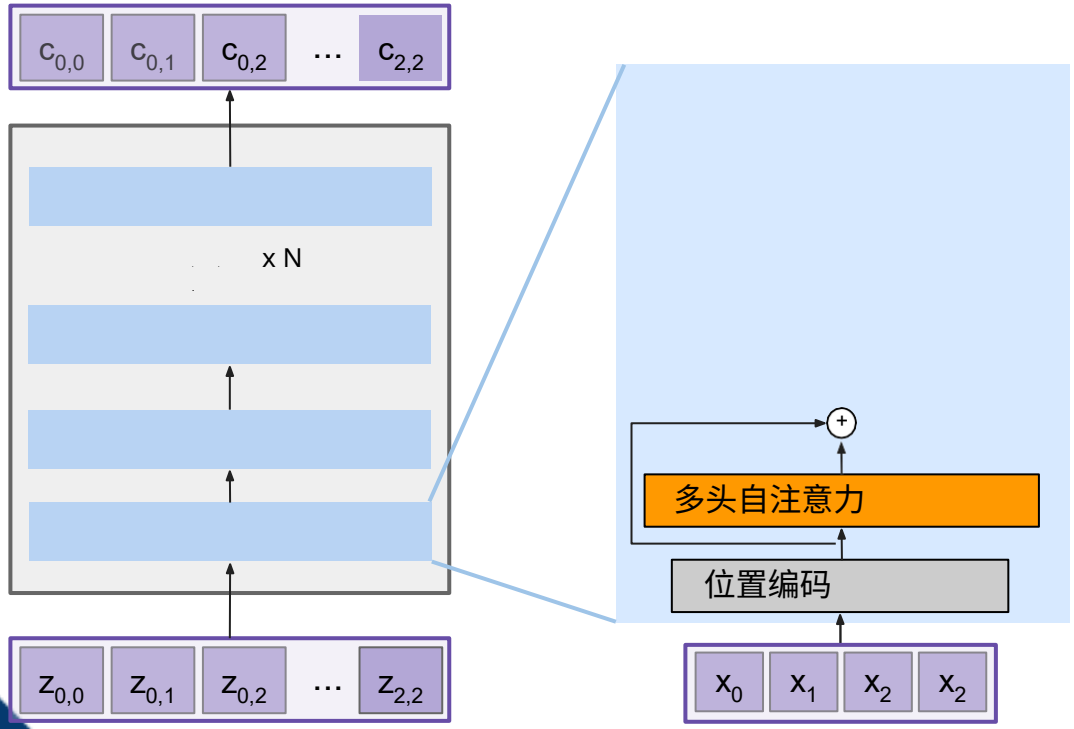


注意力关注所有向量

添加位置编码

Vaswani 等，“Attention is all you need”，NeurIPS 2017

Transformer 编码器块



残差连接

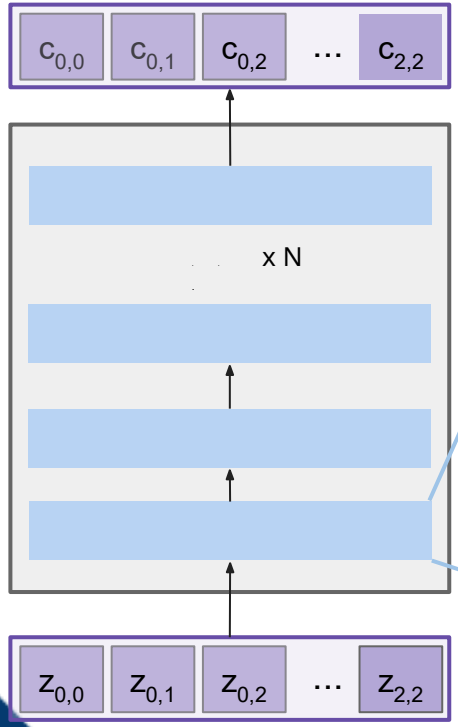
注意力机制关注所有向量

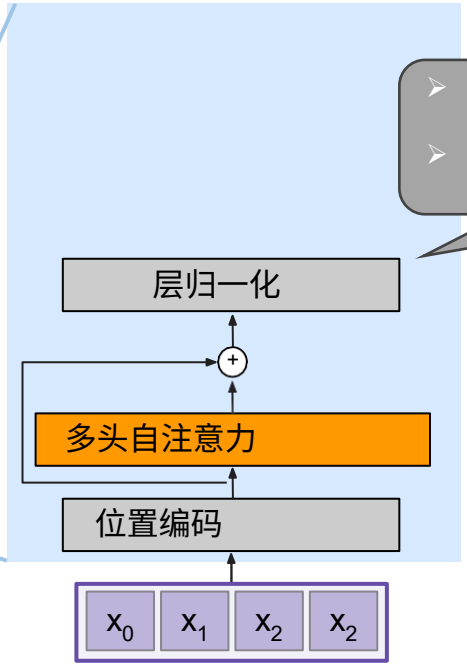
添加位置编码

Vaswani 等人，“Attention is all you need”，NeurIPS 2017

redocnerem rorsnarT

Transformer 编码器块





- BatchNorm：在批次维度上进行归一化（同一特征在不同样本间）
- LayerNorm：跨特征进行归一化（在单个样本内）

LayerNorm 分别作用于每个向量

残差连接

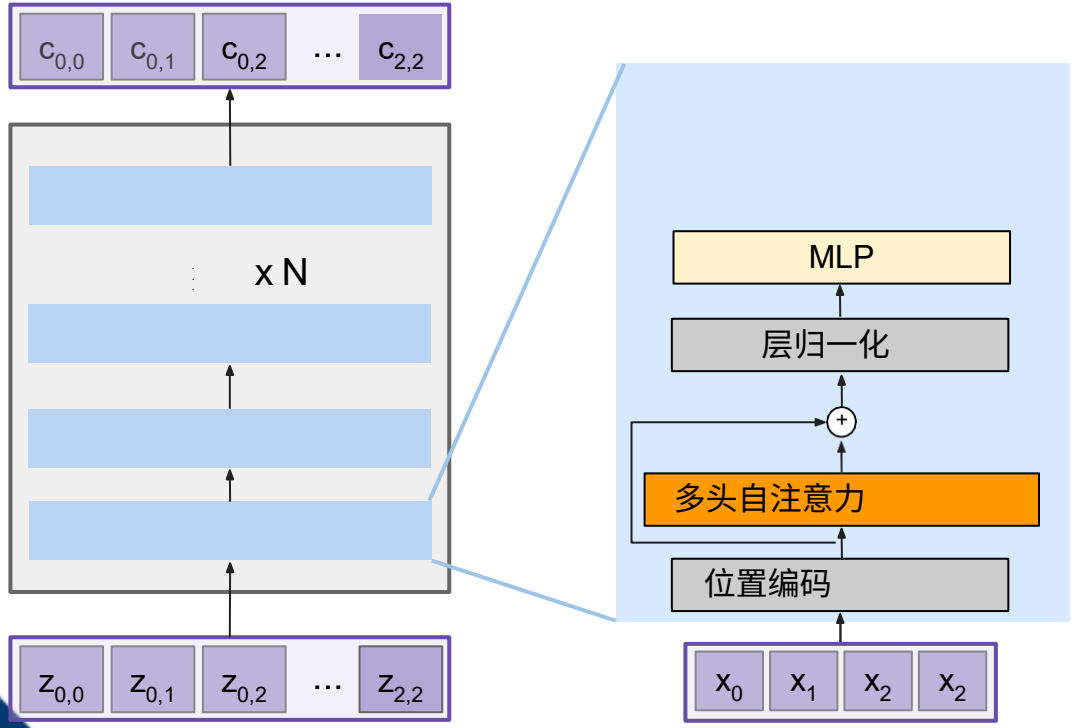
注意力机制对所有向量进行注意力计算

添加位置编码

Vaswani 等，“Attention is all you need”，NeurIPS 2017

redocnerem rorsnarT

Transformer 编码器块



对每个向量分别应用 MLP

对每个向量分别进行 LayerNorm

残差连接

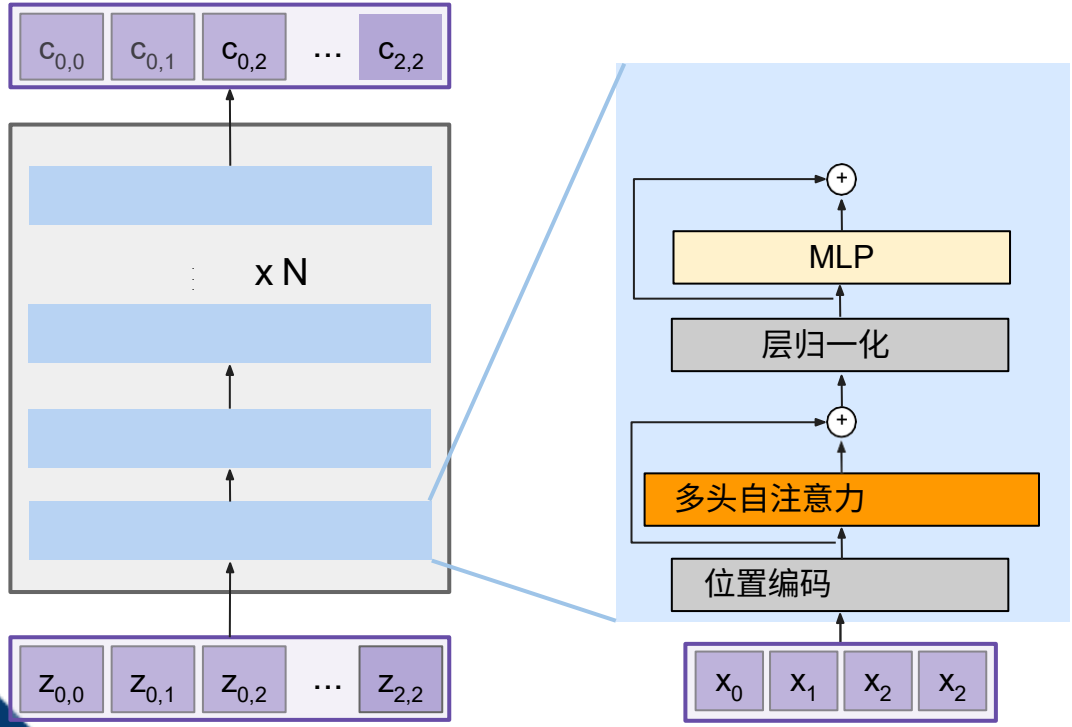
注意力机制作用于所有向量

添加位置编码

Vaswani et al, “Attention is all you need”, NeurIPS 2017

redocnerem rorsnarT

该 Transformer 编码器块



残差连接

对每个向量单独应用 MLP；对每个向量单独应用 LayerNorm；注意力机制作用于所有向量

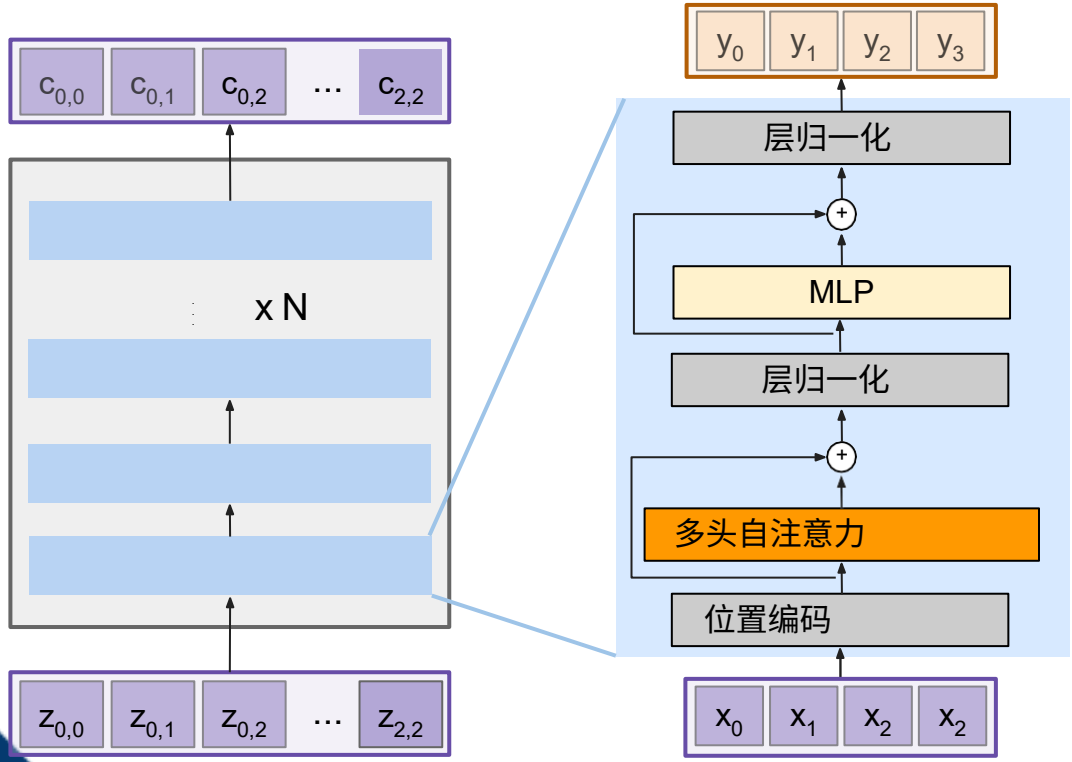
残差连接

添加位置编码

Vaswani 等，“Attention is all you need”，NeurIPS 2017

redocnerem rorsnarT

Transformer 编码器块



Transformer 编码器块：

输入：向量集合 **x** **输出**：向量集合 **y**

自注意力是向量之间唯一的交互。

层归一化和 MLP 对每个向量独立操作。

高度可扩展、高度可并行化，但内存占用高

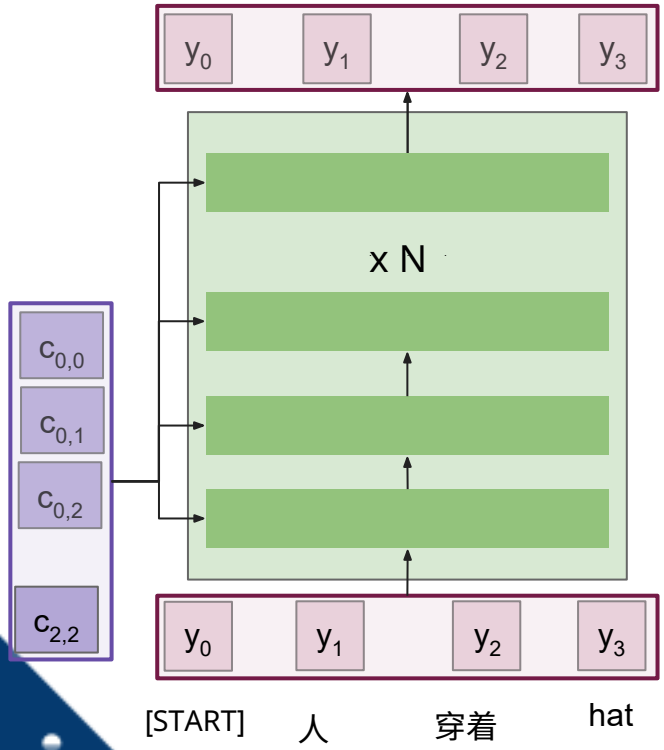
用法。

Vaswani 等人，“Attention is all you need”，NeurIPS 2017

redocnerem rorsnarT

该Transformer 解码器块

戴着帽子的人 [END]

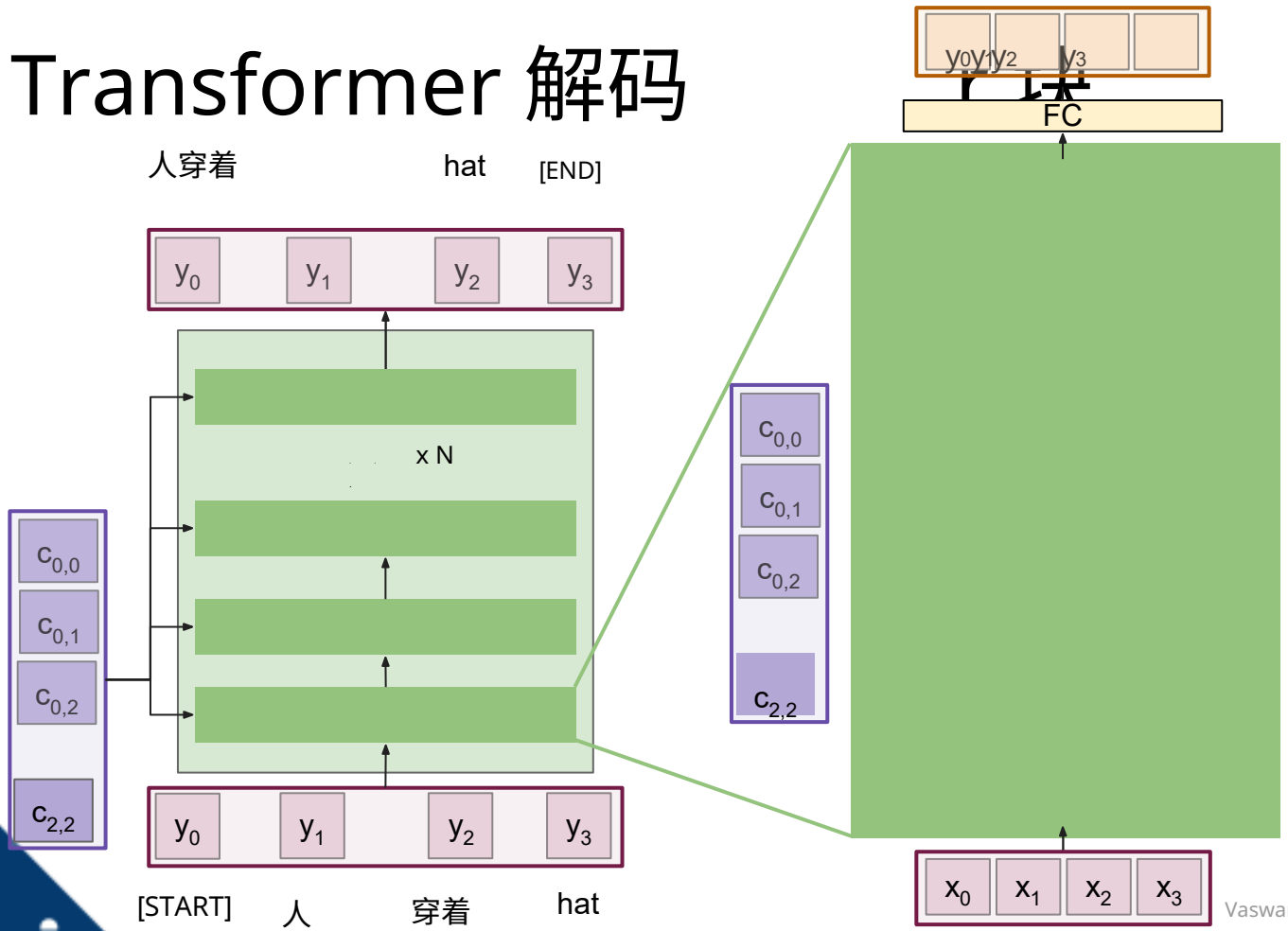


由 N 个解码器块组成。

在 Vaswani 等人的论文中，N = 6, D_q = 512

Vaswani et al, “Attention is all you need”, NeurIPS 2017

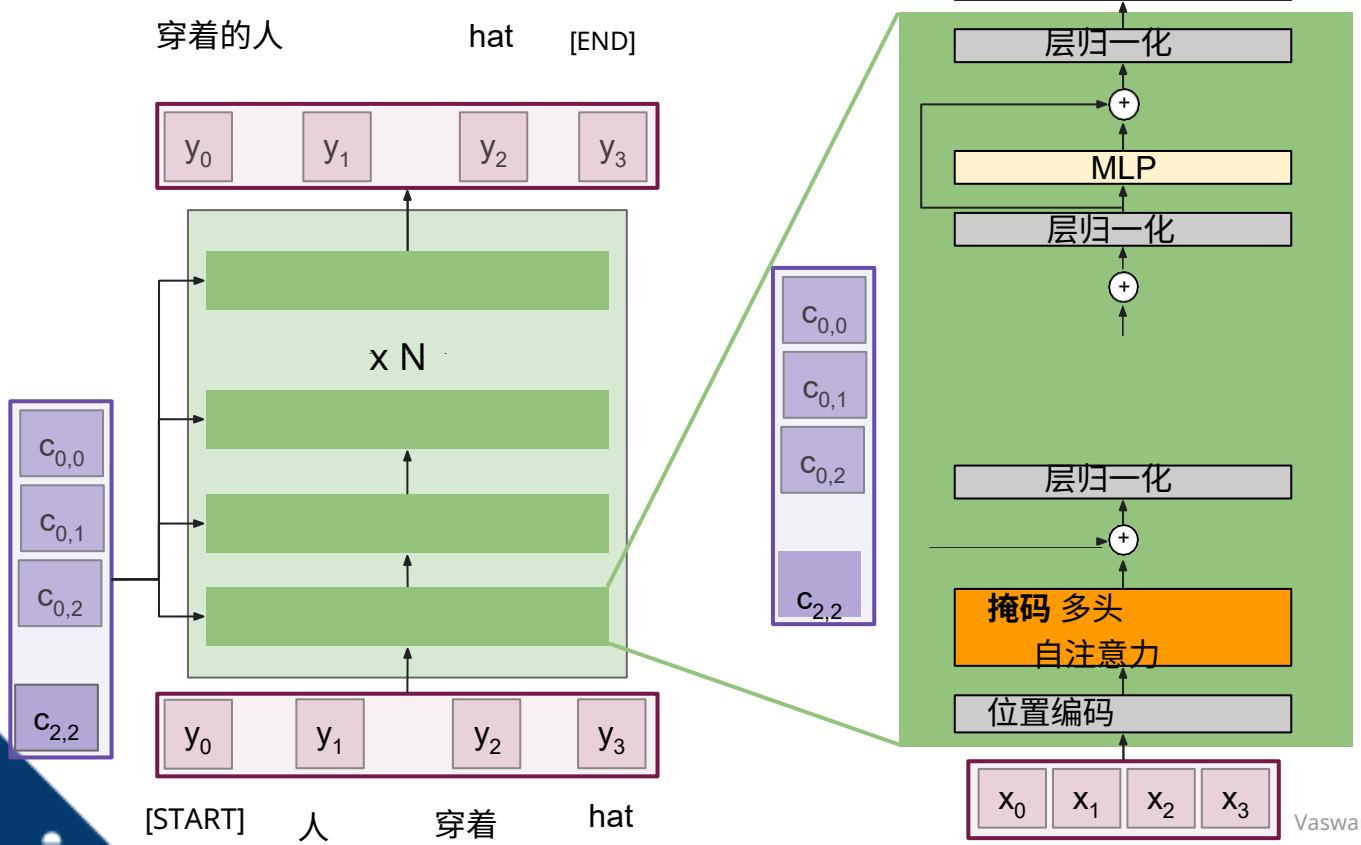
Transformer 解码



让我们深入探讨
Transformer 解码器块

Vaswani et al, “Attention is all you need”, NeurIPS 2017

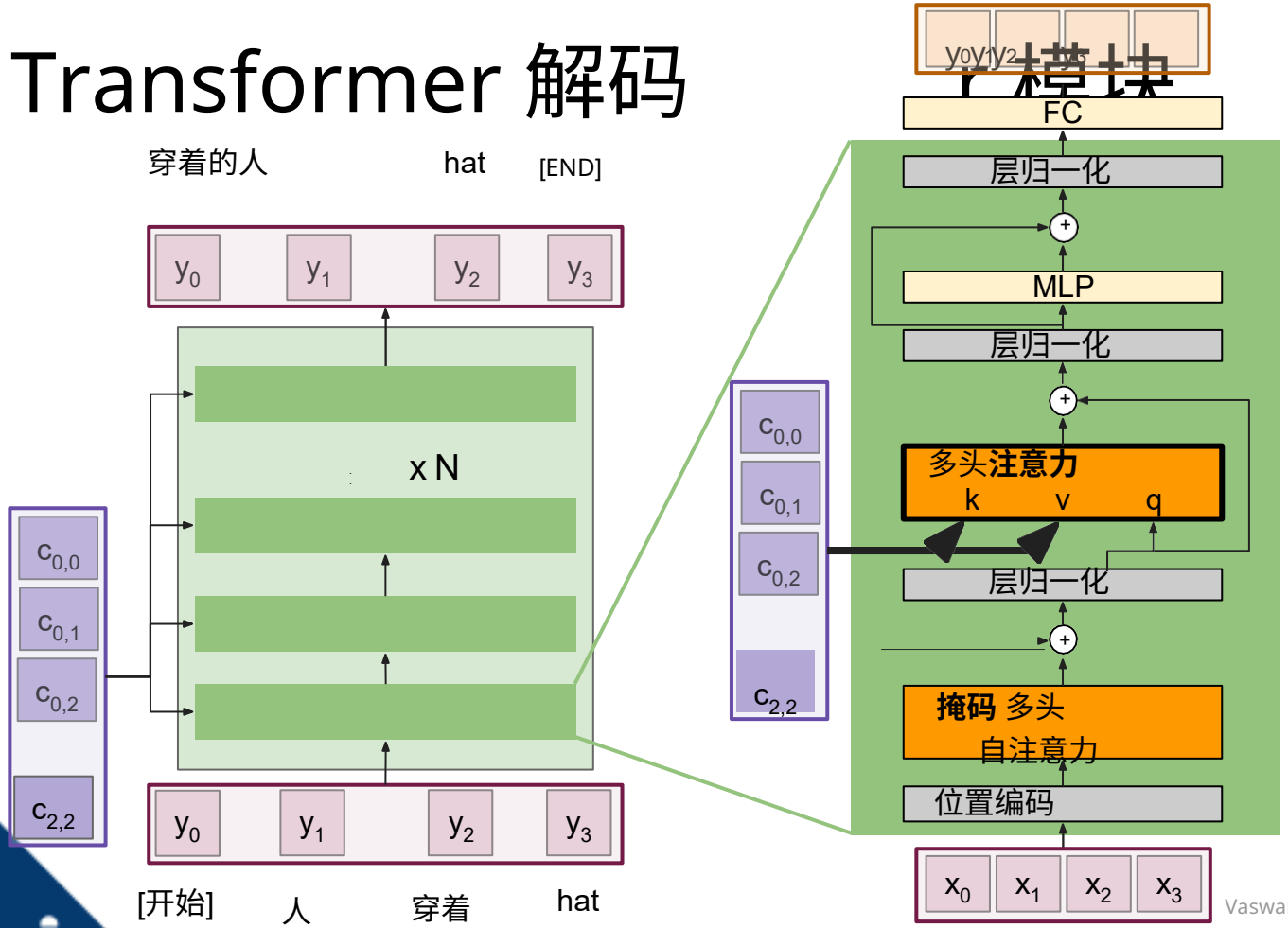
Transformer 解码



网络中大部分与 Transformer 编码器相同。

Vaswani et al, “Attention is all you need”, NeurIPS 2017

Transformer 解码

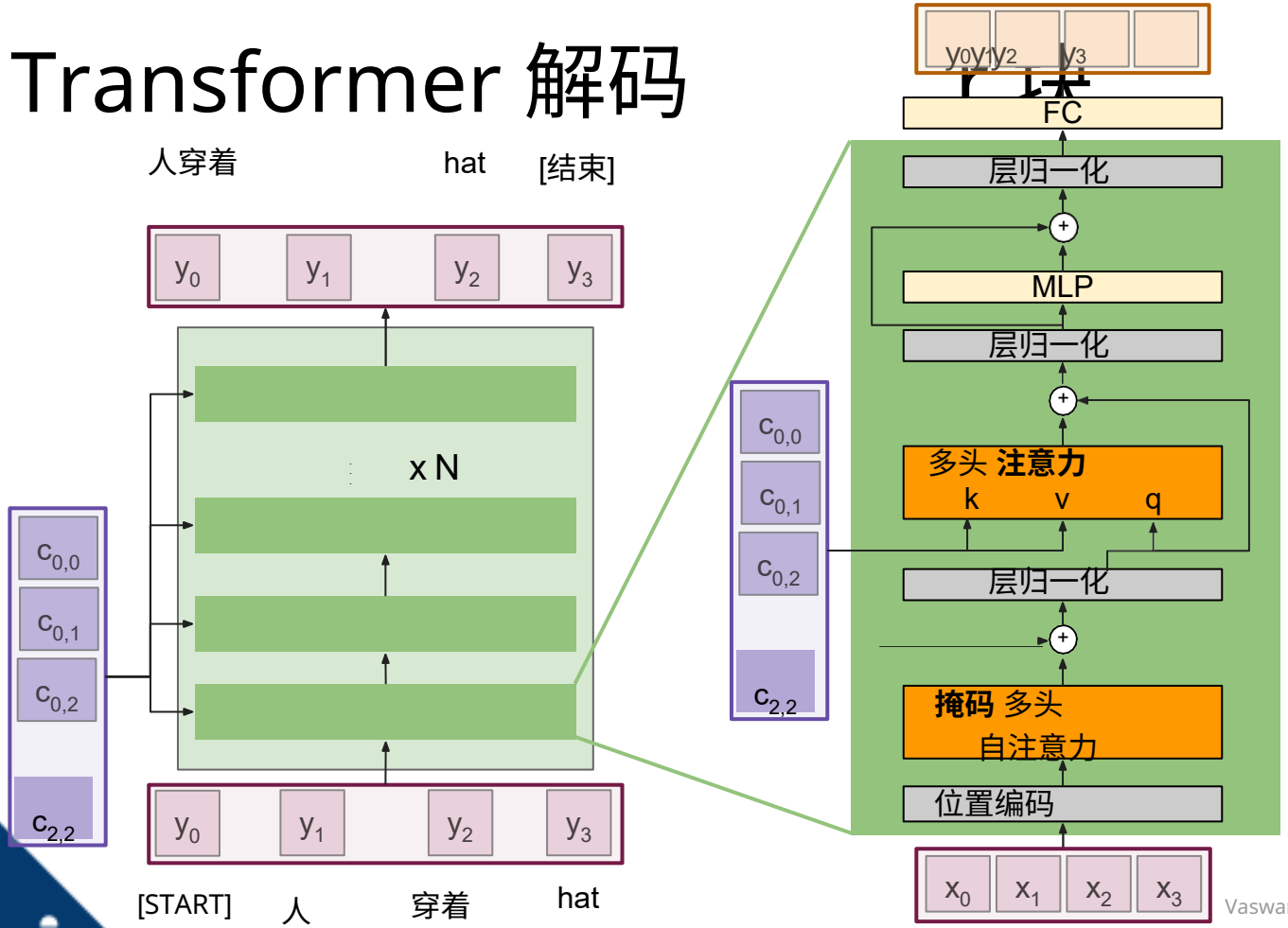


多头注意力块关注 Transformer 编码器的输出。

对于图像描述任务，这就是我们将图像特征注入解码器的方式。

Vaswani et al, “Attention is all you need”, NeurIPS 2017

Transformer 解码



Transformer 解码器块：

输入：向量集合 **x**和上下文向量集合 **c**。**输出**：向量集合 **y**。

掩码自注意力仅与过去的输入交互。

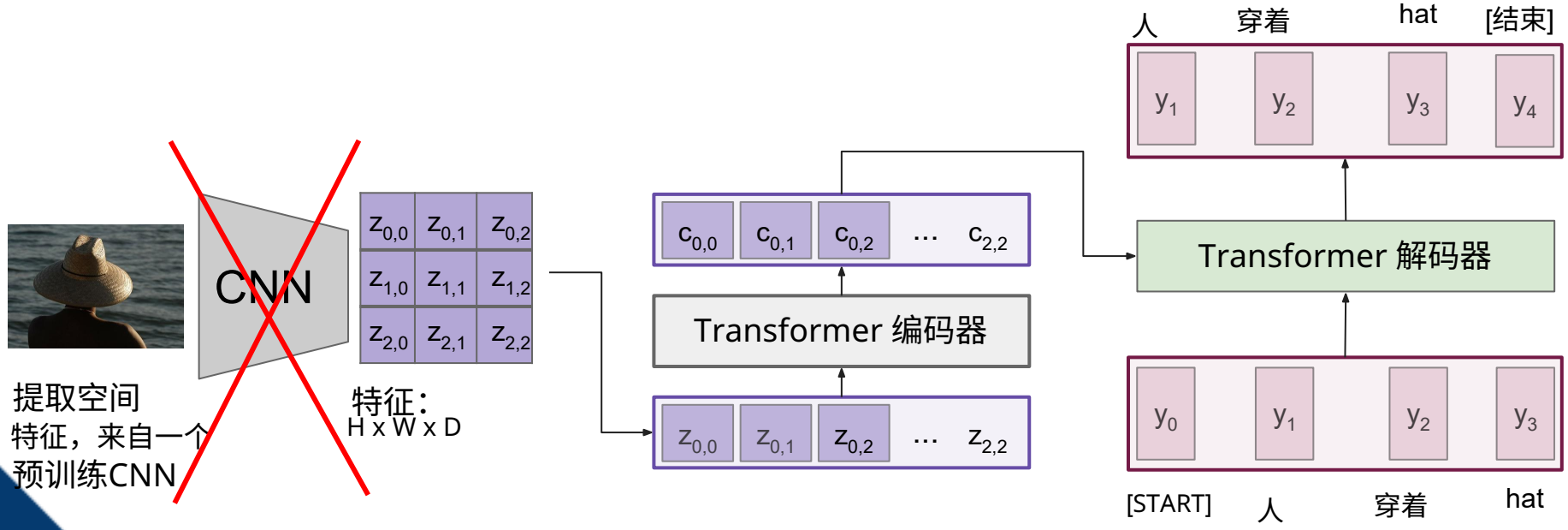
多头注意力块并非自注意力。它会对编码器输出进行注意力计算。

高度可扩展、高度可并行化，但内存占用高。

Vaswani et al, “Attention is all you need”, NeurIPS 2017

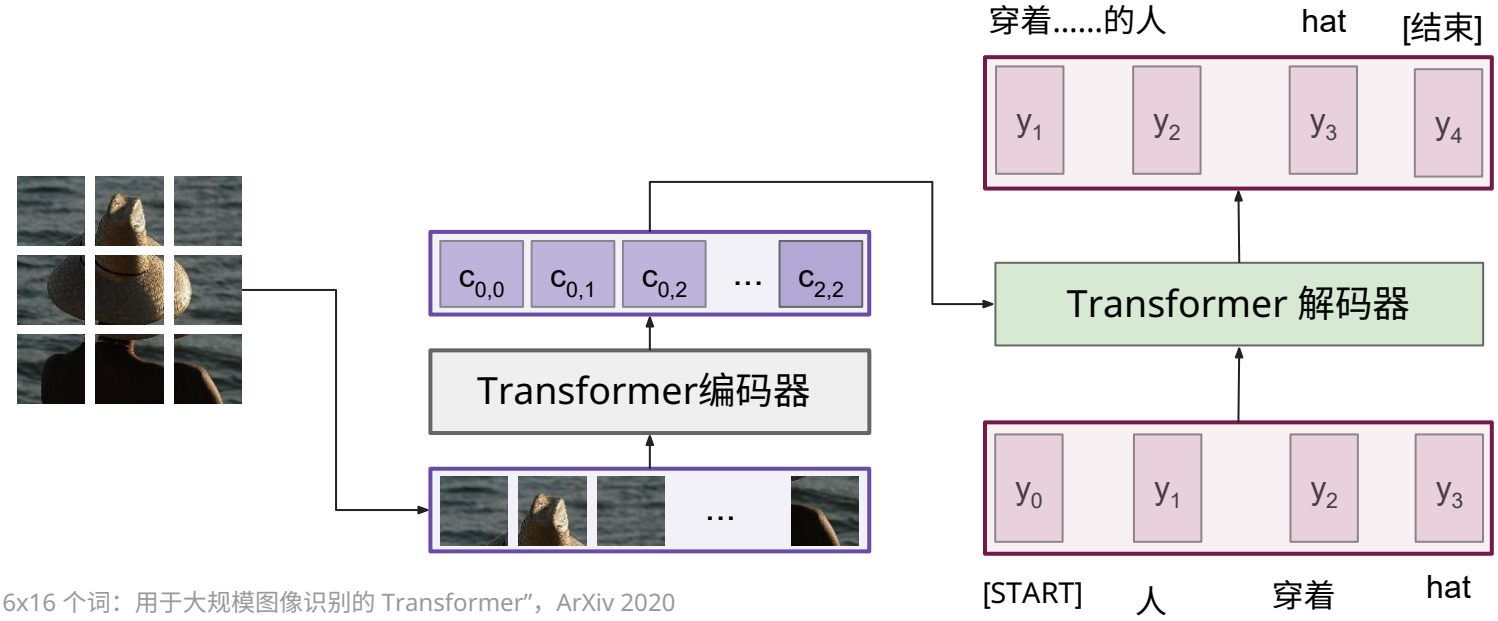
使用Transformer进行图像字幕生成

- **也许我们根本不需要卷积?**



使用仅Transformer进行图像描述

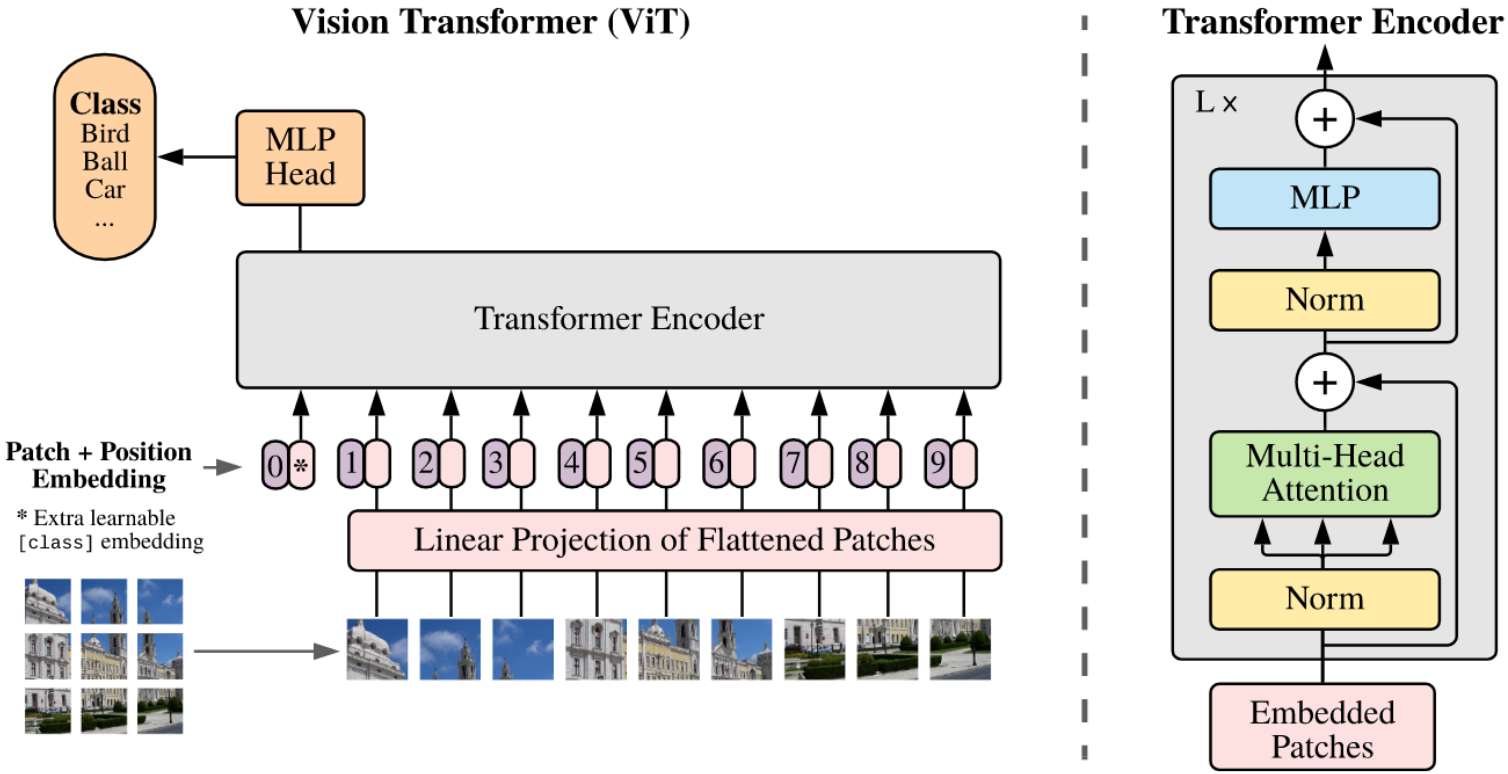
- **从像素到语言的Transformer**



Dosovitskiy 等，“一张图像值 16x16 个词：用于大规模图像识别的 Transformer”，ArXiv 2020

Colab 链接指向视觉 Transformer 的实现

视觉 Transformer



总结

- 向 RNNs 添加**注意力**，使它们能够在每个时间步“关注”输入的不同部分

- **通用注意力层** 是一种新型的层，可用于设计新的神经网络架构

- **Transformers** 是一种使用**自注意力** 以及 la 的层

  yer 归一化。

  ○ 它具有高度**可扩展性**，并且高度**可并行化** ◯ **更快的**训练、**更大的**模型、在视觉和语言任务上**更好的**性能 ○ 它们正迅速取代 RNNs、LSTMs，甚至可能取代卷积。



匿名 课堂反馈

Q&A